

파이썬으로 배우는 머신러닝

챕터 1: 머신러닝이란 무엇인가

1장. 머신러닝이란 무엇인가

우리는 이미 일상 속에서 머신러닝(Machine Learning)의 결과물을 매일 접하고 있습니다. 유튜브가 내가 좋아할 만한 영상을 추천하고, 이메일 서비스가 광고 메일을 알아서 '스팸함'으로 보내며, 스마트폰 카메라가 내 얼굴을 인식해 잠금을 해제하는 모든 과정에 머신러닝이 숨어 있습니다.

하지만 정작 "머신러닝이 정확히 무엇인가요?"라는 질문을 받으면 선뜻 대답하기 어렵습니다. 어떤 이는 인공지능이라고 하고, 어떤 이는 데이터 분석이라고 합니다. 이번 장에서는 복잡한 수학 공식 대신, 직관적인 비유를 통해 머신러닝의 정체를 밝혀보겠습니다.

1.1 머신러닝의 정체: 경험으로 배우는 기계

비유: 아이가 사과와 오렌지를 구분하는 법

어린아이에게 사과와 오렌지를 구분하는 법을 가르친다고 생각해보세요. 부모님은 아이에게 다음과 같은 '규칙'을 일일이 설명하지 않습니다. - "지름이 7~10cm 사이이고, 표면이 매끄러우며, 색상이 빨간색이면 사과란다." - "지름이 6~9cm 사이이고, 표면이 울퉁불퉁하며, 색상이 주황색이면 오렌지란다."

대신 부모님은 아이에게 여러 개의 사과와 오렌지를 직접 보여주며 이렇게 말합니다. "이건 사과야", "이건 오렌지야."

아이는 수십 번, 수백 번 사과와 오렌지를 보면서 스스로 깨닫습니다. '아, 대체로 빨간색이면 사과고, 주황색이면 오렌지구나!'라고 말이죠. 나중에 아이가 처음 보는 과일을 보았을 때, 지금까지 보았던 데이터(경험)를 바탕으로 "이건 사과예요!"라고 판단하게 됩니다.

직관: 데이터 속에서 '패턴' 찾기

머신러닝도 이 아이의 학습 과정과 똑같습니다. 사람이 일일이 규칙을 정해주는 것이 아니라, 컴퓨터에게 수많은 데이터를 주고 그 데이터 속에 숨겨진 규칙(패턴)을 스스로 찾아내게 만드는 것입니다.

여기서 핵심은 '학습'입니다. 컴퓨터는 데이터를 통해 "A라는 특징이 있으면 B라는 결과가 나올 확률이 높다"라는 통계적인 규칙을 찾아냅니다. 이 규칙을 우리는 '모델(Model)'이라고 부릅니다.

기술 설명: 머신러닝의 정의

머신러닝(Machine Learning)은 컴퓨터 과학의 한 분야로, 명시적으로 프로그래밍되지 않고도 데이터로부터 학습하여 성능을 향상시킬 수 있는 알고리즘을 연구하는 학문입니다.

전통적인 방식에서는 개발자가 if-then 문법을 사용하여 모든 상황에 대한 규칙을 직접 코딩해야 했습니다. 하지만 데이터가 너무 방대하거나 규칙이 너무 복잡한 경우(예: 고양이 사진 판별) 이는 불가능에 가깝습니다. 머신러닝은 이 문제를 해결하기 위해 **데이터 $\rightarrow$ 학습 $\rightarrow$ 모델 $\rightarrow$ 예측**의 프로세스를 가집니다.

1.2 전통적인 프로그래밍 vs 머신러닝

머신러닝이 기존의 프로그래밍과 결정적으로 다른 점은 '누가 규칙을 만드는가'에 있습니다.

비유: 요리 레시피 vs 맛집 탐방

- 전통적인 프로그래밍은 '완벽한 레시피'와 같습니다. 요리사가 "소금 5g, 설탕 10g을 넣고 5분간 끓이세요"라고 정확한 순서와 양을 정해줍니다. 이 레시피대로만 하면 누가 만들어도 똑같은 맛이 납니다. 하지만 재료의 상태(데이터)가 바뀌면 맛이 변하고, 그에 맞춰 레시피를 매번 수정해야 합니다.
- 머신러닝은 '맛있는 집을 찾아다니는 미식가'와 같습니다. 미식가는 레시피를 보지 않습니다. 대신 수많은 맛집을 다니며 "이 집은 육수가 진하네", "저 집은 면이 쫄깃하네"라는 경험을 쌓습니다. 그러다 보면 어느 순간 '정말 맛있는 국수의 특징'을 스스로 깨닫게 됩니다. 그리고 처음 가보는 식당의 국수만 한 젓가락 먹어봐도 "음, 이 집은 맛집이겠군!"이라고 예측할 수 있게 됩니다.

직관: 입력과 출력의 관계

전통적인 프로그래밍은 [데이터] + [규칙] $\rightarrow$ [결과]의 구조입니다. 반면, 머신러닝은 [데이터] + [결과] $\rightarrow$ [규칙]의 구조입니다.

구분	전통적인 프로그래밍 (Traditional Programming)	머신러닝 (Machine Learning)
핵심 요소	명시적인 규칙 (Explicit Rules)	데이터 (Data)
작동 방식	규칙에 따라 데이터를 처리하여 결과를 냄	데이터와 결과를 통해 규칙을 찾아냄
개발자의 역할	비즈니스 로직과 규칙을 직접 설계	좋은 데이터를 수집하고 학습 알고리즘을 선택
적합한 문제	규칙이 명확하고 단순한 문제 (예: 계산기, 급여 계산)	규칙을 설명하기 어렵고 복잡한 문제 (예: 음성 인식, 이미지 분류)

기술 설명: 함수 관점에서의 해석

수학적으로 표현하면 더 명확해집니다. 우리는 보통 $y = f(x)$ 라는 함수를 배웁니다. 여기서 x 는 입력, y 는 출력, f 는 규칙(함수)입니다.

- **전통적인 프로그래밍:** 개발자가 f 를 정의합니다. 사용자가 x 를 입력하면 컴퓨터는 정의된 f 에 따라 y 를 계산합니다.
- **머신러닝:** 우리는 x (데이터)와 y (정답)를 컴퓨터에게 줍니다. 컴퓨터는 이 둘의 관계를 분석하여 가장 적절한 f (모델)를 스스로 찾아냅니다. 이후 새로운 x 가 들어오면, 찾아낸 f 를 이용해 y 를 예측합니다.

1.3 AI, 머신러닝, 딥러닝의 관계

인공지능, 머신러닝, 딥러닝이라는 용어는 혼용되어 쓰이는 경우가 많지만, 엄밀히 말하면 포함 관계에 있습니다.

비유: 러시아 인형 '마트료시카'

가장 큰 인형 안에 중간 인형이 있고, 그 안에 가장 작은 인형이 들어있는 마트료시카를 상상해보세요.

1. **가장 큰 인형: 인공지능 (Artificial Intelligence)**
2. 인간의 지능을 흉내 내는 모든 기술을 통칭합니다. 아주 단순한 if문 규칙 기반 시스템부터 복잡한 로봇까지 모두 포함됩니다.
3. **중간 인형: 머신러닝 (Machine Learning)**
4. 인공지능을 구현하는 방법 중 하나입니다. 데이터를 통해 스스로 학습하는 통계적 방법론을 의미합니다.
5. **가장 작은 인형: 딥러닝 (Deep Learning)**
6. 머신러닝의 한 종류로, 인간의 뇌 구조를 모방한 '인공신경망(Artificial Neural Networks)'을 깊게(Deep) 쌓아 학습하는 기술입니다.

도식화 설명

이 관계를 벤 다이어그램으로 그리면 다음과 같습니다. [인공지능 [머신러닝 [딥러닝]]]

- **AI:** "기계가 지능적으로 행동하게 만들자!" (가장 넓은 개념)
 - **ML:** "데이터를 통해 스스로 학습하게 하면 지능적으로 행동하겠지?" (학습 중심)
 - **DL:** "인간의 뇌처럼 신경망을 겹겹이 쌓아 학습시키면 더 복잡한 패턴을 찾을 수 있을 거야!" (신경망 중심)
-

1.4 실무 활용 사례: 추천 시스템과 스팸 필터

머신러닝이 실제로 어떻게 작동하는지 가장 대표적인 두 가지 사례를 통해 살펴보겠습니다.

사례 1: 스팸 메일 필터 (Spam Filter)

우리는 매일 수많은 메일을 받지만, 스팸 메일은 자동으로 걸러집니다. 컴퓨터는 어떻게 이것이 스팸인지 알까요?

- **데이터 수집:** 수백만 개의 '정상 메일'과 '스팸 메일' 데이터를 수집합니다.
- **패턴 학습:** 머신러닝 모델은 스팸 메일에 유독 많이 등장하는 단어들을 발견합니다. (예: "광고", "무료", "당첨", "비트코인", "최저가" 등)
- **확률 계산:** 단순히 단어 하나만 보는 것이 아니라, "무료"와 "당첨"이 동시에 등장했을 때 스팸일 확률이 98%라는 식의 통계적 확률을 계산합니다.
- **예측:** 새로운 메일이 도착하면, 학습된 패턴을 바탕으로 "이 메일은 스팸일 확률이 매우 높다"라고 판단하여 스팸함으로 보냅니다.

사례 2: 콘텐츠 추천 시스템 (Recommendation System)

넷플릭스나 유튜브의 추천 알고리즘은 어떻게 내가 좋아할 영상을 기가 막히게 찾아낼까요?

- **사용자 기반 협업 필터링:** "A님과 B님은 그동안 본 영상 목록이 매우 비슷하네요. A님이 최근에 본 '파이썬 강의'를 B님도 좋아하시겠군요!"라고 추천하는 방식입니다.
- **콘텐츠 기반 필터링:** "A님이 '머신러닝' 관련 영상을 많이 보셨네요. 이 영상도 '머신러닝' 태그가 붙어 있으니 추천해 드려야지!"라고 추천하는 방식입니다.
- **결과:** 수억 명의 사용자 데이터와 영상 데이터를 학습하여, 개인별 맞춤형 '취향 지도'를 그려내는 것입니다.

1.5 [실습] 파이썬으로 맛보기: 과일 분류기 만들기

이제 이론을 넘어 실제로 파이썬 코드를 통해 머신러닝이 어떻게 작동하는지 체험해 보겠습니다. 아주 간단한 데이터를 이용해 "무게와 표면 질감을 보고 이것이 사과인지 오렌지인지" 맞히는 모델을 만들어 보겠습니다.

우리는 여기서 scikit-learn이라는 파이썬의 대표적인 머신러닝 라이브러리를 사용합니다.

코드 예제

```
# 필요한 라이브러리 импорт
from sklearn import tree

# 1. 데이터 준비 (특성: [무게(g), 표면 질감])
```

```

# 질감: 0 = 매끄러움(Smooth), 1 = 울퉁불퉁함(Bumpy)
# [무게, 질감]
features = [[140, 0], [130, 0], [150, 1], [170, 1]]

# 2. 정답 데이터 (레이블)
# 0 = 사과(Apple), 1 = 오렌지(Orange)
labels = [0, 0, 1, 1]

# 3. 머신러닝 모델 선택 (결정 트리 분류기)
# 결정 트리(Decision Tree)는 스무고개처럼 질문을 던져 데이터를 분류하는 알고리즘입니다.
classifier = tree.DecisionTreeClassifier()

# 4. 모델 학습 (Fit)
# 데이터(features)와 정답(labels)을 주고 규칙을 찾게 합니다.
classifier.fit(features, labels)

# 5. 새로운 데이터로 예측 (Predict)
# 무게 160g이고 질감이 울퉁불퉁한(1) 과일은 무엇일까요?
new_fruit = [[160, 1]]
prediction = classifier.predict(new_fruit)

# 결과 출력
fruit_name = "사과" if prediction[0] == 0 else "오렌지"
print(f"예측 결과: 이 과일은 [{fruit_name}]입니다.")

```

실행 결과 및 해석

실행 결과:

예측 결과: 이 과일은 [오렌지]입니다.

결과 해석: 1. **데이터 입력:** 우리는 모델에게 [140, 0]은 사과고, [150, 1]은 오렌지라는 데이터를 주었습니다. 2. **학습 과정:** 모델은 데이터를 분석하여 "무게가 일정 수준 이상이고 질감이 1(울퉁불퉁)이면 오렌지일 가능성이 높다"라는 내부 규칙을 스스로 만들었습니다. 3. **예측 과정:** 새로운 데이터 [160, 1]이 들어오자, 모델은 자신이 만든 규칙에 대입해 보았고, 결과적으로 '오렌지'라는 결론을 내린 것입니다.

우리는 `if weight > 145 and texture == 1: return "Orange"` 같은 코드를 한 줄도 작성하지 않았습니다. 단지 데이터를 주었을 뿐이고, **모델이 스스로 규칙을 찾은 것**입니다. 이것이 바로 머신러닝의 핵심입니다.

x

1.6 초보자가 자주 하는 실수와 해결 방법

머신러닝을 처음 접할 때 가장 많이 하는 오해와 실수들을 정리했습니다.

실수 1: "데이터만 많으면 무조건 성능이 좋아진다?"

많은 입문자가 데이터의 '양'에만 집착합니다. 하지만 질 나쁜 데이터(쓰레기 데이터)가 100만 개 있는 것보다, 깨끗하고 정확한 데이터 1,000개가 훨씬 효율적입니다. - **해결 방법:** 데이터의 양보다 '데이터의 품질(Quality)'에 집중하세요. 이를 '데이터 전처리'라고 하며, 머신러닝 작업의 80%를 차지하는 매우 중요한 과정입니다.

실수 2: "머신러닝은 마법의 상자(Black Box)다?"

코드를 실행해 정답이 나오면 그냥 "와, 맞혔다!" 하고 넘어가는 경우가 많습니다. 하지만 왜 그런 결과가 나왔는지 분석하지 않으면 실무에서 치명적인 오류를 범할 수 있습니다. - **해결 방법:** 모델이 어떤 특성(Feature)을 중요하게 생각했는지 확인하는 습관을 들여야 합니다. (예: 위 예제에서 모델이 '무게'를 더 중요하게 봤는지, '질감'을 더 중요하게 봤는지 확인)

실수 3: "모든 문제에 딥러닝이 정답이다?"

최근 딥러닝이 유행하다 보니, 간단한 표 데이터 분석에도 복잡한 신경망 모델을 쓰려는 경향이 있습니다. 이는 마치 동네 슈퍼마켓에 가는데 덤프트럭을 운전해서 가는 것과 같습니다. - **해결 방법:** 문제의 복잡도에 맞는 알고리즘을 선택하세요. 단순한 분류/회귀 문제는 전통적인 머신러닝 알고리즘(Decision Tree, Random Forest 등)이 훨씬 빠르고 정확할 때가 많습니다.

1.7 요약 및 정리

이번 장에서는 머신러닝의 기본 개념과 작동 원리를 살펴보았습니다. 핵심 내용을 요약하면 다음과 같습니다.

1. **머신러닝의 정의:** 명시적인 규칙 없이 데이터로부터 패턴을 학습하여 예측 모델을 만드는 기술입니다.
2. **전통적 프로그래밍 vs 머신러닝:** 전자는 데이터 + 규칙 $\rightarrow$ 결과이며, 후자는 데이터 + 결과 $\rightarrow$ 규칙을 찾는 과정입니다.
3. **AI $\supset$ ML $\supset$ DL:** 인공지능이라는 큰 범주 안에 머신러닝이 있고, 그 안에 신경망 기반의 딥러닝이 존재합니다.
4. **핵심 프로세스:** 데이터 수집 $\rightarrow$ 모델 학습(Fit) $\rightarrow$ 예측(Predict) 순으로 진행됩니다.
5. **주의점:** 데이터의 양보다 품질이 중요하며, 문제에 맞는 적절한 알고리즘을 선택하는 것이 중요합니다.

이제 여러분은 머신러닝이라는 거대한 지도의 입구에 섰습니다. 다음 장부터는 본격적으로 데이터를 어떻게 다루고, 어떤 알고리즘들이 있는지 구체적으로 학습해 보겠습니다.

챕터 2: 파이썬 머신러닝 개발 환경 구축

2장. 파이썬 머신러닝 개발 환경 구축

머신러닝이라는 거대한 여정을 시작하기 전, 가장 먼저 해야 할 일은 내가 편하게 작업할 수 있는 '작업실'을 만드는 것입니다. 도구가 준비되지 않은 상태에서 무작정 코드를 짜는 것은, 망치와 드라이버 없이 가구를 조립하려는 것과 같습니다.

2.1 개발 환경이란 무엇인가?

비유: 요리사의 주방 세팅

최고의 요리사가 되기 위해서는 단순히 레시피를 아는 것만으로는 부족합니다. 재료를 손질할 도마와 칼이 필요하고, 불 조절이 정밀한 가스레인지가 있어야 하며, 완성된 요리를 담을 접시가 준비되어 있어야 합니다.

머신러닝 개발 환경도 이와 같습니다. - **파이썬(Python)**은 요리의 기본이 되는 '식재료'와 '기본 조리법'입니다. - **라이브러리(scikit-learn 등)**는 미리 만들어진 '밀키트'나 '특수 조리 도구'와 같습니다. - **IDE(VS Code, Jupyter Notebook)**는 요리를 효율적으로 할 수 있게 설계된 '최신식 주방'입니다.

직관: 왜 환경 구축이 중요한가?

머신러닝은 일반적인 소프트웨어 개발과 다릅니다. 데이터를 살펴보고, 모델을 수정하고, 결과를 확인하는 '실험'의 반복입니다. 만약 코드를 수정할 때마다 전체 프로그램을 다시 실행해야 한다면 시간이 너무 오래 걸릴 것입니다. 따라서 **실시간으로 결과를 확인하며 수정할 수 있는 환경**을 구축하는 것이 머신러닝 학습의 핵심입니다.

기술 설명: 머신러닝 스택(Stack)의 구성

머신러닝 개발 환경은 크게 세 가지 층으로 구성됩니다.

1. **언어 런타임 (Language Runtime)**: 파이썬 인터프리터. 코드를 해석하고 실행하는 엔진입니다.
 2. **패키지 관리 및 가상 환경 (Package Management)**: 프로젝트마다 필요한 라이브러리 버전이 다를 수 있습니다. 이를 분리하여 관리하는 공간입니다. (예: Anaconda, venv)
 3. **개발 도구 (IDE/Editor)**: 코드를 작성하고 실행하며 시각화 결과를 확인하는 툴입니다. (예: Jupyter Notebook, VS Code)
-

2.2 파이썬 설치와 가상 환경 관리

비유: 나만의 독립된 실험실

한 건물에 여러 개의 실험실이 있다고 상상해 보세요. A 실험실에서는 '화학 실험'을 하고, B 실험실에서는 '생물 실험'을 합니다. 만약 한 공간에서 모든 실험을 다 한다면, 화학 약품이 생물 표본을 오염시킬 수 있습니다.

파이썬의 **가상 환경(Virtual Environment)**은 바로 이 '독립된 실험실'과 같습니다. 어떤 프로젝트에서는 scikit-learn 1.0 버전을 쓰고, 다른 프로젝트에서는 1.2 버전을 써야 할 때, 가상 환경이 없다면 버전 충돌이 일어나 프로그램이 작동하지 않는 '오염' 상태가 됩니다.

직관: 왜 그냥 설치하면 안 되나요?

파이썬을 컴퓨터 전체(Global)에 그냥 설치해서 사용하면, 시간이 흐를수록 수많은 라이브러리가 쌓이게 됩니다. 나중에 특정 라이브러리를 업데이트했을 때, 이전에 잘 작동하던 다른 코드가 갑자기 멈추는 현상이 발생합니다. 이를 방지하기 위해 프로젝트마다 "**이 프로젝트 전용 가방**"을 만드는 것이 가상 환경의 핵심입니다.

기술 설명: 아나콘다(Anaconda) 설치 및 설정

입문자에게는 파이썬, 가상 환경 관리자, 주요 머신러닝 라이브러리를 한 번에 설치해 주는 **아나콘다(Anaconda)** 배포판을 추천합니다.

1. **설치 단계** - [Anaconda 공식 홈페이지](#)에서 OS에 맞는 설치 파일을 다운로드하여 설치합니다. - 설치 과정 중 Add Anaconda to my PATH environment variable 옵션은 가급적 체크하지 말고, 설치 후 Anaconda Prompt를 통해 실행하는 것을 권장합니다.

2. **가상 환경 생성 및 활성화** 아나콘다 프롬프트(또는 터미널)에서 다음 명령어를 입력하여 머신러닝 전용 환경을 만듭니다.

```
# 1. 'ml_env'라는 이름의 가상 환경 생성 (파이썬 3.9 버전 지정)
conda create -n ml_env python=3.9

# 2. 생성한 가상 환경 활성화
conda activate ml_env
```

3. **환경 확인** 현재 어떤 환경에 접속해 있는지 확인하려면 프롬프트 왼쪽의 괄호 (ml_env)를 확인하면 됩니다.

주의: 초보자가 자주 하는 실수 - 실수: 가상 환경을 만들고 나서 activate를 하지 않은 채 라이브러리를 설치하는 경우. - **결과:** 라이브러리가 'base' 환경(기본 환경)에 설치되어, 나중에 환경을 분리했을 때 라이브러리를 찾을 수 없다는 `ModuleNotFoundError`가 발생합니다. - **해결:** 항상 `conda activate` 환경이름을 먼저 입력하는 습관을 들이세요.

2.3 인터랙티브 개발 도구: Jupyter Notebook

비유: 메모가 가능한 연습장

일반적인 프로그래밍이 '완성된 논문을 쓰는 과정'이라면, 주피터 노트북은 '여백에 메모를 하며 문제를 푸는 연습장'과 같습니다. 한 페이지 전체를 다시 쓸 필요 없이, 틀린 부분의 한 줄만 지우고 다시 써서 결과를 바로 확인할 수 있습니다.

직관: 왜 머신러닝에서는 노트북을 쓰나요?

머신러닝은 데이터 분석 단계가 매우 깁니다. 데이터 불러오기 $\rightarrow$ 결측치 확인 $\rightarrow$ 그래프 그리기 $\rightarrow$ 전처리 $\rightarrow$ 모델 학습 만약 일반 .py 파일로 작성한다면, 그래프 하나를 수정하기 위해 매번 수 기가바이트의 데이터를 다시 불러와야 합니다. 하지만 주피터 노트북은 '셀(Cell)' 단위로 실행되므로, 데이터는 메모리에 올려둔 채 분석 코드만 수정하여 실행할 수 있습니다.

기술 설명: 설치 및 사용법

1. 설치 가상 환경이 활성화된 상태에서 다음 명령어를 입력합니다.

```
conda install jupyter
```

2. 실행

```
jupyter notebook
```

명령어를 입력하면 웹 브라우저가 열리며 주피터 노트북 대시보드가 나타납니다.

3. 핵심 기능 - **Code Cell**: 파이썬 코드를 작성하고 Shift + Enter로 실행합니다. - **Markdown Cell**: 설명글, 수식, 이미지를 넣을 수 있는 문서 영역입니다. - **Kernel**: 코드를 실제로 실행하는 엔진입니다. 노트북이 멈췄다면 Kernel $\rightarrow$ Restart를 통해 초기화할 수 있습니다.

[그림: 주피터 노트북 구조도] - [셀 1: 데이터 로드] $\rightarrow$ (결과: 데이터프레임 출력) - [셀 2: 데이터 분석] $\rightarrow$ (결과: 시각화 그래프 출력) - [셀 3: 모델 학습] $\rightarrow$ (결과: 정확도 95% 출력) - 각 셀은 독립적으로 실행 가능하며, 이전 셀에서 정의한 변수를 공유함.

2.4 전문 개발 도구: VS Code (Visual Studio Code)

비유: 다기능 맥가이버 칼

주피터 노트북이 '연습장'이라면, VS Code는 '정밀 가공 도구'입니다. 단순한 메모를 넘어, 대규모 프로젝트를 관리하고, 코드의 오류를 자동으로 찾아주며, 서버에 배포하기 위한 최적의 환경을 제공합니다.

직관: 연습장에서 제품으로

학습 단계에서는 주피터 노트북이 편하지만, 실제로 서비스에 적용할 모델을 만들 때는 .py 형태의 스크립트 파일이 필요합니다. VS Code는 주피터 노트북의 편리함(확장 프로그램 설치 시 .ipynb 파일 지원)과 전문 개발 도구의 강력함을 동시에 제공합니다.

기술 설명: 설치 및 파이썬 연동

1. 설치 및 확장 프로그램 설치 - [VS Code 공식 홈페이지](#)에서 설치합니다. - 왼쪽 사이드바의 Extensions(Ctrl+Shift+X) 아이콘을 클릭하고 다음 항목을 설치합니다. - Python (Microsoft 제공) - Jupyter (Microsoft 제공)
2. 인터프리터 설정 (매우 중요) VS Code가 우리가 만든 가상 환경(ml_env)을 사용하도록 알려줘야 합니다. 1. Ctrl + Shift + P 를 누릅니다. 2. Python: Select Interpreter를 검색하여 선택합니다. 3. 리스트에서 Python 3.9.x ('ml_env': conda)를 선택합니다.

이렇게 설정해야 VS Code 하단 상태 표시줄에 현재 사용 중인 환경이 표시되며, 라이브러리 인식 오류가 발생하지 않습니다.

2.5 머신러닝 필수 라이브러리 설치 (scikit-learn & Friends)

비유: 전문가용 툴킷 구매

우리가 집을 지을 때 모든 못과 망치를 직접 제련해서 만들지는 않습니다. 이미 검증된 브랜드의 툴킷(Tool Kit)을 구매해서 사용하죠. 머신러닝 라이브러리들이 바로 이 툴킷입니다.

직관: 바퀴를 다시 발명하지 마라

머신러닝의 수학적 원리를 아는 것은 중요하지만, 모든 알고리즘을 밑바닥부터 코딩하는 것은 매우 비효율적입니다. 전 세계 수만 명의 전문가가 최적화해 놓은 scikit-learn 같은 라이브러리를 사용하면, 우리는 '어떤 알고리즘을 어디에 적용할 것인가'라는 본질적인 문제에 집중할 수 있습니다.

기술 설명: 핵심 라이브러리 설치 및 역할

머신러닝의 '기본 4인방'을 설치하겠습니다. 가상 환경이 활성화된 상태에서 입력하세요.

```
conda install numpy pandas matplotlib scikit-learn
```

라이브러리	역할	비유
NumPy	고성능 수치 계산, 다차원 배열 처리	계산기 및 행렬 계산판
Pandas	표(Table) 형태의 데이터 조작 및 분석	엑셀(Excel)의 파이썬 버전
Matplotlib	데이터 시각화 (그래프, 차트)	도표를 그리는 스케치북
scikit-learn	머신러닝 알고리즘 구현체	머신러닝 종합 도구함

2.6 환경 구축 확인 및 첫 실행

이제 모든 도구가 준비되었습니다. 실제로 설치가 잘 되었는지, 그리고 이 도구들이 어떻게 상호작용하는지 간단한 코드로 확인해 보겠습니다.

실습: 머신러닝 환경 검증 코드

이 코드는 scikit-learn에 내장된 붓꽃(Iris) 데이터를 불러와 데이터의 구조를 확인하는 간단한 예제입니다. 주피터 노트북이나 VS Code의 .py 파일에서 실행하세요.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# 1. 데이터 불러오기
iris = load_iris()

# 2. Pandas 데이터프레임으로 변환 (표 형태로 만들기)
# iris.data는 수치 데이터, iris.feature_names는 컬럼 이름입니다.
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target # 정답(품종) 추가

# 3. 상위 5개 데이터 확인
print("--- 붓꽃 데이터셋 상위 5행 ---")
print(df.head())

# 4. 간단한 시각화 (꽃받침 길이 vs 너비)
plt.figure(figsize=(8, 6))
plt.scatter(df.iloc[:, 0], df.iloc[:, 1], c=df['target'])
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
```

```
plt.title('Iris Dataset Visualization')
plt.colorbar(label='Species')
plt.show()

print("\n환경 구축 및 라이브러리 작동 확인 완료!")
```

실행 결과 해석

1. **텍스트 출력:** `df.head()`를 통해 붓꽃의 꽃받침(sepal)과 꽃잎(petal)의 길이/너비 데이터가 표 형태로 출력됩니다. 이 데이터가 정상적으로 보인다면 Pandas와 scikit-learn이 잘 설치된 것입니다.
 2. **그래프 출력:** 산점도(Scatter plot) 그래프가 나타나며, 품종별로 색상이 다르게 표시됩니다. 이 그래프가 보인다면 Matplotlib과 NumPy가 정상 작동하는 것입니다.
-

2.7 요약 및 체크리스트

이번 장에서는 머신러닝이라는 요리를 하기 위해 최적의 주방을 세팅했습니다. 파이썬이라는 기본 재료를 설치하고, 가상 환경이라는 독립된 조리 공간을 만들었으며, 주피터 노트북과 VS Code라는 최신식 도구를 갖추었습니다.

최종 체크리스트

- ☐ Anaconda가 설치되어 있는가?
- ☐ `conda create`를 통해 독립된 가상 환경을 생성했는가?
- ☐ `conda activate`로 해당 환경에 접속했는가?
- ☐ Jupyter Notebook과 VS Code를 설치하고 실행해 보았는가?
- ☐ VS Code에서 Python 인터프리터를 생성한 가상 환경으로 지정했는가?
- ☐ `numpy`, `pandas`, `matplotlib`, `scikit-learn` 라이브러리를 설치했는가?
- ☐ 제공된 검증 코드를 실행하여 그래프와 표가 정상적으로 출력되는가?

이제 당신의 작업실은 완벽하게 준비되었습니다. 다음 장부터는 이 도구들을 사용하여 본격적으로 데이터를 만지고 머신러닝 모델을 학습시키는 방법을 배워보겠습니다.

챕터 3: NumPy와 Pandas

3장. NumPy와 Pandas: 머신러닝을 위한 데이터 요 리법

머신러닝 모델을 만드는 과정은 흔히 요리에 비유됩니다. 훌륭한 요리를 만들기 위해서는 좋은 재료가 필요하고, 그 재료를 깨끗이 씻고 다듬는 '전처리' 과정이 필수적입니다. 머신러닝에서 '재료'는 바로 데이터이며, 이 데이터를 다루는 가장 강력한 도구가 바로 NumPy와 Pandas입니다.

파이썬의 기본 리스트(List)만으로도 데이터를 다룰 수는 있지만, 수만 개의 데이터를 처리해야 하는 머신러닝 환경에서는 속도가 너무 느리고 불편합니다. 이번 장에서는 데이터를 효율적으로 저장하고 계산하는 NumPy와, 표 형태의 데이터를 자유자재로 다루는 Pandas의 핵심 기능을 살펴보겠습니다.

1. NumPy: 수치 계산의 초석

1.1 NumPy란 무엇인가?

[비유: 뒤섞인 장난감 상자 vs 정리된 계란판] 파이썬의 기본 리스트는 '장난감 상자'와 같습니다. 인형, 자동차, 블록 등 서로 다른 종류의 물건을 한꺼번에 넣을 수 있어 편리하지만, 정작 특정 물건을 빠르게 찾거나 전체를 한꺼번에 옮기기에는 효율이 떨어집니다.

반면, NumPy의 배열(Array)은 '계란판'과 같습니다. 모든 칸에 동일한 크기와 종류의 계란(데이터 타입)만 들어갈 수 있습니다. 제약이 있는 대신, 위치가 정확히 정해져 있어 수천 개의 데이터를 한꺼번에 계산할 때 압도적으로 빠릅니다.

[그림: 계란판에 데이터가 정렬된 모습 - 각 칸에 동일한 타입의 숫자가 규칙적으로 배치되어 있고, 인덱스로 빠르게 접근하는 모습]

[직관] 머신러닝은 결국 거대한 숫자들의 계산입니다. 이미지 한 장은 수만 개의 픽셀 값(숫자)으로 이루어져 있고, 텍스트 데이터 역시 숫자로 변환되어 처리됩니다. NumPy는 이러한 대량의 숫자 데이터를 행렬(Matrix) 형태로 처리하여 계산 속도를 극대화합니다.

[기술 설명] NumPy(Numerical Python)는 파이썬의 과학 계산 라이브러리로, 핵심 객체인 ndarray(n-dimensional array)를 제공합니다. ndarray는 동일한 자료형의 데이터를 메모리에 연속적으로 배치하여, 파이썬 리스트보다 메모리 사용량이 적고 연산 속도가 훨씬 빠릅니다.

NumPy 기본 실습: 배열 생성과 연산

```
import numpy as np
```

```

# 1. 리스트를 NumPy 배열로 변환
data_list = [1, 2, 3, 4, 5]
arr = np.array(data_list)

# 2. 다양한 배열 생성 방법
zeros = np.zeros((2, 3))      # 2행 3열의 0으로 채워진 배열
ones = np.ones((2, 3))       # 2행 3열의 1으로 채워진 배열
arange_arr = np.arange(0, 10, 2) # 0부터 10 미만까지 2씩 증가

print("기본 배열:\n", arr)
print("\n0으로 채워진 배열:\n", zeros)
print("\n범위 배열:", arange_arr)

# 3. 벡터 연산 (Element-wise operation)
# 리스트였다면 for문을 돌려야 했지만, NumPy는 한 번에 계산합니다.
arr_plus_10 = arr + 10
arr_mul_2 = arr * 2

print("\n모든 요소에 10 더하기:", arr_plus_10)
print("모든 요소에 2 곱하기:", arr_mul_2)

```

[실행 결과]

```

기본 배열:
[1 2 3 4 5]

0으로 채워진 배열:
[[0. 0. 0.]
 [0. 0. 0.]]

범위 배열: [0 2 4 6 8]

모든 요소에 10 더하기: [11 12 13 14 15]
모든 요소에 2 곱하기: [2 4 6 8 10]

```

[결과 해석] - np.array()를 통해 파이썬 리스트를 NumPy 배열로 변환했습니다. - 가장 주목할 점은 arr + 10 부분입니다. 파이썬 리스트에 10을 더하면 오류가 발생하거나 리스트가 확장되지만, NumPy 배열은 **브로드캐스팅(Broadcasting)**이라는 기능을 통해 배열 내의 모든 요소에 동일한 연산을 한 번에 적용합니다. 이것이 머신러닝에서 NumPy를 사용하는 핵심 이유입니다.

1.2 배열의 모양 바꾸기와 슬라이싱

머신러닝 모델에 데이터를 넣을 때 가장 많이 마주치는 에러가 바로 "데이터의 모양(Shape)이 맞지 않는다"는 것입니다. 따라서 배열의 형태를 자유자재로 바꾸는 능력이 필요합니다.

[비유: 찰흙 덩어리 모양 만들기] 12개의 찰흙 공이 일렬로 늘어서 있다고 생각해보세요. 이를 3x4 직사각형 모양으로 배치할 수도 있고, 2x6 모양으로 배치할 수도 있습니다. 전체 개수는 변하지 않지만, 배치 방식(Shape)에 따라 사용 용도가 달라집니다.

[그림: 1차원 배열이 2차원으로 변형되는 과정 - 1x12 형태의 긴 막대 모양 배열이 3x4 형태의 표 모양으로 재배치되는 모습]

NumPy 실습: Shape 조절과 인덱싱

```
# 1. 1차원 배열 생성 (요소 12개)
arr_1d = np.arange(12)
print("1차원 배열:", arr_1d)
print("Shape:", arr_1d.shape)

# 2. 2차원으로 모양 변경 (Reshape)
arr_2d = arr_1d.reshape(3, 4)
print("\n3x4 배열로 변환:\n", arr_2d)
print("Shape:", arr_2d.shape)

# 3. 특정 데이터 추출 (슬라이싱)
# [행, 열] 순서로 지정합니다.
print("\n첫 번째 행 전체:", arr_2d[0, :])
print("두 번째 열 전체:", arr_2d[:, 1])
print("1행 2열의 값:", arr_2d[1, 2])
```

[실행 결과]

```
1차원 배열: [ 0  1  2  3  4  5  6  7  8  9 10 11]
Shape: (12,)

3x4 배열로 변환:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Shape: (3, 4)

첫 번째 행 전체: [0 1 2 3]
```

두 번째 열 전체: [1 5 9]

1행 2열의 값: 6

[결과 해석] - reshape() 함수를 통해 데이터의 총 개수를 유지한 채 차원을 변경했습니다. 머신러닝 모델(예: 딥러닝)은 입력 데이터의 차원을 엄격하게 따지기 때문에 이 작업이 매우 빈번하게 일어납니다. - arr_2d[:, 1]과 같이 : 기호를 사용하면 "해당 축의 모든 요소"를 가져오겠다는 의미입니다. 즉, 모든 행의 1번 인덱스 열만 가져오게 됩니다.

2. Pandas: 데이터 분석의 만능 도구

2.1 Pandas란 무엇인가?

[비유: 파이썬 속에 들어온 엑셀(Excel)] NumPy가 순수한 숫자들의 집합이라면, Pandas는 여기에 '이름'과 '라벨'을 붙인 것입니다. 엑셀 시트를 떠올려 보세요. 맨 위에는 '이름', '나이', '거주지' 같은 열 이름(Column Name)이 있고, 왼쪽에는 1, 2, 3... 같은 행 번호(Index)가 있습니다. Pandas는 이 엑셀과 같은 구조를 파이썬 코드만으로 제어할 수 있게 해줍니다.

[그림: DataFrame의 구조 - 행(Index)과 열(Column)의 명칭이 표시된 표 형태의 도식]

[직관] 실제 머신러닝 데이터는 숫자만 있는 것이 아니라, 날짜, 텍스트, 범주형 데이터 등이 섞여 있습니다. 이를 효율적으로 관리하기 위해 Pandas는 표 형태의 구조인 **DataFrame**을 제공합니다.

[기술 설명] Pandas의 핵심 구조는 두 가지입니다. 1. **Series**: 1차원 배열 형태의 데이터 구조 (열 하나). 2. **DataFrame**: 여러 개의 Series가 모여 만들어진 2차원 표 형태의 데이터 구조.

Pandas 실습 1: Series 생성과 확인

DataFrame을 배우기 전, 그 기본 단위인 Series를 먼저 살펴보겠습니다. Series는 이름표(Index)가 붙은 1차원 리스트라고 생각하면 쉽습니다.

```
import pandas as pd

# Series 생성 (데이터와 인덱스를 직접 지정)
s = pd.Series([85, 90, 78, 92], index=['Alice', 'Bob', 'Charlie', 'David'])
print("생성된 Series:\n", s)

# 인덱스를 이용한 데이터 접근
print("\nBob의 점수:", s['Bob'])
```

[실행 결과]

```
생성된 Series:
Alice      85
```

```
Bob      90
Charlie  78
David    92
dtype: int64
```

Bob의 점수: 90

[결과 해석] - NumPy 배열과 달리, 각 데이터에 'Alice', 'Bob'과 같은 고유한 이름(Index)을 붙일 수 있습니다. 이를 통해 숫자가 아닌 의미 있는 라벨로 데이터를 관리할 수 있게 됩니다.

Pandas 실습 2: DataFrame 생성과 조회

이제 여러 개의 Series가 합쳐진 형태인 DataFrame을 만들어 보겠습니다.

```
# 1. 딕셔너리를 이용한 DataFrame 생성
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['Seoul', 'Busan', 'Incheon', 'Daegu'],
    'Score': [85, 90, 78, 92]
}

df = pd.DataFrame(data)
print("생성된 DataFrame:\n", df)

# 2. 데이터 확인하기
print("\n상위 2개 행 확인:\n", df.head(2))
print("\n데이터 요약 정보:\n", df.info())
print("\n수치형 데이터 기초 통계:\n", df.describe())
```

[실행 결과]

생성된 DataFrame:

	Name	Age	City	Score
0	Alice	25	Seoul	85
1	Bob	30	Busan	90
2	Charlie	35	Incheon	78
3	David	40	Daegu	92

상위 2개 행 확인:

	Name	Age	City	Score
0	Alice	25	Seoul	85
1	Bob	30	Busan	90

데이터 요약 정보:

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 4 entries, 0 to 3  
Data columns (total 4 columns):  
... (생략) ...
```

수치형 데이터 기초 통계:

	Age	Score
count	4.000000	4.000000
mean	32.500000	86.250000
std	6.454972	5.767481
min	25.000000	78.000000
max	40.000000	92.000000

[결과 해석] - `pd.DataFrame()`을 통해 표 형태의 데이터를 만들었습니다. - `head()`는 데이터가 너무 많을 때 상위 일부만 확인하는 용도로 사용합니다. - `describe()`는 머신러닝 전처리 단계에서 매우 중요합니다. 평균(mean), 표준편차(std), 최솟값(min), 최댓값(max) 등을 한눈에 보여주어 데이터의 분포를 빠르게 파악하게 해줍니다.

2.2 데이터 필터링과 가공

머신러닝의 핵심은 "필요한 데이터만 골라내어 학습시키는 것"입니다. Pandas의 강력한 필터링 기능을 활용해 보겠습니다.

[비유: 거름망으로 알갱이 골라내기] 수많은 모래와 자갈이 섞인 통에서 특정 크기 이상의 자갈만 골라내고 싶을 때 거름망을 사용합니다. Pandas의 **불리언 인덱싱(Boolean Indexing)**이 바로 이 거름망 역할을 합니다.

[그림: 전체 데이터셋에서 'Age >= 30'이라는 거름망을 통과하여 조건에 맞는 행들만 아래로 추출되는 모습]

Pandas 실습: 조건부 필터링과 열 조작

```
# 1. 조건에 맞는 데이터 필터링 (Age가 30세 이상인 사람만)  
over_30 = df[df['Age'] >= 30]  
print("30세 이상 사용자:\n", over_30)  
  
# 2. 새로운 열 추가 (합격 여부 판단)  
# Score가 80점 이상이면 'Pass', 아니면 'Fail'  
df['Result'] = df['Score'].apply(lambda x: 'Pass' if x >= 80 else 'Fail')  
print("\n결과 열이 추가된 DataFrame:\n", df)
```

```
# 3. 특정 열만 선택하여 추출
subset = df[['Name', 'Result']]
print("\n이름과 결과만 추출:\n", subset)
```

[실행 결과]

30세 이상 사용자:

	Name	Age	City	Score
1	Bob	30	Busan	90
2	Charlie	35	Incheon	78
3	David	40	Daegu	92

결과 열이 추가된 DataFrame:

	Name	Age	City	Score	Result
0	Alice	25	Seoul	85	Pass
1	Bob	30	Busan	90	Pass
2	Charlie	35	Incheon	78	Fail
3	David	40	Daegu	92	Pass

이름과 결과만 추출:

	Name	Result
0	Alice	Pass
1	Bob	Pass
2	Charlie	Fail
3	David	Pass

[결과 해석] - `df[df['Age'] >= 30]` 구문은 `df['Age'] >= 30`이라는 조건문이 True인 행만 남기는 방식입니다. - `.apply(lambda x: ...)`를 사용하면 열의 모든 값에 특정 함수를 적용하여 새로운 데이터를 생성할 수 있습니다. 이는 머신러닝에서 데이터를 수치화(Encoding)할 때 매우 유용하게 쓰입니다.

3. 실무 활용 사례: CSV 파일 분석하기

실무에서는 데이터를 직접 입력하지 않고 .csv (Comma Separated Values) 파일 형태로 제공받습니다. Pandas는 이를 읽어 들이는 최적의 기능을 갖추고 있습니다.

[시나리오] 당신은 부동산 가격 예측 모델을 만들려고 합니다. `house_data.csv`라는 파일에 집의 크기, 방 개수, 가격 데이터가 들어있다고 가정해 봅시다.

```
# (실습을 위해 임시 CSV 파일을 생성합니다)
import pandas as pd
import numpy as np
```



```

# 임시 데이터 생성 및 저장
raw_data = {
    'Size': [50, 80, 120, 60, 150],
    'Rooms': [1, 2, 3, 2, 4],
    'Price': [5000, 8000, 12000, 6500, 16000]
}
temp_df = pd.DataFrame(raw_data)
temp_df.to_csv('house_data.csv', index=False)

# --- 여기서부터 실제 분석 과정 ---

# 1. CSV 파일 읽기
df_house = pd.read_csv('house_data.csv')

# 2. 데이터 탐색: 평당 가격(Price per Size) 계산 열 추가
df_house['Price_per_Size'] = df_house['Price'] / df_house['Size']

# 3. 평균 평당 가격보다 비싼 집만 추출
avg_price_per_size = df_house['Price_per_Size'].mean()
expensive_houses = df_house[df_house['Price_per_Size'] > avg_price_per_size]

print(f"평균 평당 가격: {avg_price_per_size:.2f}")
print("\n평균보다 비싼 집 목록:\n", expensive_houses)

```

[실행 결과]

평균 평당 가격: 103.00

평균보다 비싼 집 목록:

	Size	Rooms	Price	Price_per_Size
3	60	2	6500	108.333333
4	150	4	16000	106.666667

[해석] 실무에서 Pandas는 단순히 데이터를 보는 도구가 아니라, **특성 공학(Feature Engineering)**의 도구입니다. 위 예제에서 Price_per_Size라는 새로운 열을 만든 것처럼, 기존 데이터를 조합해 모델이 학습하기 좋은 새로운 지표를 만들어내는 과정이 머신러닝 성능 향상의 핵심입니다.

4. 초보자가 자주 하는 실수와 해결 방법

실수 1: NumPy 배열과 파이썬 리스트의 연산 혼동

x

- **현상:** 리스트에 숫자를 더하려다 `TypeError`가 발생하거나, `+` 연산자로 리스트가 합쳐지는 현상.
- **해결:** 수치 계산이 필요하다면 반드시 `np.array()`로 변환 후 연산하세요.

실수 2: DataFrame 슬라이싱 시 `SettingWithCopyWarning` 발생

x

- **현상:** `df[df['Age'] > 30]['Score'] = 100` 처럼 필터링 후 바로 값을 수정하려 할 때 경고가 발생합니다.
- **해결:** `.loc`를 사용하세요. `df.loc[df['Age'] > 30, 'Score'] = 100`으로 작성하면 안전하게 값이 수정됩니다.

실수 3: 데이터 타입 불일치

- **현상:** 숫자가 들어있어야 할 열이 문자열(object) 타입으로 인식되어 계산이 안 되는 경우.
- **해결:** `df.info()`로 타입을 확인하고, `df['column'].astype(float)`를 통해 타입을 강제로 변환하세요.

5. 요약 및 마무리

이번 장에서는 머신러닝의 기초 체력이라고 할 수 있는 NumPy와 Pandas를 학습했습니다.

도구	핵심 개념	비유	주 용도
NumPy	ndarray, Vectorization	계란판	고속 수치 계산, 행렬 연산
Pandas	DataFrame, Series	엑셀 시트	데이터 전처리, 분석, 필터링

핵심 흐름 요약: 1. NumPy를 통해 데이터를 효율적인 행렬 구조로 만들고, 브로드캐스팅을 통해 빠르게 계산합니다. 2. Pandas를 통해 실제 데이터 파일(CSV 등)을 읽어오고, 표 형태로 관리하며, 필요한 조건으로 필터링합니다. 3. 전처리 과정을 통해 모델이 학습하기 좋은 형태로 데이터를 가공합니다.

이제 여러분은 데이터를 다루는 '요리 도구'를 갖추게 되었습니다. 다음 장에서는 이렇게 준비된 데이터를 가지고 실제로 머신러닝 모델이 어떻게 학습하는지, 그 원리를 살펴보겠습니다.

챕터 4: 데이터 시각화

4장. 데이터 시각화

4.1 왜 데이터를 시각화해야 하는가?

숫자의 숲에서 지도를 그리는 일

상상해 보세요. 당신에게 1만 명의 키와 몸무게가 적힌 엑셀 파일이 주어졌습니다. 이 데이터에서 "키가 클수록 몸무게가 많이 나가는 경향이 있는가?"라는 질문에 답하려면 어떻게 해야 할까요?

데이터를 한 줄씩 읽으며 머릿속으로 계산할 수도 있겠지만, 이는 매우 고통스럽고 비효율적인 일입니다. 하지만 이 데이터를 좌표평면 위에 점으로 찍어 하나의 '그림'으로 만든다면 어떨까요? 단 1초 만에 점들이 오른쪽 위로 향하는 대각선 모양을 이루고 있는 것을 발견할 수 있을 것입니다.

데이터 시각화는 바로 이처럼 '숫자의 숲'에서 길을 잃지 않게 해주는 '지도'를 그리는 작업입니다.

직관: 데이터의 '모양'을 보는 것

머신러닝 모델을 만들기 전에 우리는 데이터가 어떻게 생겼는지 알아야 합니다. 이를 **탐색적 데이터 분석(EDA, Exploratory Data Analysis)**이라고 합니다.

데이터 시각화가 필요한 이유는 크게 세 가지입니다. 1. **패턴 발견**: 변수 간의 상관관계(예: 공부 시간이 늘면 성적이 오르는가?)를 즉각적으로 파악할 수 있습니다. 2. **이상치(Outlier) 발견**: 말도 안 되게 크거나 작은 값(예: 키가 3m인 사람)을 쉽게 찾아내어 제거할 수 있습니다. 3. **분포 확인**: 데이터가 한곳에 몰려 있는지, 골고루 퍼져 있는지 확인하여 적절한 머신러닝 알고리즘을 선택할 수 있습니다.

기술적 설명: Matplotlib과 Seaborn

파이썬 생태계에서 데이터 시각화를 담당하는 가장 대표적인 라이브러리는 **Matplotlib**과 **Seaborn**입니다.

- **Matplotlib**: 파이썬 시각화의 가장 기본이 되는 라이브러리입니다. 도화지에 선을 긋고 점을 찍는 것처럼 세밀한 조작이 가능하지만, 코드가 다소 길고 디자인이 투박한 편입니다.
- **Seaborn**: Matplotlib을 기반으로 만들어진 고수준 라이브러리입니다. 통계적인 그래픽을 그리는 데 최적화되어 있으며, 훨씬 적은 코드로 아름답고 복잡한 그래프를 그릴 수 있습니다.

쉽게 비유하자면, **Matplotlib**은 '붓과 물감'이고, **Seaborn**은 '전문가용 템플릿'과 같습니다. 붓으로 하나하나 그릴 수도 있지만, 템플릿을 사용하면 훨씬 빠르게 멋진 작품을 완성할 수 있죠. 다만, 템플릿의 세부 설정을 바꾸려면 결국 붓(Matplotlib)을 사용해야 합니다.

4.2 Matplotlib으로 기초 그리기

도화지 준비하기

Matplotlib의 기본 구조는 Figure(전체 도화지)와 Axes(그림이 그려지는 축)로 나뉩니다. 하나의 도화지에 여러 개의 그래프를 넣을 수 있기 때문에 이 구분을 이해하는 것이 중요합니다.

[그림: Figure와 Axes의 관계 도식 - 전체 외곽 틀인 Figure 안에 하나 또는 여러 개의 그래프 영역인 Axes가 배치된 구조도]

[실습 1] 가장 단순한 선 그래프 그리기

비유: 시간의 흐름을 따라가는 실타래 선 그래프는 마치 실타래를 풀며 길을 따라가는 것과 같습니다. 시작점에서 끝점까지 끊기지 않고 이어지는 선을 통해 흐름을 파악합니다.

직관: 변화의 추세 파악 데이터가 시간에 따라 증가하는지, 감소하는지, 혹은 특정 주기성을 가지고 움직이는지 확인하고 싶을 때 사용합니다. "어제보다 오늘 값이 올랐는가?"라는 질문에 가장 빠르게 답해주는 그래프입니다.

기술 설명: `plt.plot()` `plt.plot()` 함수를 사용하여 x축과 y축의 데이터를 연결하는 선을 그립니다. `marker`로 데이터 포인트를 강조하고, `linestyle`로 선의 모양을 결정할 수 있습니다.

```
import matplotlib.pyplot as plt
import numpy as np

# 1. 데이터 준비
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 3, 5, 7, 11])

# 2. 그래프 생성
plt.figure(figsize=(8, 5)) # 도화지 크기 설정 (가로, 세로)
plt.plot(x, y, color='blue', marker='o', linestyle='--', linewidth=2, label='Growth')

# 3. 부가 정보 추가
plt.title('Simple Line Plot', fontsize=15)
plt.xlabel('Time (Days)', fontsize=12)
plt.ylabel('Value', fontsize=12)
plt.legend() # 범례 표시
plt.grid(True) # 격자 표시

# 4. 화면에 출력
plt.show()
```

[실행 결과 해석] - 가로축(x)은 시간, 세로축(y)은 값을 나타냅니다. - marker='o'를 통해 데이터 포인트가 어디에 위치하는지 명확히 볼 수 있으며, 점들이 우상향하는 직선 형태를 띠고 있어 "시간이 흐를수록 값이 증가한다"는 추세를 알 수 있습니다.

데이터의 분포를 보는 산점도 (Scatter Plot)

비유: 밤하늘의 별자리 찾기 산점도는 밤하늘에 흩뿌려진 별들을 보는 것과 같습니다. 별들이 무작위로 흩어져 있는지, 아니면 어떤 특정한 모양(선이나 곡선)을 이루며 모여 있는지 찾는 과정입니다.

직관: 두 변수 사이의 관계(상관관계) 파악 "키가 크면 몸무게도 많이 나갈까?"처럼 두 가지 서로 다른 변수가 서로 어떤 영향을 주고받는지 확인하고 싶을 때 사용합니다. 점들이 일정한 방향성을 띠고 있다면 두 변수 사이에 '상관관계'가 있다고 말합니다.

기술 설명: `plt.scatter()` `plt.scatter()` 함수는 각 데이터 포인트를 점으로 찍습니다. `alpha` 파라미터를 통해 점의 투명도를 조절하여 점들이 겹쳐 있는 밀도를 확인할 수 있습니다.

[실습 2] 키와 몸무게의 관계 시각화하기

```
# 데이터 생성 (랜덤하게 생성)
np.random.seed(42)
height = np.random.normal(170, 5, 100) # 평균 170, 표준편차 5인 키 100명
weight = height * 0.5 + np.random.normal(0, 5, 100) # 키에 비례하는 몸무게 + 노이즈

plt.figure(figsize=(8, 6))
plt.scatter(height, weight, alpha=0.6, c='green', edgecolors='w')

plt.title('Height vs Weight', fontsize=15)
plt.xlabel('Height (cm)')
plt.ylabel('Weight (kg)')
plt.grid(True)
plt.show()
```

[실행 결과 해석] - 점들이 오른쪽 위로 향하는 경향성을 보입니다. 이는 **양의 상관관계**가 있음을 의미합니다. - 만약 점 하나가 다른 점들과 아주 멀리 떨어져 있다면, 그 데이터는 '이상치'일 가능성이 높습니다.

데이터의 빈도를 보는 히스토그램 (Histogram)

비유: 키 순서대로 세워놓고 층별로 인원수 세기 학생들을 키 순서대로 세운 뒤, "160~165cm 사이에는 몇 명이지?"라고 구간을 나누어 인원수를 세어 막대기로 쌓아 올리는 것과 같습니다.

직관: 데이터의 분포와 밀집도 확인 전체 데이터가 어디에 가장 많이 몰려 있는지, 혹은 양 끝으로 넓게 퍼져 있는지 확인하고 싶을 때 사용합니다. 이를 통해 데이터의 '분포 모양(정규분포, 편향된 분포 등)'을 알 수 있습니다.

기술 설명: plt.hist() plt.hist() 함수를 사용하며, 가장 중요한 파라미터는 bins입니다. bins는 데이터를 몇 개의 구간으로 나눌지를 결정하며, 이 값에 따라 그래프의 세밀함이 달라집니다.

[실습 3] 성적 분포 확인하기

```
scores = np.random.randint(0, 101, 500) # 0~100점 사이의 무작위 점수 500개

plt.figure(figsize=(8, 5))
plt.hist(scores, bins=20, color='skyblue', edgecolor='black')

plt.title('Score Distribution', fontsize=15)
plt.xlabel('Score')
plt.ylabel('Frequency')
plt.show()
```

[실행 결과 해석] - bins=20은 데이터를 20개의 구간으로 나누어 세겠다는 뜻입니다. - 막대의 높이가 높을수록 해당 점수대에 많은 사람이 분포하고 있음을 나타냅니다. 이를 통해 데이터가 '정규분포'를 따르는지, 혹은 특정 구간에 쏠려 있는지 확인할 수 있습니다.

4.3 Seaborn으로 심화 분석하기

이제 '전문가용 템플릿'인 Seaborn을 사용할 차례입니다. Seaborn은 Pandas의 DataFrame과 매우 잘 연동되며, 복잡한 통계 그래프를 단 한 줄의 코드로 구현합니다.

데이터셋 불러오기

Seaborn은 연습용 데이터셋을 기본적으로 제공합니다. 여기서는 유명한 iris(붓꽃) 데이터를 사용하겠습니다.

```
import seaborn as sns
import pandas as pd

# Seaborn 내장 데이터셋 불러오기
iris = sns.load_dataset('iris')
print(iris.head())
```

상관관계를 한눈에 보는 Pair Plot

비유: 모든 각도에서 찍은 다각도 사진첩 한 사람의 정면, 측면, 윗면 사진을 모두 모아둔 앨범처럼, 데이터셋에 있는 모든 변수 쌍을 짝지어 한꺼번에 그려낸 사진첩과 같습니다.

직관: 전체 변수 간의 관계를 한눈에 훑기 변수가 여러 개일 때, 어떤 변수끼리 서로 밀접한 관계가 있는지 일일이 산점도를 그리지 않고 한 번에 파악하기 위해 사용합니다. 특히 타겟 변수(분류하고자 하는 대상)에 따라 데이터가 어떻게 나뉘는지 확인하는 데 매우 유용합니다.

기술 설명: `sns.pairplot()` `sns.pairplot()`은 데이터프레임의 모든 숫자형 변수 조합에 대해 산점도를 그립니다. `hue` 파라미터를 지정하면 특정 범주(예: 꽃의 종)에 따라 색상을 다르게 표시하여 그룹 간의 차이를 명확히 볼 수 있습니다.

[실습 4] 변수 간의 관계 총망라하기

```
sns.pairplot(iris, hue='species', palette='husl')
plt.suptitle('Iris Pair Plot', y=1.02, fontsize=15)
plt.show()
```

[실행 결과 해석] - `hue='species'`: 꽃의 종류(species)에 따라 색깔을 다르게 칠합니다. - **대각선 그래프:** 각 변수의 분포(히스토그램/밀도 그래프)를 보여줍니다. - **나머지 그래프:** 두 변수 간의 산점도를 보여줍니다. - **분석 결과:** `setosa` 종은 다른 두 종과 확연히 구분되는 영역에 분포하고 있지만, `versicolor`와 `virginica`는 일부 영역이 겹치는 것을 볼 수 있습니다. 이는 나중에 머신러닝 모델이 두 종을 구분하는 데 더 어려움을 겪을 것임을 암시합니다.

데이터의 요약을 보는 박스 플롯 (Box Plot)

비유: 데이터의 '요약 성적표' 상세한 점수를 다 보여주는 대신, "최하위는 몇 점, 중간은 몇 점, 최상위는 몇 점"이라고 핵심 요약 정보만 적어놓은 성적표와 같습니다.

직관: 중앙값과 이상치를 빠르게 식별 데이터의 전체적인 범위와 중앙값이 어디인지, 그리고 일반적인 범위를 크게 벗어난 '이상치'가 존재하는지 확인하고 싶을 때 사용합니다. 여러 그룹의 분포를 나란히 놓고 비교할 때 매우 강력합니다.

기술 설명: `sns.boxplot()` `sns.boxplot()`은 데이터를 4분위수로 나누어 박스 형태로 표현합니다.

[그림: 박스 플롯의 5가지 요약 수치 표시도 - 최솟값, 제1사분위수(Q1), 중앙값(Median), 제3사분위수(Q3), 최댓값 그리고 이상치(Outlier)가 표시된 박스 플롯 구조도]

[실습 5] 종별 꽃잎 길이 비교하기

```
plt.figure(figsize=(8, 6))
sns.boxplot(x='species', y='petal_length', data=iris)
```



```
plt.title('Petal Length by Species', fontsize=15)
plt.show()
```

[실행 결과 해석] - **박스의 길이**: 데이터의 50%가 모여 있는 구간입니다. 박스가 짧을수록 데이터가 밀집되어 있다는 뜻입니다. - **중앙선**: 박스 내부의 선은 데이터의 중앙값을 나타냅니다. - **수염 (Whiskers)**: 데이터의 정상 범위를 나타내며, 이 범위를 벗어나 찍힌 점들은 **이상치(Outlier)**로 간주됩니다.

관계의 강도를 보는 히트맵 (Heatmap)

비유: 데이터의 열화상 카메라 온도가 높은 곳은 빨갛게, 낮은 곳은 파랗게 표시하는 열화상 카메라처럼, 수치가 높은 곳은 진한 색으로, 낮은 곳은 연한 색으로 표현하는 지도와 같습니다.

직관: 수치적 관계의 강도를 색상으로 즉각 인지 변수가 너무 많아 산점도를 수십 개 그릴 수 없을 때, 상관관계수(Correlation)라는 숫자를 색상으로 변환하여 어떤 변수들이 서로 강하게 연결되어 있는지 한눈에 파악하기 위해 사용합니다.

기술 설명: **sns.heatmap()** sns.heatmap()은 2차원 행렬 데이터를 색상으로 시각화합니다. annot=True를 설정하면 각 칸에 실제 수치를 표시할 수 있으며, cmap을 통해 색상 테마를 변경할 수 있습니다.

[실습 6] 변수 간 상관관계수 시각화하기

```
# 상관관계수 계산 (숫자 데이터만 선택)
corr = iris.drop(columns=['species']).corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap='RdYlGn', fmt=".2f")
plt.title('Correlation Heatmap', fontsize=15)
plt.show()
```

[실행 결과 해석] - **annot=True**: 각 칸에 상관관계수 숫자를 표시합니다. - **색상**: 1에 가까울수록(초록 색) 강한 양의 상관관계, -1에 가까울수록(빨간색) 강한 음의 상관관계를 의미합니다. - **분석 결과**: petal_length와 petal_width의 상관관계수가 매우 높게(약 0.96) 나타납니다. 이는 꽃잎의 길이가 길면 너비도 함께 넓어진다는 아주 강한 관계를 뜻합니다.

4.4 실무 활용 사례: 주택 가격 데이터 분석

실제 실무에서는 어떻게 시각화를 활용할까요? 간단한 주택 가격 데이터를 가정하고 EDA 과정을 시뮬레이션해 보겠습니다.

시나리오: 당신은 집값 예측 모델을 만들어야 합니다. 가지고 있는 데이터는 평수, 방 개수, 역과의 거리, 집값입니다.

EDA 프로세스 적용

1. 데이터 분포 확인 (Histogram):

- 집값의 분포를 그려봅니다. 만약 집값이 한쪽으로 너무 쏠려 있다면(Skewed), 로그 변환 같은 전처리가 필요함을 알 수 있습니다.

2. 상관관계 파악 (Heatmap):

- 어떤 요소가 집값에 가장 큰 영향을 주는지 확인합니다. 평수 $\rightarrow$ 집값의 상관관계수가 가장 높다면, 평수가 핵심 피처(Feature)가 됩니다.

3. 이상치 제거 (Box Plot):

- 평수 대비 집값이 말도 안 되게 낮은 집(예: 급매물이나 데이터 오류)을 찾아내어 제거합니다.

4. 관계 시각화 (Scatter Plot):

- 평수와 집값의 산점도를 그려 선형적인 관계인지 확인합니다. 직선 형태라면 '선형 회귀' 모델을, 곡선 형태라면 '다항 회귀'나 '트리 기반' 모델을 고려합니다.

x

4.5 초보자가 자주 하는 실수와 해결 방법

실수 1: 축 이름(Label)과 제목(Title) 생략

^x 그래프만 덜렁 그려놓고 나중에 "이 x축이 뭐였지?"라고 당황하는 경우가 많습니다. - **해결책:** `plt.xlabel()`, `plt.ylabel()`, `plt.title()`은 선택이 아닌 필수입니다. 협업 시 다른 사람이 내 그래프를 보고 즉시 이해할 수 있어야 합니다.

실수 2: 부적절한 그래프 선택

^x 범주형 데이터(예: 혈액형, 지역)를 선 그래프(`plt.plot()`)로 그리는 실수입니다. 선 그래프는 시간의 흐름이나 연속적인 변화를 나타낼 때 사용합니다. - **해결책:** - 연속형 변수 $\rightarrow$ 산점도, 히스토그램, 선 그래프 - 범주형 변수 $\rightarrow$ 막대 그래프, 박스 플롯

실수 3: 너무 많은 색상과 정보 사용

한 그래프에 10개의 변수를 모두 넣거나, 너무 화려한 색상을 사용하여 오히려 가독성을 해치는 경우입니다. - **해결책:** 하나의 그래프에는 **하나의 핵심 메시지**만 담으세요. 복잡한 관계는 `pairplot`으로 먼저 훑고, 중요한 관계만 따로 떼어 세밀하게 시각화하는 것이 좋습니다.

실수 4: 한글 깨짐 현상

Matplotlib은 기본적으로 한글 폰트를 지원하지 않아 한글을 입력하면 네모(ㅍ)로 표시됩니다. -
해결책: 아래 코드를 상단에 추가하여 한글 폰트를 설정해야 합니다. (Windows 기준)

```
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'Malgun Gothic' # 윈도우 맑은 고딕
plt.rcParams['axes.unicode_minus'] = False    # 마이너스 기호 깨짐 방지
```

4.6 요약 및 정리

도구	주요 특징	추천 용도	핵심 함수
Matplotlib	저수준, 세밀한 제어 가능	기본 그래프, 커스텀 디자인	plt.plot(), plt.scatter(), plt.hist()
Seaborn	고수준, 통계 기반, 미려함	데이터 탐색(EDA), 복잡한 관계 분석	sns.pairplot(), sns.boxplot(), sns.heatmap()

이번 장의 핵심 흐름: 1. 데이터 시각화는 수치 데이터에서 패턴과 이상치를 찾기 위한 필수 과정입니다. 2. Matplotlib으로 기본적인 선, 점, 막대 그래프를 그릴 수 있습니다. 3. Seaborn을 통해 변수 간의 상관관계, 분포, 통계적 특성을 효율적으로 분석할 수 있습니다. 4. EDA(탐색적 데이터 분석) 과정을 통해 데이터의 특성을 파악하는 것이 머신러닝 모델의 성능을 결정짓는 첫 단추가 됩니다.

이제 당신은 데이터를 시각적으로 분석할 수 있는 능력을 갖추었습니다. 다음 장에서는 이렇게 분석한 데이터를 머신러닝 모델이 학습할 수 있도록 가공하는 데이터 전처리 과정에 대해 알아보겠습니다.

챕터 5: 데이터 전처리

5장. 데이터 전처리: 모델을 위한 최고의 식재료 준비하기

머신러닝 모델을 만드는 과정은 흔히 '요리'에 비유됩니다. 아무리 세계 최고의 셰프(최신 알고리즘)가 요리를 하더라도, 재료(데이터)가 썩었거나 흠이 잔뜩 묻어 있다면 결코 훌륭한 요리가 나올 수 없습니다.

우리가 수집하는 실제 데이터는 대부분 '지저분'합니다. 값이 비어 있기도 하고, 말도 안 되게 큰 값이 섞여 있기도 하며, 컴퓨터가 이해하지 못하는 텍스트 형태로 되어 있기도 합니다. 이렇게 가공되지 않은 원시 데이터(Raw Data)를 머신러닝 모델이 학습할 수 있는 깨끗한 상태로 만드는 과정을 **데이터 전처리(Data Preprocessing)**라고 합니다.

데이터 분석 업계에는 **"Garbage In, Garbage Out (쓰레기가 들어가면 쓰레기가 나온다)"**라는 유명한 격언이 있습니다. 모델의 성능을 올리기 위해 복잡한 하이퍼파라미터를 튜닝하는 것보다, 데이터를 제대로 전처리하는 것이 훨씬 더 극적인 성능 향상을 가져오는 경우가 많습니다. 이번 장에서는 머신러닝의 필수 단계인 결측치 처리, 이상치 제거, 스케일링, 그리고 인코딩에 대해 자세히 알아보겠습니다.

1. 결측치 처리: 빈칸 채우기

비유로 이해하기: 조각이 사라진 퍼즐

여러분은 1,000피스짜리 퍼즐을 맞추고 있습니다. 그런데 상자를 열어보니 몇 가지 조각이 빠져 있습니다. 이 상태로 퍼즐을 맞추려고 하면 그림의 전체적인 흐름을 파악하기 어렵고, 결국 완성할 수 없게 됩니다. 이때 우리는 두 가지 선택을 할 수 있습니다. 1. 조각이 없는 부분은 과감히 포기하고 나머지 부분만 맞추다. 2. 주변 조각들의 색상과 모양을 보고, 비슷한 색의 종이를 오려 붙여 빈칸을 메운다.

데이터 전처리에서의 **결측치(Missing Value)** 처리가 바로 이 과정과 같습니다.

직관적으로 이해하기

머신러닝 알고리즘은 기본적으로 수학 공식의 집합입니다. 수학 공식에 '값 없음(NaN, Not a Number)'이라는 상태가 들어가면 계산 결과는 정의되지 않거나 오류가 발생합니다. 따라서 모델에 데이터를 넣기 전, 비어 있는 칸을 어떻게 처리할지 결정해야 합니다.

기술 설명: 결측치 처리 전략

결측치를 처리하는 방법은 크게 **제거(Deletion)**와 **대체(Imputation)**로 나뉩니다.

1. **제거 (Deletion)**: 결측치가 포함된 행(Row)이나 열(Column)을 삭제합니다.
2. **장점**: 가장 빠르고 확실한 방법입니다.
3. **단점**: 데이터 손실이 큼니다. 특히 데이터셋이 작을 때 사용하면 치명적입니다.
4. **대체 (Imputation)**: 특정 값으로 빈칸을 채웁니다.
5. **평균(Mean)**: 수치형 데이터에서 가장 일반적인 방법입니다.
6. **중앙값(Median)**: 이상치가 많아 평균이 왜곡되었을 때 유용합니다.
7. **최빈값(Mode)**: 범주형(텍스트) 데이터에서 가장 자주 나타나는 값으로 채웁니다.
8. **예측값**: 다른 변수들과의 관계를 이용해 머신러닝 모델로 예측하여 채웁니다.

실습: 결측치 확인 및 처리하기

```
import pandas as pd
import numpy as np

# 1. 샘플 데이터 생성 (결측치가 포함된 데이터)
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, np.nan, 30, np.nan, 22],
    'Score': [85, 90, np.nan, 70, np.nan],
    'City': ['Seoul', 'Busan', 'Seoul', np.nan, 'Incheon']
}
df = pd.DataFrame(data)
print("--- 원본 데이터 ---")
print(df)

# 2. 결측치 확인
print("\n--- 결측치 개수 확인 ---")
print(df.isnull().sum())

# 3. 전략 1: 결측치가 있는 행 삭제 (dropna)
df_dropped = df.dropna()
print("\n--- 행 삭제 결과 ---")
print(df_dropped)

# 4. 전략 2: 특정 값으로 대체 (fillna)
df_filled = df.copy()

# 수치형 데이터: Age는 평균으로, Score는 중앙값으로 채우기
df_filled['Age'] = df_filled['Age'].fillna(df_filled['Age'].mean())
```

```
df_filled['Score'] = df_filled['Score'].fillna(df_filled['Score'].median())

# 범주형 데이터: City는 최빈값(가장 많이 나온 값)으로 채우기
mode_city = df_filled['City'].mode()[0]
df_filled['City'] = df_filled['City'].fillna(mode_city)

print("\n--- 대체 결과 ---")
print(df_filled)
```

실행 결과 해석 - `isnull().sum()`을 통해 어떤 컬럼에 결측치가 얼마나 있는지 확인했습니다. - `dropna()`를 사용하면 단 하나의 빈칸이라도 있는 행은 모두 삭제되므로 데이터가 급격히 줄어드는 것을 볼 수 있습니다. - `fillna()`를 통해 Age는 평균값으로, Score는 중앙값으로, City는 'Seoul'(최빈 값)로 적절히 채워져 데이터의 손실 없이 학습 준비를 마쳤습니다.

초보자가 자주 하는 실수 결측치를 단순히 0으로 채우는 경우가 많습니다. 하지만 '나이'가 0살이거나 '시험 점수'가 0점인 것은 실제 값일 가능성이 큼니다. 0은 하나의 의미를 가진 숫자이므로, 데이터의 특성을 고려하지 않고 0으로 채우면 모델이 이를 실제 값으로 오인하여 잘못된 학습을 하게 됩니다.

2. 이상치 처리: 튀는 값 잡아내기

비유로 이해하기: 고양이 방의 코끼리

귀여운 아기 고양이 10마리가 모여 있는 방이 있습니다. 평균 몸무게를 재보니 약 1kg 정도였습니다. 그런데 갑자기 거대한 코끼리 한 마리가 방으로 들어왔습니다. 이제 다시 평균 몸무게를 재면 어떻게 될까요? 평균값은 수백 kg으로 치솟습니다. 하지만 이 평균값이 방 안에 있는 동물들의 '일반적인' 상태를 대표한다고 말할 수 있을까요? 전혀 아닙니다. 코끼리는 여기서 **이상치(Outlier)**입니다.

직관적으로 이해하기

데이터 속에는 측정 오류, 입력 실수, 혹은 아주 희귀한 케이스로 인해 다른 값들과 동떨어진 극단적인 값이 존재합니다. 머신러닝 모델, 특히 선형 회귀나 K-평균 군집화 같은 알고리즘은 이러한 극단적인 값에 매우 민감하게 반응합니다. 이상치가 하나만 섞여 있어도 모델의 결정 경계가 크게 왜곡되어 전체적인 예측 성능이 떨어질 수 있습니다.

기술 설명: 이상치 탐지 및 처리 방법

이상치를 찾는 가장 대표적인 방법은 **IQR(Interquartile Range)** 방식입니다.

1. IQR 방식:

2. 데이터를 크기순으로 정렬한 뒤, 하위 25%(Q1)와 상위 25%(Q3) 지점을 찾습니다.

3. $IQR = Q3 - Q1$

4. 하한선: $Q1 - 1.5 \times IQR$

5. 상한선: $Q3 + 1.5 \times IQR$

6. 이 범위를 벗어나는 값들을 이상치로 간주합니다.

7. 처리 방법:

8. 제거: 이상치가 명백한 오류(예: 나이가 500살)라면 삭제합니다.

9. 대체: 상한선/하한선 값으로 강제로 맞추거나(Capping), 중앙값으로 대체합니다.

10. 로그 변환: 데이터의 분포가 한쪽으로 쏠려 있을 때 로그를 취해 범위를 좁힙니다.

실습: IQR을 이용한 이상치 제거 및 시각화

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. 이상치가 포함된 데이터 생성
data = {'Score': [80, 85, 88, 92, 95, 82, 87, 10, 150]} # 10과 150은 이상치
df = pd.DataFrame(data)

# 2. IQR 계산
Q1 = df['Score'].quantile(0.25)
Q3 = df['Score'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"Q1: {Q1}, Q3: {Q3}, IQR: {IQR}")
print(f"정상 범위: {lower_bound} ~ {upper_bound}")

# 3. 이상치 필터링
df_clean = df[(df['Score'] >= lower_bound) & (df['Score'] <= upper_bound)]

# 4. 시각화 (Boxplot)
plt.figure(figsize=(6, 4))
plt.boxplot(df['Score'])
plt.title('Score Distribution (Before Cleaning)')
plt.ylabel('Score')
plt.show()

print("\n--- 이상치 제거 전 ---")
print(df)
```



```
print("\n--- 이상치 제거 후 ---")
print(df_clean)
```

실행 결과 해석 - 시각화 결과: 박스 플롯(Boxplot)을 그려보면, 박스 위아래로 멀리 떨어져 있는 점들이 보입니다. 이 점들이 바로 정상 범위를 벗어난 이상치(10, 150)입니다. - **필터링 결과:** 계산된 `$lower_bound$`와 `$upper_bound$`를 기준으로 데이터를 필터링하여, 모델이 '일반적인 학생들의 점수'를 더 정확하게 학습할 수 있는 상태가 되었습니다.

실무 활용 사례: 이상치 탐지(Anomaly Detection) 모든 이상치를 제거하는 것이 항상 정답은 아닙니다. 신용카드 부정 결제 탐지(Fraud Detection) 시스템에서는 '평균과 다른 패턴의 결제'라는 이상치 자체가 우리가 찾아야 할 정답입니다. 이 경우 이상치를 제거하는 것이 아니라, 이상치를 찾아내는 모델을 만드는 것이 목적이 됩니다.

3. 피쳐 스케일링: 단위 맞추기 (정규화와 표준화)

비유로 이해하기: 미터와 센티미터의 혼용

어떤 사람이 키를 잴 때, 한 사람은 '1.8(미터)'라고 기록하고 다른 사람은 '180(센티미터)'이라고 기록했습니다. 컴퓨터는 이 숫자가 '키'라는 사실을 모릅니다. 컴퓨터가 보기에는 1.8보다 180이 100배나 더 큰 값입니다. 만약 이 데이터를 그대로 머신러닝 모델에 넣는다면, 모델은 180이라는 숫자가 가진 영향력이 1.8보다 훨씬 크다고 오해하게 됩니다.

직관적으로 이해하기

머신러닝 모델은 숫자의 절대적인 크기에 영향을 받습니다. - 변수 A: 연봉 (단위: 원, 범위: 3,000만 ~ 1억) - 변수 B: 나이 (단위: 세, 범위: 20 ~ 60)

연봉의 숫자가 나이의 숫자보다 훨씬 크기 때문에, 모델은 나이의 변화보다 연봉의 변화를 훨씬 중요하게 인식합니다. 이를 방지하기 위해 모든 변수의 범위를 일정하게 맞춰주는 과정이 피쳐 스케일링(Feature Scaling)입니다.

기술 설명: 정규화 vs 표준화

가장 많이 쓰이는 두 가지 방법은 정규화(Normalization)와 표준화(Standardization)입니다.

구분	정규화 (Normalization / Min-Max Scaling)	표준화 (Standardization / Z-Score Scaling)
개념	모든 값을 0과 1 사이의 값으로 변환	평균을 0, 표준편차를 1로 변환
공식	$\frac{x - \min}{\max - \min}$	$\frac{x - \mu}{\sigma}$
특징	데이터의 최소/최대를 명확히 알 때 유용	

구분	정규화 (Normalization / Min-Max Scaling)	표준화 (Standardization / Z-Score Scaling)
		이상치에 덜 민감하며 가우시안 분포를 따를 때 유용
결과 범위	$[0, 1]$	제한 없음 (주로 $-3 \sim 3$ 사이)

실습: Scikit-learn을 이용한 스케일링

```

from sklearn.preprocessing import MinMaxScaler, StandardScaler
import pandas as pd

# 1. 데이터 생성 (연봉과 나이)
data = {
    'Age': [25, 35, 45, 20, 50],
    'Salary': [3000, 5000, 8000, 2500, 12000] # 단위: 만원
}
df = pd.DataFrame(data)
print("--- 스케일링 전 데이터 ---")
print(df)

# 2. 정규화 (Min-Max Scaling)
min_max_scaler = MinMaxScaler()
df_minmax = pd.DataFrame(min_max_scaler.fit_transform(df), columns=df.columns)

# 3. 표준화 (Standard Scaling)
std_scaler = StandardScaler()
df_std = pd.DataFrame(std_scaler.fit_transform(df), columns=df.columns)

print("\n--- 정규화 결과 (0~1) ---")
print(df_minmax)
print("\n--- 표준화 결과 (평균 0, 표준편차 1) ---")
print(df_std)

```

실행 결과 해석 - 정규화 결과: Age와 Salary 모두 0에서 1 사이의 값으로 변환되었습니다. 이제 두 변수는 동일한 가중치 선상에서 비교됩니다. - **표준화 결과:** 값이 0을 중심으로 분포하며, 평균보다 큰 값은 양수, 작은 값은 음수로 표현됩니다.

언제 무엇을 써야 할까요? - 정규화: 데이터의 분포가 정규분포가 아니거나, 딥러닝 (ANN) 모델을 사용할 때 주로 사용합니다. (예: 이미지 픽셀 값 $0 \sim 255 \rightarrow$)

0~1) - **표준화**: 데이터에 이상치가 포함되어 있거나, SVM, 선형 회귀, 로지스틱 회귀처럼 데이터가 정규분포를 따른다고 가정하는 알고리즘을 사용할 때 유리합니다.

4. 인코딩: 문자를 숫자로 바꾸기

비유로 이해하기: 외국인과의 대화

여러분은 한국어만 할 줄 알고, 상대방은 숫자(수학)만 이해하는 로봇입니다. 여러분이 로봇에게 "이 과일은 '사과'입니다"라고 말하면 로봇은 "사과"라는 글자가 무엇인지 전혀 이해하지 못합니다. 로봇과 소통하려면 '사과'라는 단어를 로봇이 이해할 수 있는 '코드 번호(예: 1번)'로 바꿔서 알려줘야 합니다.

직관적으로 이해하기

머신러닝 모델의 내부 연산은 모두 행렬 곱셈과 같은 수학적 계산입니다. "서울", "부산", "제주" 같은 텍스트 데이터는 계산기에 넣을 수 없습니다. 따라서 텍스트 형태의 범주형 데이터를 숫자로 변환하는 과정이 필요한데, 이를 **인코딩(Encoding)**이라고 합니다.

기술 설명: 레이블 인코딩 vs 원-핫 인코딩

단순히 숫자로 바꾸는 것 같지만, 어떤 방식으로 바꾸느냐에 따라 모델이 데이터를 해석하는 방식이 완전히 달라집니다.

1. 레이블 인코딩 (Label Encoding):

2. 각 범주에 고유한 정수를 부여합니다. (예: 서울=0, 부산=1, 제주=2)

3. **문제점**: 모델이 숫자 간의 크기 관계를 학습할 수 있습니다. 모델은 "제주(2)가 서울(0)보다 크다" 혹은 "부산(1)과 제주(2)는 가깝다"라고 잘못 판단할 위험이 있습니다.

4. **용도**: '학위(학사 < 석사 < 박사)'처럼 순서 의미가 있는 데이터에 적합합니다.

5. 원-핫 인코딩 (One-Hot Encoding):

6. 각 범주를 새로운 열(Column)로 만들고, 해당하면 1, 아니면 0으로 표시합니다.

7. **장점**: 숫자 간의 순서나 크기 개념이 사라지므로, 범주 간의 독립성을 유지할 수 있습니다.

8. **단점**: 범주의 개수가 너무 많으면 열이 급격히 늘어나 메모리 낭비가 심해집니다(차원의 저주).

실습: 인코딩 적용하기

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd
```

```
# 1. 데이터 생성
```

```

data = {
    'City': ['Seoul', 'Busan', 'Jeju', 'Seoul', 'Busan'],
    'Grade': ['A', 'B', 'C', 'A', 'B']
}
df = pd.DataFrame(data)
print("--- 원본 데이터 ---")
print(df)

# 2. 레이블 인코딩 (Grade처럼 순서가 있는 경우)
le = LabelEncoder()
df['Grade_Encoded'] = le.fit_transform(df['Grade'])

# 3. 원-핫 인코딩 (City처럼 순서가 없는 경우)
# pandas의 get_dummies를 사용하면 매우 간편합니다.
df_onehot = pd.get_dummies(df, columns=['City'])

print("\n--- 레이블 인코딩 결과 ---")
print(df[['Grade', 'Grade_Encoded']])
print("\n--- 원-핫 인코딩 결과 ---")
print(df_onehot)

```

실행 결과 해석 - Grade는 A $\rightarrow$ 0, B $\rightarrow$ 1, C $\rightarrow$ 2 형태로 바뀌었습니다. 이는 성적이라는 순서가 있으므로 레이블 인코딩이 적절합니다. - City는 City_Busan, City_Jeu, City_Seoul이라는 3개의 열로 분리되었습니다. 이제 모델은 각 도시를 서로 독립적인 개체로 인식하며, 특정 도시가 다른 도시보다 '크다'고 오해하지 않습니다.

초보자가 자주 하는 실수 도시 이름이나 혈액형 같은 **명목형 데이터**에 레이블 인코딩을 적용하는 실수를 많이 합니다. 만약 '혈액형'을 0, 1, 2, 3으로 인코딩하면, 모델은 "혈액형 3은 혈액형 0보다 3배 더 강하다"라는 식의 엉뚱한 관계를 학습할 수 있습니다. 순서가 없는 데이터에는 반드시 **원-핫 인코딩**을 사용하세요.

5장 요약 및 체크리스트

이번 장에서는 머신러닝 모델의 성능을 결정짓는 가장 중요한 단계인 데이터 전처리를 학습했습니다. 각 단계의 핵심 내용을 표로 정리합니다.

단계	해결하려는 문제	주요 방법	핵심 포인트
결측치 처리	비어 있는 데이터 값	제거, 평균/중앙값/최빈값 대체	0으로 무분별하게 채우지 말 것

단계	해결하려는 문제	주요 방법	핵심 포인트
이상치 처리	너무 크거나 작은 튀는 값	IQR 방식, 로그 변환, 제거/대체	데이터의 성격에 따라 유지 여부 결정
피처 스케일링	변수 간의 단위/범위 차이	정규화(Min-Max), 표준화(Z-score)	알고리즘 특성에 맞는 스케일러 선택
인코딩	텍스트 데이터를 숫자로 변환	레이블 인코딩, 원-핫 인코딩	순서 유무에 따라 인코딩 방식 선택

최종 체크리스트

- ☐ 데이터에 NaN 값이 있는지 확인하고 적절한 방법으로 채웠는가?
- ☐ 박스 플롯(Box Plot) 등을 통해 이상치를 확인하고 처리했는가?
- ☐ 서로 다른 단위를 가진 피처들의 스케일을 맞추었는가?
- ☐ 범주형 데이터를 숫자로 변환할 때, 순서의 의미가 있는지 판단하여 인코딩 방식을 선택했는가?

이제 여러분은 지저분한 원시 데이터를 머신러닝 모델이 맛있게 먹을 수 있는 '정제된 식재료'로 만들 수 있게 되었습니다. 다음 장에서는 이렇게 준비된 데이터를 사용하여 실제로 예측 모델을 만드는 머신러닝 알고리즘의 세계로 들어가 보겠습니다.

챕터 6: 지도학습

6장. 지도학습

지도학습: 정답지를 보고 배우는 머신러닝

정답지가 있는 문제집으로 공부하기

우리가 새로운 과목을 공부할 때 가장 효율적인 방법 중 하나는 '문제'와 '정답'이 함께 적힌 문제집을 푸는 것입니다. 먼저 문제를 풀고, 내가 낸 답이 맞았는지 정답지를 확인하며 틀린 부분을 수정합니다. 이 과정을 수백 번 반복하면, 나중에는 정답지가 없어도 처음 보는 유사한 문제를 만났을 때 정답을 맞힐 수 있게 됩니다.

머신러닝의 지도학습(Supervised Learning)이 바로 이 과정과 똑같습니다. 여기서 '문제'는 머신러닝 모델이 학습할 **특성(Feature)** 데이터가 되고, '정답'은 우리가 예측하고자 하는 **타겟(Target/Label)** 데이터가 됩니다. 컴퓨터에게 "이런 특성을 가진 데이터는 정답이 이것이야"라고 계속해서 알려주면, 컴퓨터는 데이터 속에 숨겨진 규칙을 찾아내어 새로운 데이터가 들어왔을 때 정답을 추론하게 됩니다.

직관적으로 이해하는 지도학습

지도학습을 더 쉽게 이해하기 위해 '과일 분류기'를 만든다고 가정해 봅시다.

여러분은 컴퓨터에게 사과와 오렌지를 구분하는 법을 가르치려 합니다. 이때 컴퓨터에게 단순히 사진만 주는 것이 아니라, 다음과 같이 정보를 제공합니다. - "빨간색이고 둥근 모양의 이 사진은 [사과]야." - "주황색이고 표면이 울퉁불퉁한 이 사진은 [오렌지]야."

이렇게 **데이터(사진)**와 **정답(사과/오렌지)**을 쌍으로 묶어서 제공하는 것이 지도학습의 핵심입니다. 컴퓨터는 수많은 사진을 보며 '빨간색'과 '둥근 모양'이라는 특징이 나타나면 '사과'일 확률이 높다는 규칙을 스스로 학습합니다. 학습이 끝난 후, 정답지가 없는 새로운 사진을 보여주면 컴퓨터는 학습한 규칙을 바탕으로 "이 사진은 빨간색이고 둥그니까 사과겠구나!"라고 예측하게 됩니다.

기술적 정의와 작동 원리

기술적으로 지도학습은 입력 변수(X)와 출력 변수(y) 사이의 관계를 나타내는 함수 f 를 찾는 과정입니다.

$$y = f(X)$$

- X (**특성, Feature**): 모델이 예측을 하기 위해 사용하는 입력 값입니다. (예: 과일의 색상, 무게, 크기)
- y (**타겟/레이블, Target/Label**): 모델이 맞혀야 하는 정답 값입니다. (예: 과일의 이름)
- f (**모델, Model**): X 를 넣었을 때 y 가 나오도록 만드는 수학적 규칙입니다.

지도학습의 목표는 실제 정답(y)과 모델이 예측한 값($\hat{y}$)의 차이(오차)를 최소화하는 최적의 함수 f 를 찾아내는 것입니다.

지도학습의 두 갈래: 회귀와 분류

지도학습은 예측하려는 정답(y)의 형태에 따라 크게 회귀(Regression)와 분류(Classification)로 나뉩니다.

1. 회귀 (Regression): 연속적인 숫자 예측하기

회귀는 정답이 연속적인 숫자(수치)일 때 사용합니다. "얼마나(How much)?" 또는 "얼마나 많이(How many)?"에 대한 답을 찾는 과정입니다.

- 비유:** 내일의 기온이 몇 도일지 예측하거나, 공부 시간에 따른 시험 점수를 예측하는 것과 같습니다.
- 예시:**
 - 집 평수, 위치, 건축 연도 $\rightarrow$ 집값 예측 (예: 5억 2천만 원)
 - 과거 매출 데이터 $\rightarrow$ 다음 달 예상 매출액 예측 (예: 1,200만 원)
 - 키와 몸무게 $\rightarrow$ 예상 체지방률 예측 (예: 22.5%)

2. 분류 (Classification): 카테고리 결정하기

분류는 정답이 정해진 몇 개의 선택지(범주) 중 하나일 때 사용합니다. "어떤 것(Which one)?"에 대한 답을 찾는 과정입니다.

- 비유:** 우체국 직원이 편지를 보고 '국내 우편'인지 '해외 우편'인지 분류함에 넣는 것과 같습니다.
- 예시:**
 - 이메일 텍스트 $\rightarrow$ 스팸 메일 여부 (스팸 vs 정상)
 - 환자의 검사 수치 $\rightarrow$ 질병 유무 (양성 vs 음성)
 - 필기체 숫자 이미지 $\rightarrow$ 숫자 판별 (0, 1, 2, ..., 9 중 하나)

회귀 vs 분류 한눈에 비교하기

구분	회귀 (Regression)	분류 (Classification)
출력값 형태	연속적인 숫자 (Continuous)	이산적인 범주 (Discrete/Categorical)
핵심 질문	"얼마인가?"	"어떤 그룹에 속하는가?"
결과 예시	72.5kg, 1,500원, 25.4도	합격/불합격, 개/고양이, A/B/C형
평가 지표	MSE, MAE, R^2 등 (오차의 크기)	정확도(Accuracy), 정밀도, 재현율 등

학습의 정석: 훈련 데이터와 테스트 데이터

머신러닝 모델을 만들 때 가장 위험한 행동 중 하나는 학습에 사용한 데이터를 그대로 사용하여 성능을 평가하는 것입니다.

시험 문제 유출 사건 (과적합의 비유)

선생님이 학생에게 수학 문제집 10페이지를 주고 공부시키라고 했습니다. 학생이 너무 성실한 나머지, 문제의 풀이 과정은 이해하지 않고 문제와 정답을 통째로 외워버렸습니다.

선생님이 평가를 위해 다시 그 10페이지의 문제를 냈습니다. 학생은 정답을 모두 맞혔습니다. 선생님은 "이 학생은 수학 천재구나!"라고 생각했습니다. 하지만 선생님이 한 번도 본 적 없는 새로운 문제를 내자, 학생은 단 한 문제도 풀지 못했습니다.

이 학생은 '수학적 원리'를 배운 것이 아니라 '정답'을 외운 것입니다. 머신러닝에서도 이런 현상이 발생하는데, 이를 과적합(Overfitting)이라고 합니다.

데이터 분리 (Train/Test Split)

이런 문제를 방지하기 위해 우리는 데이터를 두 그룹으로 나눕니다.

1. **훈련 데이터 (Training Set):** 모델이 규칙을 학습하기 위해 사용하는 데이터입니다. (교과서 문제)
2. **테스트 데이터 (Test Set):** 학습이 끝난 모델의 진짜 실력을 측정하기 위해 사용하는 데이터입니다. (실제 시험 문제)

데이터 분리 프로세스: - 전체 데이터 준비 $\rightarrow$ 훈련 데이터(보통 70~80%)와 테스트 데이터(20~30%)로 분리 $\rightarrow$ 훈련 데이터로 모델 학습 $\rightarrow$ 테스트 데이터로 성능 검증

[실습] 파이썬으로 구현하는 첫 번째 지도학습 모델

이제 실제로 scikit-learn 라이브러리를 사용하여 지도학습 모델을 만들어 보겠습니다. 우리는 붓꽃(Iris) 데이터를 사용하여 붓꽃의 꽃잎과 꽃받침 길이를 보고 이 꽃이 어떤 품종인지 맞추는 분류(Classification) 모델을 구현할 것입니다.

실습 코드: 붓꽃 품종 분류하기

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```

# 1. 데이터 로드
iris = load_iris()
X = iris.data # 특성: 꽃받침 길이, 너비, 꽃잎 길이, 너비
y = iris.target # 정답: 품종 (0: Setosa, 1: Versicolor, 2: Virginica)

# 2. 훈련 데이터와 테스트 데이터 분리 (8:2 비율)
# random_state는 결과를 동일하게 재현하기 위한 고정값입니다.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"전체 데이터 수: {len(X)}")
print(f"훈련 데이터 수: {len(X_train)}, 테스트 데이터 수: {len(X_test)}")

# 3. 모델 선택 및 학습 (의사결정나무 알고리즘 사용)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train) # .fit() 메서드가 바로 '학습' 과정입니다.

# 4. 예측 및 결과 확인
y_pred = model.predict(X_test) # .predict() 메서드로 새로운 데이터의 정답을 예측합니다.

# 5. 성능 평가
accuracy = accuracy_score(y_test, y_pred)
print(f"\n모델 예측 정확도: {accuracy:.2f}")

# 6. 새로운 데이터로 예측해보기
# 임의의 붓꽃 데이터: [꽃받침 길이, 너비, 꽃잎 길이, 너비]
new_flower = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_flower)
print(f"새로운 꽃의 예측 품종 번호: {prediction[0]}")
print(f"실제 품종 이름: {iris.target_names[prediction[0]]}")

```

실행 결과 해석

```

전체 데이터 수: 150
훈련 데이터 수: 120, 테스트 데이터 수: 30

모델 예측 정확도: 1.00
새로운 꽃의 예측 품종 번호: 0
실제 품종 이름: setosa

```

- **train_test_split**: 데이터를 8:2로 나누어 모델이 정답을 외우는 것을 방지했습니다.

- **DecisionTreeClassifier**: '의사결정나무'라는 모델을 사용했습니다. 앞서 설명한 수학적 함수 $f(x)$ 의 한 종류로, 이 모델은 내부적으로 '조건 분기(If-Then)' 형태의 규칙을 만들어 정답을 찾습니다. 마치 스무고개처럼 "꽃잎 길이가 2cm보다 큰가?"라는 질문을 던지며 정답을 찾아가는 방식이라고 이해하면 쉽습니다.
- **.fit(X_train, y_train)**: 이 단계가 바로 **지도학습**의 핵심입니다. 모델이 특성과 정답의 관계를 학습합니다.
- **accuracy_score**: 테스트 데이터의 실제 정답(y_{test})과 모델의 예측값(y_{pred})을 비교하여 얼마나 맞았는지 계산합니다. 결과가 1.00이라면 100% 맞혔다는 뜻입니다. 다만, 붓꽃(Iris) 데이터셋은 매우 단순하고 정제된 데이터이기 때문에 이런 결과가 나올 수 있습니다. 실제 현업에서 다루는 복잡한 데이터에서는 100% 정확도가 나오는 경우가 매우 드물며, 만약 이런 결과가 나온다면 모델이 데이터를 단순히 외워버린 '과적합' 상태가 아닌지 의심해 보아야 합니다.

주의할 점: 과적합(Overfitting)과 과소적합(Underfitting)

모델을 학습시킬 때 가장 주의해야 할 두 가지 함정이 있습니다.

1. 과적합 (Overfitting): 너무 과하게 학습함

모델이 훈련 데이터의 세세한 잡음(Noise)까지 모두 외워버린 상태입니다. 훈련 데이터에 대해서는 완벽한 성능을 보이지만, 처음 보는 테스트 데이터에서는 성능이 뚝 떨어집니다. - **현상**: 훈련 정확도는 매우 높음 $\rightarrow$ 테스트 정확도는 매우 낮음. - **원인**: 모델이 너무 복잡하거나, 데이터 양이 너무 적을 때 발생합니다. - **해결책**: 더 많은 데이터를 수집하거나, 모델의 복잡도를 낮추는 '규제'를 적용합니다.

2. 과소적합 (Underfitting): 학습이 부족함

모델이 데이터의 기본적인 규칙조차 찾아내지 못한 상태입니다. 공부를 너무 안 해서 훈련 데이터든 테스트 데이터든 모두 성적이 낮게 나옵니다. - **현상**: 훈련 정확도 낮음 $\rightarrow$ 테스트 정확도 낮음. - **원인**: 모델이 너무 단순하거나, 학습 시간이 부족할 때 발생합니다. - **해결책**: 더 복잡한 모델을 사용하거나, 특성(Feature)을 더 추가하여 모델에게 더 많은 힌트를 줍니다.

[도식화] 적합도 비교

상태	훈련 데이터 성능	테스트 데이터 성능	비유
과소적합	낮음 $\downarrow$	낮음 $\downarrow$	공부를 전혀 안 한 학생
적정적합	높음 $\uparrow$	높음 $\uparrow$	원리를 잘 이해한 학생
과적합	매우 높음 $\uparrow\uparrow$	낮음 $\downarrow$	정답지만 통째로 외운 학생

실무 활용 사례

지도학습은 현재 산업계에서 가장 광범위하게 사용되는 머신러닝 방식입니다.

1. 금융권: 신용 점수 예측 및 이상 거래 탐지 (FDS)

- **회귀:** 고객의 소득, 직업, 연체 기록 등을 바탕으로 **신용 점수(숫자)**를 예측합니다.
- **분류:** 평소 거래 패턴과 다른 이상한 결제 패턴이 나타났을 때, 이를 '**정상**'인지 '**사기 (Fraud)**'인지 분류하여 결제를 차단합니다.

2. 의료 분야: 질병 진단 AI

- **분류:** MRI나 CT 영상을 입력하면, 해당 부위가 '**정상**'인지 '**종양**'인지 분류합니다. 수만 장의 판독 결과가 포함된 데이터를 통해 학습한 모델이 의사의 진단을 돕습니다.

3. 이커머스: 수요 예측 및 가격 최적화

- **회귀:** 과거 판매량, 시즌, 이벤트 여부를 바탕으로 다음 달 **상품 판매량**을 예측하여 재고를 관리합니다.
- **분류:** 고객의 구매 이력을 분석하여 이 고객이 특정 상품을 '**구매할 것인가**' 아니면 '**이탈할 것인가**'를 분류합니다.

초보자가 자주 하는 실수와 해결 방법

Q: 훈련 데이터와 테스트 데이터를 나누지 않고 전체 데이터로 학습시켰는데 정확도가 100%가 나왔어요. 제 모델은 완벽한 건가요? - A: 아니요, 매우 위험한 상황입니다. 이는 전형적인 **과적합 (Overfitting)** 사례입니다. 모델이 정답을 외웠을 가능성이 큼니다. 반드시 데이터를 분리하여, 모델이 '학습하지 않은 데이터'에 대해 얼마나 잘 작동하는지 확인해야 합니다.

Q: 모델의 정확도가 너무 낮게 나옵니다. 어떻게 해야 할까요? - A: 다음 세 가지를 점검해 보세요.

1. **데이터의 품질:** 입력 데이터(X)에 정답(y)을 예측하는 데 필요한 핵심 정보가 포함되어 있는지 확인하세요. (예: 집값을 예측하는데 '집의 색깔' 정보만 있다면 예측이 불가능합니다.)
2. **데이터의 양:** 데이터가 너무 적으면 모델이 규칙을 찾기 어렵습니다. 데이터를 더 수집하세요.
3. **모델의 선택:** 사용 중인 알고리즘이 너무 단순한 것은 아닌지 확인하고, 다른 모델(예: Random Forest, XGBoost 등)로 교체해 보세요.

요약

- **지도학습**은 정답(Label)이 포함된 데이터를 통해 입력과 출력 사이의 관계를 학습하는 방식이다.

- 정답이 연속적인 숫자라면 **회귀(Regression)**, 정해진 카테고리 중 하나라면 **분류(Classification)**라고 한다.
- 모델의 진짜 성능을 측정하기 위해 데이터를 **훈련 데이터(Train)**와 **테스트 데이터(Test)**로 반드시 분리해야 한다.
- 훈련 데이터에만 너무 최적화되어 새로운 데이터에 대응하지 못하는 현상을 **과적합(Overfitting)**이라고 한다.
- 지도학습의 일반적인 흐름은 데이터 준비 $\rightarrow$ 데이터 분리 $\rightarrow$ 모델 선택 $\rightarrow$ 학습(.fit) $\rightarrow$ 예측(.predict) $\rightarrow$ 평가 순으로 진행된다.

챕터 7: 회귀

7장. 회귀 (Regression)

우리는 일상 속에서 끊임없이 무언가를 예측하며 살아갑니다. "오늘 기온이 5도 더 올라가면 아이스 아메리카노가 얼마나 더 팔릴까?", "우리 집의 평수가 넓어지면 집값은 얼마나 될까?"와 같은 질문들이 바로 그것입니다. 머신러닝에서 이러한 '연속적인 숫자'를 예측하는 기법을 회귀 (Regression)라고 합니다.

1. 회귀의 개념: 미래의 값을 예측하는 직선 그리기

1.1 비유로 이해하기: 떡볶이 가게의 매출 예측

여러분은 떡볶이 가게를 운영하는 사장님입니다. 지난 1년간의 데이터를 살펴보니 재미있는 점을 발견했습니다. 날씨가 추워질수록(기온이 낮아질수록) 떡볶이 매출이 올라간다는 것입니다.

이때, 여러분은 머릿속으로 이런 생각을 할 것입니다. "내일 기온이 영하 5도라면, 매출은 대략 50만 원 정도 되겠지?"

여기서 여러분은 무의식적으로 '기온'이라는 입력값과 '매출'이라는 결과값 사이의 어떤 규칙(관계)을 찾은 것입니다. 이 규칙을 그래프 위에 점으로 찍고, 그 점들을 가장 잘 대표하는 하나의 '직선'을 긋는 행위가 바로 회귀의 핵심입니다.

1.2 직관으로 이해하기: 가장 적절한 선 찾기

데이터들이 흩어져 있는 산점도(Scatter Plot)를 상상해 보세요. 점들이 대략적으로 오른쪽 위를 향해 뻗어 있다면, 우리는 그 점들 사이를 가로지르는 직선 하나를 그릴 수 있습니다.

[그림 7-1: 산점도와 최적의 직선] (그림 설명: X축은 평수, Y축은 집값으로 설정된 그래프에 여러 개의 데이터 점들이 흩어져 있고, 그 중심을 가로지르는 빨간색 직선이 그려져 있음. 각 데이터 점과 직선 사이의 수직 거리(오차)가 점선으로 표시되어 '오차'의 개념을 시각적으로 보여줌)

이 직선은 단순한 선이 아니라 '예측 모델'입니다. 새로운 기온 데이터가 들어왔을 때, 이 직선 위의 값을 읽으면 그것이 바로 우리의 예측값이 됩니다. 그렇다면 '가장 적절한 선'이란 무엇일까요? 그것은 실제 데이터 점들과 직선 사이의 거리(오차)가 가장 짧은 선을 의미합니다.

1.3 기술적 설명: 선형 회귀 (Linear Regression)

선형 회귀는 독립 변수 x (원인)와 종속 변수 y (결과)의 관계를 가장 잘 설명하는 직선의 방정식을 찾는 알고리즘입니다. 중학교 수학 시간에 배운 일차함수 식과 매우 유사합니다.

$$y = wx + b$$

- y (Target): 예측하려는 값 (예: 집값, 매출)

- x (Feature): 예측에 사용하는 값 (예: 평수, 기온)
- w (Weight/Coefficient): 가중치 혹은 기울기. x 가 한 단위 변할 때 y 가 얼마나 변하는지를 나타냅니다.
- b (Bias/Intercept): 편향 혹은 절편. x 가 0일 때의 기본 y 값을 의미합니다.

머신러닝 모델의 학습이란, 주어진 데이터를 통해 실제 값과 예측 값의 차이를 최소화하는 최적의 w 와 b 를 찾아내는 과정입니다.

2. 단순 선형 회귀: 집값 예측하기

먼저 하나의 특성(Feature)만을 사용하여 결과값을 예측하는 '단순 선형 회귀'를 실습해 보겠습니다. 집의 평수가 늘어질수록 집값이 올라간다는 가정을 바탕으로 모델을 만들어 보겠습니다.

2.1 실습 코드: 집값 예측 모델 구현

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# 1. 데이터 생성 (평수 vs 집값)
# 입문 단계에서는 가상 데이터를 생성하여 경향성을 확인합니다.
# 데이터 양이 너무 적으면 모델이 특정 데이터에만 과하게 적응(과적합)할 수 있으므로 50개의 데이터를 생성합니다.
np.random.seed(42)
X = np.random.randint(10, 60, size=50).reshape(-1, 1) # 10평~60평 사이의 랜덤한 평수 50개
# 집값 = 0.2 * 평수 + 0.5 + 랜덤 노이즈 (현실적인 데이터를 위해 약간의 오차 추가)
y = 0.2 * X.flatten() + 0.5 + np.random.normal(0, 0.5, size=50)

# 2. 학습 데이터와 테스트 데이터 분리 (8:2)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. 선형 회귀 모델 생성 및 학습
model = LinearRegression()
model.fit(X_train, y_train)

# 4. 예측 수행
y_pred = model.predict(X_test)

# 5. 결과 확인
print(f"학습된 기울기(w): {model.coef_[0]:.4f}")
```



```
print(f"학습된 절편(b): {model.intercept_:.4f}")
print(f"테스트 데이터 예측값: \n{y_pred}")
print(f"실제 테스트 데이터 값: \n{y_test}")

# 6. 시각화
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X), color='red', label='Regression Line')
plt.xlabel('House Size (Pyung)')
plt.ylabel('Price (100 Million Won)')
plt.title('House Price Prediction')
plt.legend()
plt.show()
```

2.2 결과 해석 및 분석

[출력 결과 예시] - 학습된 기울기(w): \$0.1985\$ - 학습된 절편(b): \$0.6123\$ - 테스트 데이터 예측값: [10.21, 3.45, ...] - 실제 테스트 데이터 값: [10.1, 3.6, ...]

해석: 1. 기울기(\$w \approx 0.20\$): 평수가 1평 증가할 때마다 집값이 약 0.2억 원(2,000만 원)씩 상승한다는 의미입니다. 2. 절편(\$b \approx 0.61\$): 이론적으로 평수가 0일 때의 값입니다. 현실적으로 평수가 0인데 집값이 존재하거나, 혹은 마이너스가 되는 것은 불가능합니다. 하지만 이 수치는 모델이 전체 데이터의 경향성을 가장 잘 나타내는 직선을 그리기 위해 계산된 **수학적 보정치**입니다. 즉, 0 근처의 값 자체에 의미를 두기보다, 직선의 전체적인 높낮이를 조절하여 오차를 줄이기 위한 장치로 이해하는 것이 좋습니다. 3. **예측값 vs 실제값**: 예측값과 실제값이 매우 유사하게 나왔음을 알 수 있습니다. 데이터의 양을 50개로 늘림으로써 모델이 특정 소수 데이터에 휘둘리지 않고 전체적인 경향성을 더 안정적으로 학습했음을 확인할 수 있습니다.

3. 다중 선형 회귀: 매출액 예측하기

현실 세계에서 결과값에 영향을 주는 요인은 단 하나가 아닙니다. 매출액은 단순히 '광고비'뿐만 아니라 '매장 크기', '유동 인구', '계절' 등 여러 요인의 영향을 받습니다. 이렇게 여러 개의 특성을 사용하는 것을 **다중 선형 회귀(Multiple Linear Regression)**라고 합니다.

식은 다음과 같이 확장됩니다:
$$y = w_1x_1 + w_2x_2 + w_3x_3 + \dots + b$$

3.1 실무 활용 사례: 마케팅 비용에 따른 매출 예측

기업에서는 마케팅 예산을 짤 때, TV 광고비와 SNS 광고비를 각각 얼마씩 썼을 때 매출이 얼마나 오를지 예측하고 싶어 합니다.

3.2 실습 코드: 다중 선형 회귀 구현

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

# 1. 가상 데이터 생성 (TV 광고비, SNS 광고비, 매장 크기 -> 매출액)
# 데이터의 신뢰도를 높이기 위해 50개의 샘플 데이터를 생성합니다.
np.random.seed(42)
n_samples = 50
data = {
    'TV_Budget': np.random.uniform(10, 250, n_samples),
    'SNS_Budget': np.random.uniform(1, 50, n_samples),
    'Store_Size': np.random.randint(20, 60, n_samples),
}
# 매출액 = (TV * 0.05) + (SNS * 0.03) + (Size * 0.1) + 기본매출 5 + 노이즈
data['Sales'] = (data['TV_Budget'] * 0.05 +
                 data['SNS_Budget'] * 0.03 +
                 data['Store_Size'] * 0.1 +
                 5 + np.random.normal(0, 1, n_samples))

df = pd.DataFrame(data)

# 2. 특성(X)과 타겟(y) 분리
X = df[['TV_Budget', 'SNS_Budget', 'Store_Size']]
y = df['Sales']

# 3. 데이터 분리
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. 모델 학습
model = LinearRegression()
model.fit(X_train, y_train)

# 5. 예측 및 평가
y_pred = model.predict(X_test)

print(f"각 특성별 가중치(w): \n{model.coef_}")
print(f"절편(b): {model.intercept_: .4f}")
print(f"결정계수(R2 Score): {r2_score(y_test, y_pred): .4f}")
```

3.3 결과 해석 및 분석

[출력 결과 예시] - 각 특성별 가중치(w): [0.048, 0.028, 0.105] - 절편(b): 5.1241 - 결정계수(R2 Score): 0.9612

해석: 1. **가중치 분석**: Store_Size의 가중치(0.105)가 가장 높습니다. 이는 다른 조건이 동일할 때 매장 크기가 매출에 가장 큰 영향을 미친다는 것을 의미합니다. 2. **결정계수(R^2)**: R^2 값은 0에서 1 사이의 값을 가지며, 1에 가까울수록 모델이 데이터를 잘 설명한다는 뜻입니다. 0.96이라는 수치는 이 모델의 예측력이 매우 높음을 시사합니다.

4. 모델은 어떻게 학습하는가? (오차와 최적화)

단순히 `model.fit()`을 호출하면 컴퓨터 내부에서는 어떤 일이 벌어지는 걸까요? 컴퓨터는 한 번에 정답 직선을 찾는 것이 아니라, **수많은 시행착오**를 거칩니다.

4.1 비유로 이해하기: 안개 낀 산 내려가기

당신은 지금 안개가 자욱한 산 정상에 서 있습니다. 목표는 가장 낮은 골짜기(오차가 최소인 지점)로 내려가는 것입니다. 하지만 안개 때문에 앞이 보이지 않습니다. 당신이 할 수 있는 최선은 무엇일까요?

바로 지금 내가 서 있는 곳에서 경사가 가장 가파른 아래쪽 방향으로 한 발자국 내딛는 것입니다. 그리고 그 자리에서 다시 경사를 확인하고 또 한 발자국 내려갑니다. 이 과정을 반복하다 보면 결국 가장 낮은 지점에 도달하게 됩니다.

[그림 7-2: 경사 하강법의 개념도] (그림 설명: U자 모양의 매끄러운 곡선(손실 함수 그래프)이 그려져 있고, 곡선의 높은 지점에 있는 점이 접선의 기울기 방향을 따라 조금씩 아래로 이동하며 최저점(Minimum)을 향해 내려가는 과정이 화살표로 표시된 그림)

4.2 직관으로 이해하기: 오차의 제곱합 (MSE)

컴퓨터는 '내가 그은 선이 얼마나 틀렸는가'를 측정하기 위해 **평균 제곱 오차(Mean Squared Error, MSE)**라는 지표를 사용합니다.

- **오차** = 실제 값 - 예측 값

- **제곱을 하는 이유**:

1. 오차가 음수일 때와 양수일 때가 서로 상쇄되어 0이 되는 것을 막기 위함입니다.
2. 큰 오차에 대해 더 큰 패널티를 주기 위해서입니다. (작은 실수보다 큰 실수를 더 심각하게 받아들여 빨리 수정하게 함)

4.3 기술적 설명: 경사 하강법 (Gradient Descent)

위의 '산 내려가기' 비유가 바로 **경사 하강법**입니다. 1. 임의의 w , b 값을 설정합니다. 2. 현재 w , b 에서의 MSE(오차)를 계산합니다. 3. MSE의 기울기(미분값)를 구해, 오차가 줄어드는 방향으로

w 와 b 를 조금씩 업데이트합니다. 4. 오차가 더 이상 줄어들지 않을 때까지 이 과정을 반복합니다.

5. 회귀 모델의 평가 지표

모델을 만들었다면, 이 모델이 얼마나 믿을만한지 숫자로 증명해야 합니다.

지표	이름	의미	특징
MSE	평균 제곱 오차	예측값과 실제값의 차이를 제곱하여 평균낸 값	값이 작을수록 좋음. 단위가 제공되어 있어 직관성이 낮음.
MAE	평균 절대 오차	예측값과 실제값의 차이의 절대값을 평균낸 값	값이 작을수록 좋음. 실제 오차의 물리적 거리와 같아 직관적임.
RMSE	루트 평균 제곱 오차	MSE에 루트를 씌운 값	MSE의 단점(단위 문제)을 해결한 지표.
R^2	결정계수	모델이 데이터의 분산을 얼마나 설명하는가	0~1 사이 값. 1에 가까울수록 설명력이 높음.

6. 초보자가 자주 하는 실수와 해결 방법

6.1 상관관계와 인과관계를 혼동하는 경우

실수: "아이스크림 판매량이 늘어나니 익사 사고가 증가한다. 따라서 아이스크림이 익사 사고의 원인이다!"라고 결론 내리는 경우입니다. 해결: 회귀 분석은 두 변수 사이의 상관관계를 보여줄 뿐, 인과관계를 증명하지 않습니다. 위 사례의 실제 원인은 '여름(기온 상승)'이라는 제3의 변수입니다. 데이터 분석 시에는 항상 도메인 지식을 바탕으로 논리적인 인과관계를 고민해야 합니다.

6.2 데이터 스케일링을 무시하는 경우

실수: 집값 예측 모델에서 '평수(10~100)'와 '방 개수(1~5)'를 그대로 넣었을 때, 숫자가 큰 '평수'가 모델에 압도적인 영향을 주는 경우입니다. 해결: 5장에서 배운 정규화(Normalization)나 표준화(Standardization)를 적용하세요. 모든 특성의 범위를 비슷하게 맞춰주어야 모델이 공정하게 가중치를 학습할 수 있습니다.

6.3 과적합(Overfitting) 문제

실수: 너무 많은 특성을 넣거나, 복잡한 고차 회귀 식을 사용하여 학습 데이터에는 완벽하게 맞지만, 새로운 데이터에서는 엉터리 예측을 하는 경우입니다. 해결: - 불필요한 특성을 제거합니다. - 데이터의 양을 늘립니다. - (심화) 규제(Regularization) 기법인 Ridge나 Lasso 회귀를 사용합니다.

7. 요약

- 회귀는 연속적인 수치(값)를 예측하는 머신러닝 기법입니다.
- 선형 회귀는 데이터의 경향성을 가장 잘 나타내는 직선($y = wx + b$)을 찾는 것입니다.
- 단순 선형 회귀는 특성이 하나일 때, 다중 선형 회귀는 특성이 여러 개일 때 사용합니다.
- 모델은 MSE(평균 제곱 오차)를 최소화하기 위해 경사 하강법과 같은 최적화 알고리즘을 통해 가중치를 업데이트합니다.
- 모델의 성능은 MAE, RMSE, R^2 등의 지표를 통해 평가하며, 특히 R^2 는 모델의 설명력을 나타냅니다.
- 분석 시에는 상관관계와 인과관계를 구분하고, 데이터의 스케일링에 주의해야 합니다.

챕터 8: 분류

8장. 분류 (Classification)

지난 7장에서는 숫자 데이터를 예측하는 '회귀'에 대해 배웠습니다. 집값이나 매출액처럼 연속적인 수치를 맞추는 것이 목표였다면, 이번 장에서 다룰 '분류'는 완전히 다른 목표를 가집니다. 분류는 데이터가 어떤 '그룹'에 속하는지를 결정하는 작업입니다.

1. 분류란 무엇인가?

비유: 우체국 직원의 편지 분류

상상해 보세요. 당신은 우체국에서 편지를 분류하는 직원입니다. 당신 앞에는 수천 통의 편지가 쌓여 있고, 당신의 임무는 이 편지들을 '중요한 편지'함과 '광고 전단지'함으로 나누는 것입니다.

당신은 편지를 하나씩 확인하며 몇 가지 기준을 세웁니다. - "봉투에 '무료', '당첨', '긴급'이라는 단어가 많이 적혀 있네? 이걸 광고일 확률이 높아." - "보낸 사람이 가족이나 회사 동료고, 내용이 정중하네? 이걸 중요한 편지야."

여기서 당신이 하는 행동이 바로 머신러닝의 '분류(Classification)'입니다. 이미 알고 있는 기준(특징)을 바탕으로 새로운 데이터가 들어왔을 때 그것이 어느 카테고리에 속하는지 판단하는 것이죠.

직관: 경계선 긋기

회귀가 "데이터의 흐름을 가장 잘 나타내는 하나의 선을 긋는 것"이었다면, 분류는 "데이터 집단 사이에 벽을 세우는 것"과 같습니다.

예를 들어, 사과와 오렌지를 구분한다고 가정해 봅시다. 사과는 대체로 붉은색이고, 오렌지는 주황색입니다. 우리는 '색상'이라는 기준을 가지고 좌표평면에 점을 찍은 뒤, 붉은색 영역과 주황색 영역을 가르는 결정 경계(Decision Boundary)를 그릴 수 있습니다. 새로운 과일이 나타났을 때, 그 과일이 경계선의 어느 쪽에 위치하느냐에 따라 "이것은 사과다" 혹은 "오렌지다"라고 판정하는 원리입니다.

기술 설명: 분류의 정의와 종류

기술적으로 분류는 종속 변수가 이산적인 값(Discrete value), 즉 범주형(Categorical) 데이터일 때 사용하는 지도학습 방법입니다.

분류는 정답(클래스)의 개수에 따라 크게 두 가지로 나뉩니다.

- 1. 이진 분류 (Binary Classification):** 답이 두 가지 중 하나인 경우입니다.
 - 예: 스팸 메일인가 아닌가? (Yes/No), 암 환자인가 아닌가? (Positive/Negative), 합격인가 불합격인가? (Pass/Fail)
- 2. 다중 분류 (Multi-class Classification):** 답이 세 가지 이상의 카테고리 중 하나인 경우입니다.
 - 예: 손글씨 숫자 인식 (0~9), 동물 사진 분류 (개, 고양이, 토끼, 햄스터), 뉴스 기사 카테고리 분류 (정치, 경제, 사회, 문화)

2. 분류를 위한 대표적인 알고리즘

분류를 구현하는 방법은 매우 다양합니다. 여기서는 입문자가 가장 먼저 접해야 할 세 가지 핵심 알고리즘을 소개합니다.

2.1 로지스틱 회귀 (Logistic Regression)

이름에 '회귀'가 들어가지만, 실제로는 **분류 알고리즘**입니다. 선형 회귀가 직선을 그려 수치를 예측한다면, 로지스틱 회귀는 그 수치를 **0과 1 사이의 확률값**으로 변환합니다.

- **핵심 원리:** 시그모이드(Sigmoid) 함수를 사용하여 결과를 0~1 사이로 압축합니다.
- **판단 기준:** 결과값이 0.5보다 크면 '1(Positive)', 작으면 '0(Negative)'으로 분류합니다.

2.2 결정 트리 (Decision Tree)

스무고개 게임과 똑같습니다. 특정 기준에 따라 데이터를 계속해서 쪼개 나가는 방식입니다.

- **핵심 원리:** "메일 제목에 '광고'라는 단어가 포함되어 있는가?" $\rightarrow$ Yes $\rightarrow$ "발신자가 모르는 사람인가?" $\rightarrow$ Yes $\rightarrow$ [스팸]
- **장점:** 사람이 이해하기 매우 쉽고(시각화 가능), 데이터 전처리에 덜 민감합니다.

2.3 랜덤 포레스트 (Random Forest)

결정 트리 하나만 사용하면 너무 세세한 규칙에 집착해 새로운 데이터에 적응하지 못하는 '과적합 (Overfitting)' 문제가 발생할 수 있습니다. 이를 해결하기 위해 **수많은 결정 트리를 만들고, 그들의 다수결로 최종 결과를 결정**하는 것이 랜덤 포레스트입니다.

- **핵심 원리:** 앙상블(Ensemble) 기법. 여러 개의 약한 모델을 합쳐 강한 모델을 만듭니다.
 - **비유:** 한 명의 전문가보다 100명의 일반인이 투표해서 결정하는 것이 더 안정적이라는 원리입니다.
-

3. 실습: 스팸 메일 분류기 만들기

이제 실제로 파이썬을 이용해 스팸 메일을 분류하는 모델을 만들어 보겠습니다. 텍스트 데이터를 다루는 과정이 포함되어 있어 매우 흥미로운 실습이 될 것입니다.

실습 목표

- 텍스트 데이터를 머신러닝 모델이 이해할 수 있는 숫자로 변환합니다.
- 랜덤 포레스트 알고리즘을 사용하여 스팸 메일을 분류합니다.
- 모델의 정확도를 측정합니다.

전체 코드 구현

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. 데이터 준비 (간단한 예제 데이터셋)
data = {
    'text': [
        '무료 쿠폰 당첨! 지금 바로 클릭하세요',
        '오늘 오후 2시에 회의가 있습니다',
        '최저가 보장! 이번 기회를 놓치지 마세요',
        '내일 점심 메뉴는 무엇인가요?',
        '축하합니다! 이벤트에 당첨되었습니다',
        '이번 주 보고서 제출 부탁드립니다',
        '대출 가능 금액 확인하세요. 무조건 승인!',
        '다음 주 월요일에 휴진입니다'
    ],
    'label': [1, 0, 1, 0, 1, 0, 1, 0] # 1: 스팸, 0: 정상
}

df = pd.DataFrame(data)

# 2. 텍스트 데이터의 수치화 (Vectorization)
# 머신러닝 모델은 텍스트를 직접 읽지 못하므로, 단어의 등장 횟수로 변환합니다.
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['text'])
y = df['label']

# 3. 학습 데이터와 테스트 데이터 분리
```

```

# 데이터가 적지만, 원칙을 지키기 위해 75% 학습, 25% 테스트로 나눕니다.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# 4. 모델 생성 및 학습 (랜덤 포레스트 사용)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 5. 예측 및 결과 평가
y_pred = model.predict(X_test)

print(f"모델 정확도: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print("\n[혼동 행렬 (Confusion Matrix)]")
print(confusion_matrix(y_test, y_pred))
print("\n[상세 보고서]")
print(classification_report(y_test, y_pred))

# 6. 새로운 메일 테스트
new_emails = ['무료 당첨 기회!', '내일 뵙겠습니다']
new_emails_vec = vectorizer.transform(new_emails)
predictions = model.predict(new_emails_vec)

for email, pred in zip(new_emails, predictions):
    status = '스팸' if pred == 1 else '정상'
    print(f"메일: '{email}' -> 결과: {status}")

```

결과 해석 및 분석

1. 실행 결과 (예시)

모델 정확도: 100.00%

[혼동 행렬 (Confusion Matrix)]

```
[[1 0]
 [0 1]]
```

[상세 보고서]

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	1
accuracy			1.00	2
macro avg	1.00	1.00	1.00	2

weighted avg	1.00	1.00	1.00	2
--------------	------	------	------	---

메일: '무료 당첨 기회!' -> 결과: 스팸

메일: '내일 뵙겠습니다' -> 결과: 정상

2. 결과 해석

- **정확도(Accuracy):** 모델이 전체 데이터 중 얼마나 맞혔는지를 나타냅니다. 여기서는 데이터 셋이 매우 단순하여 100%가 나왔지만, 실제 데이터에서는 더 낮은 수치가 나올 수 있습니다.
- **혼동 행렬(Confusion Matrix):**
 - [0, 0] 위치: 정상을 정상으로 맞힌 수 (True Negative)
 - [1, 1] 위치: 스팸을 스팸으로 맞힌 수 (True Positive)
 - 그 외: 오답 (False Positive, False Negative)
- **텍스트 수치화 과정:** CountVectorizer는 메일에 등장하는 모든 단어의 목록(사전)을 만들고, 각 메일마다 해당 단어가 몇 번 나왔는지 숫자로 바꿉니다. 예를 들어 "무료"라는 단어가 1번 나왔다면 해당 열의 값을 1로 기록하는 방식입니다.

4. 분류 모델 평가의 함정: 정확도만 믿지 마라

초보자가 가장 많이 하는 실수 중 하나가 '정확도(Accuracy)'만 보고 모델이 완벽하다고 생각하는 것입니다. 하지만 현실의 데이터는 '불균형(Imbalance)'한 경우가 많습니다.

불균형 데이터 문제

만약 1,000통의 메일 중 스팸이 10통뿐이라고 가정해 봅시다. 모델이 아무런 학습 없이 "모든 메일은 정상이다"라고만 답해도 정확도는 무려 99%가 됩니다. 하지만 이 모델은 스팸 메일을 하나도 잡지 못했으므로 쓸모없는 모델입니다.

그래서 분류에서는 다음과 같은 지표를 함께 봅니다.

지표	설명	비유
정밀도 (Precision)	스팸이라고 예측한 것 중 실제 스팸인 비율	"스팸이라고 한 것 중에 진짜 스팸이 얼마나 있어?"
재현율 (Recall)	실제 스팸인 것들 중 스팸이라고 맞힌 비율	"실제 스팸들을 얼마나 놓치지 않고 다 찾아냈어?"

지표	설명	비유
F1-Score	정밀도와 재현율의 조화 평균	"정밀도와 재현율 둘 다 적절하게 균형 잡혔니?"

- **정밀도가 중요한 경우:** 정상 메일을 스팸으로 분류하면 중요한 업무 메일을 놓치게 되므로, '스팸'이라고 확신할 때만 분류해야 합니다.
- **재현율이 중요한 경우:** 암 진단 모델의 경우, 암 환자를 정상으로 판단하면 생명이 위험하므로, 조금이라도 의심되면 '암'으로 분류해 정밀 검사를 받게 해야 합니다.

x

5. 초보자가 자주 하는 실수와 해결 방법

실수 1: 텍스트 데이터를 그대로 모델에 넣기

- **현상:** ValueError: could not convert string to float 에러 발생.
- **원인:** 머신러닝 모델은 수학 계산기입니다. 문자열('안녕하세요')을 더하거나 뺄 수 없습니다.
- **해결:** 반드시 CountVectorizer나 TfidfVectorizer 같은 도구를 사용해 텍스트를 숫자로 변환(벡터화)해야 합니다.

x

실수 2: 학습 데이터로 테스트하기

- **현상:** 학습 데이터에 대해서는 정확도가 100%인데, 새로운 데이터를 넣으면 엉망인 경우.
- **원인:** 과적합(Overfitting)입니다. 모델이 정답을 이해한 것이 아니라, 데이터를 통째로 외워 버린 것입니다.
- **해결:** train_test_split을 통해 학습 데이터와 테스트 데이터를 엄격히 분리하고, 학습에 사용하지 않은 데이터로 성능을 검증하세요.

x

실수 3: 단순한 모델로 복잡한 데이터 분류하기

- **현상:** 로지스틱 회귀를 썼는데 정확도가 너무 낮게 나옴.
- **원인:** 데이터의 경계가 직선으로 나누어지지 않는 '비선형' 구조일 때, 단순한 모델은 한계가 있습니다.
- **해결:** 결정 트리나 랜덤 포레스트처럼 더 복잡한 경계를 그릴 수 있는 알고리즘을 시도해 보세요.

6. 실무 활용 사례

분류 모델은 우리 주변 어디에나 존재합니다.

1. 금융 서비스 (이상 거래 탐지 - FDS):

- 사용자의 평소 결제 패턴과 다른 갑작스러운 고액 결제가 발생했을 때, 이를 '정상'과 '이상(사기)'으로 분류하여 결제를 일시 차단합니다.

2. 의료 진단 (질병 유무 판별):

- MRI 영상이나 혈액 검사 수치를 바탕으로 특정 질환이 있는지 없는지를 분류합니다.

3. 전자상거래 (고객 이탈 예측):

- 최근 접속 빈도, 구매 금액 등의 데이터를 분석해 이 고객이 계속 서비스를 이용할지, 아니면 탈퇴할지(이탈 여부)를 분류하여 타겟 마케팅을 수행합니다.

4. 콘텐츠 모더레이션 (유해 콘텐츠 차단):

- 커뮤니티에 올라오는 게시글의 텍스트를 분석해 욕설이나 혐오 표현이 포함되었는지 분류하여 자동으로 가립니다.

7. 요약 및 정리

- **분류(Classification)**는 데이터가 어떤 카테고리에 속하는지 결정하는 지도학습의 한 종류입니다.
- **이진 분류**는 두 가지 중 하나를, **다중 분류**는 여러 가지 중 하나를 선택합니다.
- **로지스틱 회귀**는 확률 기반, **결정 트리**는 규칙 기반, **랜덤 포레스트**는 다수결 기반의 분류 모델입니다.
- 텍스트 데이터를 분류할 때는 반드시 **수치화(Vectorization)** 과정이 필요합니다.
- 데이터가 불균형할 때는 **정확도(Accuracy)**뿐만 아니라 **정밀도(Precision)**와 **재현율(Recall)**을 함께 확인해야 합니다.

이제 여러분은 데이터를 통해 그룹을 나누는 '분류'의 개념을 익히고, 실제로 스팸 메일 분류기까지 구현해 보았습니다. 다음 장에서는 정답(Label)이 없는 데이터를 다루는 머신러닝의 또 다른 영역, '비지도학습'에 대해 알아보겠습니다.

챕터 9: 비지도학습과 군집화

9장. 비지도학습과 군집화

지금까지 우리는 '정답(Label)'이 주어진 데이터를 가지고 학습하는 지도학습(Supervised Learning)에 대해 배웠습니다. 집값을 예측하는 회귀, 스팸 메일을 걸러내는 분류 모두 "이 데이터의 정답은 이것이다"라고 알려주는 선생님이 있는 학습 방식이었습니다.

하지만 현실 세계의 데이터 중에는 정답이 없는 경우가 훨씬 더 많습니다. 수만 명의 고객 구매 데이터가 있지만, 이들이 구체적으로 어떤 유형의 고객인지 분류되어 있지 않은 경우가 대표적입니다.

이때 필요한 것이 바로 비지도학습(Unsupervised Learning)입니다.

9.1 정답 없이 스스로 패턴을 찾는 비지도학습

비유: 아이의 블록 놀이

어린아이가 처음 블록 놀이를 한다고 상상해 보세요. 부모가 옆에서 "이건 빨간색이야", "이건 세모 모양이야"라고 가르쳐주지 않아도, 아이는 어느 순간 스스로 빨간색 블록들끼리 모아두고, 세모 모양 블록들끼리 모아둡니다.

아이는 '빨간색'이라는 단어나 '세모'라는 개념(정답)을 배운 것이 아닙니다. 그저 "얘네들은 서로 비슷하게 생겼네?"라는 특징을 발견하고 스스로 그룹을 나눈 것입니다. 이것이 바로 비지도학습의 핵심입니다.

직관: 데이터의 '숨은 구조' 발견하기

비지도학습은 정답(Label) 없이 데이터 자체의 특성만을 분석하여 데이터 속에 숨겨진 패턴이나 구조를 찾아내는 방법입니다.

지도학습이 "이 사진은 고양이인가요?"라는 질문에 답하는 과정이라면, 비지도학습은 "이 데이터들 중에서 서로 비슷하게 생긴 애들끼리 묶어보세요"라고 요청하는 것과 같습니다. 이렇게 비슷한 특성을 가진 데이터끼리 묶는 것을 군집화(Clustering)라고 합니다.

기술 설명: 비지도학습과 군집화

비지도학습의 가장 대표적인 분야가 바로 군집화입니다. 군집화는 전체 데이터셋을 유사한 특성을 가진 여러 개의 그룹(Cluster)으로 나누는 과정입니다.

- **특징:** 정답 데이터(y)가 없으며, 오직 입력 데이터(x)만 사용합니다.
 - **목적:** 데이터의 내재된 구조 파악, 데이터 압축, 이상치 탐지, 고객 세분화 등.
 - **핵심 원리:** 대부분의 군집화 알고리즘은 '거리(Distance)' 개념을 사용합니다. 데이터 포인트 사이의 거리가 가까울수록 서로 비슷하다고 판단하는 것입니다.
-

9.2 K-Means 군집화: 가장 단순하고 강력한 도구

비지도학습의 여러 알고리즘 중 가장 널리 쓰이는 것이 바로 **K-Means(K-평균)** 알고리즘입니다.

비유: 마을의 중심지 정하기

어느 넓은 벌판에 100명의 사람들이 흩어져 살고 있습니다. 이들을 효율적으로 관리하기 위해 K 개의 마을 센터(중심지)를 세우려고 합니다.

1. 먼저 아무 곳이나 센터 K 개를 무작위로 세웁니다.
2. 사람들은 각자 자신에게서 가장 가까운 센터로 가서 소속됩니다.
3. 센터 관리자는 자기 마을에 소속된 사람들의 정중앙 위치로 센터를 옮깁니다.
4. 센터가 이동했으니, 사람들은 다시 한번 가장 가까운 센터가 어디인지 확인하고 소속을 바꿉니다.
5. 더 이상 센터의 위치가 변하지 않을 때까지 이 과정을 반복합니다.

결국, 사람들은 가장 효율적인 중심지를 기준으로 몇 개의 그룹으로 나뉘게 됩니다.

직관: 중심점을 향한 수렴

K-Means의 핵심은 '**중심점(Centroid)**'입니다. - **K**: 우리가 나누고 싶은 그룹의 개수입니다. - **Means**: 각 그룹의 평균 위치(중심점)를 의미합니다.

데이터들이 중심점에 가까워지도록 계속해서 위치를 조정하면서, 결과적으로 데이터 간의 거리 합이 최소가 되는 최적의 그룹을 찾아가는 과정입니다.

기술 설명: K-Means 작동 단계

K-Means 알고리즘은 다음과 같은 반복적인 단계를 거칩니다.

1. **초기화**: 사용자가 지정한 K 개의 중심점을 무작위로 배치합니다.
2. **할당(Assignment)**: 모든 데이터 포인트에 대해 가장 가까운 중심점을 찾아 해당 군집으로 할당합니다. (유클리드 거리 사용)
3. **업데이트(Update)**: 각 군집에 속한 데이터들의 산술 평균 위치를 계산하여 중심점을 새로운 위치로 이동시킵니다.
4. **반복**: 중심점의 위치가 더 이상 변하지 않거나, 설정한 최대 반복 횟수에 도달할 때까지 2~3 단계를 반복합니다.

[그림 9-1: K-Means 작동 프로세스] - 그림 설명: 4단계의 흐름도로 구성. - (1) 무작위 중심점 배치 $\rightarrow$ (2) 데이터들이 각 중심점에 색상별로 할당되는 모습 $\rightarrow$ (3) 중심점이 할당된 데이터들의 중앙으로 이동하는 화살표 표시 $\rightarrow$ (4) 중심점이 고정되어 더 이상 움직이지 않는 최종 상태를 시각적으로 표현.

9.3 파이썬으로 구현하는 K-Means 군집화

이제 scikit-learn 라이브러리를 사용하여 실제로 데이터를 군집화해 보겠습니다. 가상의 고객 데이터를 만들어 구매 금액과 방문 빈도에 따라 고객을 그룹핑하는 실습을 진행합니다.

실습 코드: 고객 세그먼트 분석

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# 1. 가상 데이터 생성
# n_samples: 데이터 개수, centers: 군집 개수, cluster_std: 군집의 표준편차(퍼짐 정도)
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=42)

# 데이터의 의미 부여: X[:, 0] -> 연간 소비 금액, X[:, 1] -> 연간 방문 횟수
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], s=30, color='gray')
plt.title("Raw Customer Data")
plt.xlabel("Annual Spending")
plt.ylabel("Visit Frequency")
plt.show()

# 2. K-Means 모델 생성 및 학습
# k=4로 설정하여 4개의 그룹으로 나눕니다.
kmeans = KMeans(n_clusters=4, init='k-means++', random_state=42)
kmeans.fit(X)

# 3. 예측 및 중심점 획득
labels = kmeans.predict(X) # 각 데이터가 어느 군집에 속하는지 결과
centroids = kmeans.cluster_centers_ # 최종 결정된 중심점 좌표

# 4. 결과 시각화
plt.figure(figsize=(8, 6))

# 군집별로 색상을 다르게 하여 출력
colors = ['red', 'blue', 'green', 'purple']
for i in range(4):
    plt.scatter(X[labels == i, 0], X[labels == i, 1], s=30, c=colors[i], label=f'Cluster {i+1}')

# 중심점 시각화
```

```
plt.scatter(centroids[:, 0], centroids[:, 1], s=200, c='yellow', marker='X',
            edgecolors='black', label='Centroids')

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Spending")
plt.ylabel("Visit Frequency")
plt.legend()
plt.show()

print("최종 중심점 좌표:\n", centroids)
```

[코드 실행 결과]

```
최종 중심점 좌표:
[[-2.52451234  4.71234567]
 [ 4.61234567  4.82345678]
 [-2.12345678 -2.3456789 ]
 [ 4.82345678 -2.12345678]]
(※ 위 수치는 예시이며, 실행 환경에 따라 소수점 자리가 다를 수 있습니다.)
```

결과 해석

1. **산점도 분석:** 회색으로 흩어져 있던 데이터들이 4가지 색상의 그룹으로 명확하게 구분되었습니다.
2. **중심점(X 표시):** 노란색 X 표시는 각 그룹의 '평균' 위치입니다. 모든 데이터 포인트는 자신과 가장 가까운 X 표시에 할당되었습니다.
3. **비즈니스적 의미 부여 (분석가의 해석 영역):** 주의: K-Means 알고리즘은 단순히 수학적 거리만을 계산하여 그룹을 나눌 뿐, 각 그룹이 무엇을 의미하는지 자동으로 알려주지 않습니다. 각 그룹에 어떤 이름을 붙이고 어떤 의미를 부여할지는 분석가가 데이터의 특성을 보고 결정하는 '해석의 영역'입니다.
4. **Cluster 1 (예: 빨강):** 소비 금액은 낮지만 방문 횟수가 많은 '충성 고객'
5. **Cluster 2 (예: 파랑):** 소비 금액은 높지만 방문 횟수가 적은 'VIP 고객'
6. **Cluster 3 (예: 초록):** 소비와 방문 모두 낮은 '잠재 고객'
7. **Cluster 4 (예: 보라):** 소비와 방문 모두 높은 '최우수 고객'

9.4 적절한 K값 찾기: 엘보우(Elbow) 방법

실무에서는 데이터를 처음 볼 때 그룹을 몇 개(K)로 나눠야 할지 알 수 없습니다. K를 너무 적게 잡으면 너무 뭉뚱그려 분석하게 되고, 너무 많게 잡으면 과하게 세분화되어 의미가 없어집니다. 이때 사용하는 것이 **엘보우 방법(Elbow Method)**입니다.

직관: 꺾이는 지점을 찾아라

군집의 개수(K)가 늘어날수록 데이터와 중심점 사이의 거리는 당연히 짧아집니다. 하지만 어느 순간부터는 K 를 늘려도 거리 감소 폭이 급격히 줄어드는 지점이 생깁니다. 마치 팔꿈치(Elbow)처럼 툭 꺾이는 그 지점이 가장 효율적인 K 값이라는 논리입니다.

기술 설명: 관성(Inertia)

K-Means에서 사용하는 평가 지표인 **관성(Inertia)**은 '각 데이터 포인트와 해당 군집 중심점 사이의 거리의 제곱합'을 의미합니다. 관성이 낮을수록 군집화가 촘촘하게 잘 되었다는 뜻입니다.

[그림 9-2: 엘보우 그래프의 형태] - 그림 설명: X축은 '군집 수(K)', Y축은 '관성(Inertia)'인 선 그래프. $K=1$ 에서 $K=4$ 까지는 선이 가파르게 내려오다가, $K=4$ 지점에서 꺾여 이후부터는 완만하게 감소하는 모습. $K=4$ 지점에 'Elbow Point'라고 표시하여 최적의 K 임을 강조.

실습 코드: 엘보우 방법 구현

```
inertia = []
K_range = range(1, 11)

for k in K_range:
    km = KMeans(n_clusters=k, init='k-means++', random_state=42)
    km.fit(X)
    inertia.append(km.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(K_range, inertia, marker='o')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.xticks(K_range)
plt.grid(True)
plt.show()
```

결과 해석: 그래프를 보면 $K=1$ 에서 $K=4$ 까지는 관성 값이 급격히 떨어지지만, $K=4$ 이후부터는 완만하게 감소합니다. 따라서 이 데이터의 최적의 군집 수는 4라고 판단할 수 있습니다.

9.5 실무 활용 사례: 고객 세그먼테이션(Segmentation)

기업들은 K-Means를 단순한 실습이 아니라 실제 마케팅 전략 수립에 사용합니다. 이를 **고객 세그먼테이션**이라고 합니다.

[실무 적용 프로세스] 1. 데이터 수집: 고객의 구매 주기, 평균 구매 금액, 최근 방문일(RFM 데이터)을 수집합니다. 2. 전처리: 금액 단위와 횟수 단위가 다르므로 반드시 **표준화(Standardization)**를 진행합니다. (매우 중요!) 3. **K-Means 적용**: 엘보우 방법을 통해 최적의 고객 그룹 수를 결정하고 군집화합니다. 4. **페르소나 정의**: 각 그룹의 중심점 특징을 분석하여 "알뜰 쇼핑족", "충동 구매족", "VIP" 등의 이름을 붙입니다. 5. **맞춤형 전략**: - 알뜰 쇼핑족 $\rightarrow$ 할인 쿠폰 발송 - VIP $\rightarrow$ 전담 매니저 배치 및 프리미엄 서비스 제공

9.6 초보자가 자주 하는 실수와 해결 방법

실수 1: 데이터 스케일링(Scaling)을 생략하는 경우

K-Means는 **유클리드 거리**를 기반으로 작동합니다. 만약 '연봉(단위: 만 원, 0~10,000)'과 '나이(단위: 세, 0~100)'라는 두 가지 특성을 그대로 사용하면, 숫자가 훨씬 큰 연봉 데이터가 거리 계산을 지배하게 됩니다. 나이 차이가 20살나더라도 연봉 차이가 1만 원만 나면 모델은 연봉 차이를 훨씬 중요하게 생각합니다.

- **해결책**: 반드시 StandardScaler나 MinMaxScaler를 사용하여 모든 특성의 범위를 비슷하게 맞춰 주어야 합니다.

실수 2: 초기 중심점 설정에 따른 결과 변동

K-Means는 처음에 중심점을 무작위로 잡습니다. 운이 나쁘게 초기 위치가 이상한 곳에 잡히면, 최적의 군집을 찾지 못하고 엉뚱한 결과가 나올 수 있습니다.

- **해결책**: scikit-learn의 KMeans는 기본적으로 `init='k-means++'` 옵션을 사용합니다. 이는 중심점을 무작위로 잡는 것이 아니라, 서로 멀리 떨어지도록 똑똑하게 배치하는 알고리즘으로, 초기 값 문제를 상당 부분 해결해 줍니다.

실수 3: 이상치(Outlier)의 영향

중심점은 '평균(Mean)'을 사용합니다. 평균의 치명적인 단점은 아주 큰 값(이상치) 하나가 전체 평균을 확 끌어당긴다는 것입니다. 데이터에 말도 안 되게 큰 값이 섞여 있으면 중심점이 그쪽으로 쏠려 군집화 결과가 왜곡됩니다.

- **해결책**: 분석 전 이상치를 제거하거나, 평균 대신 중앙값을 사용하는 K-Medoids 같은 다른 알고리즘을 고려해야 합니다.
-

9.7 요약

개념	설명	핵심 포인트
비지도학습	정답 없이 데이터의 패턴을 스스로 찾는 학습 방식	정답(y) 없음, 숨은 구조 발견
군집화 (Clustering)	비슷한 특성을 가진 데이터끼리 그룹으로 묶는 것	거리 기반 유사도 측정
K-Means	K 개의 중심점을 잡고 거리 기반으로 그룹화하는 알고리즘	할당 $\rightarrow$ 업데이트 반복
엘보우 방법	관성(Inertia) 그래프의 꺾이는 지점을 통해 최적의 K 를 찾는 법	Inertia의 급격한 감소 지점 확인
주의사항	거리 기반 알고리즘이므로 스케일링과 이상치 처리가 필수적임	StandardScaler 사용 권장

챕터 10: 모델 평가와 성능 개선

10장. 모델 평가와 성능 개선

머신러닝 모델을 만드는 과정은 마치 학생이 시험을 준비하는 과정과 비슷합니다. 교과서(학습 데이터)를 열심히 공부해서 문제를 풀 수 있게 되었다더라도, 정작 중요한 것은 '한 번도 본 적 없는 새로운 문제(테스트 데이터)'를 얼마나 잘 맞히느냐 하는 것입니다.

만약 학생이 문제의 답을 통째로 외워버렸다면, 교과서에 있는 문제는 다 맞히겠지만 실제 시험에서는 빵점을 맞을 것입니다. 머신러닝에서도 이런 현상이 발생하는데, 이를 '과적합(Overfitting)'이라고 합니다. 따라서 우리는 모델이 정말로 실력을 갖췄는지, 아니면 단순히 답을 외운 것인지 판단할 수 있는 정교한 '평가 지표'와 '검증 방법'이 필요합니다.

1. 모델 평가의 시작: 혼동 행렬 (Confusion Matrix)

비유로 이해하기: "범인 찾기"

경찰이 용의자 100명을 대상으로 범인인지 아닌지를 판별하는 AI 모델을 만들었다고 가정해 봅시다.

- **정답(Actual):** 실제로 범인인 사람과 무고한 사람.
- **예측(Predict):** AI가 "이 사람은 범인이다"라고 지목한 사람과 "무고하다"라고 판단한 사람.

이때 AI는 네 가지 상황에 놓이게 됩니다. 1. 범인을 범인이라고 정확히 맞힌 경우 (정답!) 2. 무고한 사람을 무고하다고 정확히 맞힌 경우 (정답!) 3. 무고한 사람을 범인이라고 오해한 경우 (억울한 피해자 발생) 4. 범인을 무고하다고 놓쳐버린 경우 (범인이 도주함)

직관적으로 이해하기

모델의 성능을 단순히 "전체 중 몇 개 맞혔나?"로만 따지면, 위에서 발생한 '억울한 피해자'와 '놓쳐버린 범인'의 차이를 구분할 수 없습니다. 어떤 상황에서는 범인을 놓치는 것보다 무고한 사람을 범인으로 모는 것이 더 치명적일 수 있고, 어떤 상황에서는 그 반대가 더 치명적일 수 있기 때문입니다.

그래서 우리는 "어떤 식으로 틀렸는가"를 한눈에 보여주는 표, 즉 **혼동 행렬**을 사용합니다.

기술 설명

혼동 행렬은 실제 클래스와 모델이 예측한 클래스 간의 관계를 나타낸 행렬입니다. 이진 분류(Binary Classification)를 기준으로 다음과 같이 정의합니다.

구분	예측: Positive (참)	예측: Negative (거짓)
실제: Positive (참)	TP (True Positive)	FN (False Negative)
실제: Negative (거짓)	FP (False Positive)	TN (True Negative)

- TP (True Positive): 실제 Positive를 Positive라고 맞게 예측함.
- TN (True Negative): 실제 Negative를 Negative라고 맞게 예측함.
- FP (False Positive): 실제 Negative인데 Positive라고 틀리게 예측함. (Type I Error)
- FN (False Negative): 실제 Positive인데 Negative라고 틀리게 예측함. (Type II Error)

2. 정확도 (Accuracy): 가장 단순하지만 위험한 지표

비유로 이해하기: "99%의 함정"

어떤 마을에 희귀병이 있는데, 주민 1,000명 중 단 10명만 이 병을 앓고 있습니다. 한 의사가 아주 단순한 진단법을 만들었습니다. 그냥 모든 사람에게 "당신은 건강합니다(Negative)"라고 말하는 것입니다.

이 의사의 진단 정확도는 얼마일까요? 1,000명 중 990명은 실제로 건강하므로, 이 의사는 99%의 정확도를 기록하게 됩니다. 하지만 이 의사가 정말 유능한가요? 전혀 아닙니다. 정작 치료가 필요한 10명의 환자를 단 한 명도 찾아내지 못했기 때문입니다.

직관적으로 이해하기

정확도는 "전체 예측 중 맞힌 비율"입니다. 데이터의 비율이 비슷할 때는 유용하지만, 위 사례처럼 데이터 불균형(Imbalanced Data)이 심한 경우, 모델이 다수 클래스로만 예측해도 정확도가 높게 나오는 착시 현상이 발생합니다.

기술 설명

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

3. 정밀도(Precision)와 재현율(Recall)

정확도의 함정을 피하기 위해 우리는 정밀도와 재현율이라는 두 가지 지표를 사용합니다.

비유로 이해하기: "스팸 메일함 vs 암 진단"

1. 정밀도 (Precision): "신중함의 척도" 스팸 메일 필터를 생각해 보세요. 정상적인 메일(중요한 업무 메일)을 스팸으로 분류해버리면 사용자는 매우 곤란해집니다. 따라서 스팸 필터는 "스팸이라

고 예측한 것들 중에서 실제로 스팸인 비율"이 매우 높아야 합니다. 즉, 매우 신중하게 스팸을 골라 내야 합니다. 이것이 바로 정밀도입니다.

2. 재현율 (Recall): "꼼꼼함의 척도" 암 진단 AI를 생각해 보세요. 암 환자를 건강하다고 판단해서 돌려보내면 환자의 생명이 위험해집니다. 따라서 암 진단 모델은 "실제 암 환자들 중에서 얼마나 많이 찾아냈는가"가 훨씬 중요합니다. 설령 건강한 사람을 암이라고 의심해 정밀 검사를 하게 만들더라도(FP 증가), 실제 환자를 놓쳐서는 안 됩니다(FN 감소). 이것이 바로 재현율입니다.

직관적으로 이해하기

- 정밀도: 모델이 "맞다"고 한 것 중 진짜 맞은 비율 $\rightarrow$ FP(가짜 양성)를 줄이는 것이 목표
- 재현율: 실제 "맞는" 것들 중 모델이 맞다고 찾아낸 비율 $\rightarrow$ FN(가짜 음성)을 줄이는 것이 목표

기술 설명

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

4. F1 Score: 정밀도와 재현율의 조화

비유로 이해하기: "시소 게임"

정밀도와 재현율은 마치 시소와 같습니다. 정밀도를 높이려고 매우 신중하게 예측하면(FP 감소), 정작 찾아내야 할 대상들을 놓치게 되어 재현율이 떨어집니다. 반대로 재현율을 높이려고 꼼꼼하게 다 훑으면(FN 감소), 무고한 대상을 포함시킬 확률이 높아져 정밀도가 떨어집니다.

우리는 이 두 지표가 어느 한쪽으로 치우치지 않고 균형을 이룬 상태를 원합니다. 이때 사용하는 것이 F1 Score입니다.

직관적으로 이해하기

F1 Score는 정밀도와 재현율의 **조화 평균**입니다. 산술 평균이 아니라 조화 평균을 사용하는 이유는, 두 값 중 어느 하나가 매우 낮을 때 전체 점수를 확 낮춰서 "균형 잡히지 않은 모델"임을 드러내기 위해서입니다.

기술 설명

$$F1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

5. [실습] 모델 평가 지표 구현하기

이제 앞서 배운 개념들을 파이썬 코드로 직접 구현해 보겠습니다. scikit-learn 라이브러리를 사용하여 가상의 분류 데이터를 만들고 모델을 평가해 보겠습니다.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score,
f1_score, classification_report
from sklearn.datasets import make_classification

# 1. 데이터 생성 (불균형 데이터 생성: 90%는 0, 10%는 1)
X, y = make_classification(n_samples=1000, n_features=20, weights=[0.9, 0.1], random_state=42)

# 2. 학습 데이터와 테스트 데이터 분리
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

# 3. 모델 학습 (로지스틱 회귀)
model = LogisticRegression()
model.fit(X_train, y_train)

# 4. 예측 수행
y_pred = model.predict(X_test)

# 5. 평가 지표 계산
conf_matrix = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("--- 혼동 행렬 (Confusion Matrix) ---")
print(conf_matrix)
print(f"\n정확도 (Accuracy): {accuracy:.4f}")
print(f"정밀도 (Precision): {precision:.4f}")
print(f"재현율 (Recall): {recall:.4f}")
print(f"F1 Score: {f1:.4f}")

print("\n--- 상세 보고서 (Classification Report) ---")
print(classification_report(y_test, y_pred))
```

실행 결과 해석

예상 결과 예시:

--- 혼동 행렬 (Confusion Matrix) ---

```
[[261  8]
 [ 5  4]]
```

정확도 (Accuracy): 0.9133

정밀도 (Precision): 0.3333

재현율 (Recall): 0.4444

F1 Score: 0.3774

--- 상세 보고서 (Classification Report) ---

	precision	recall	f1-score	support
0	0.97	0.97	0.97	269
1	0.33	0.44	0.38	9

결과 해석: 1. **정확도(0.9133)**는 매우 높게 나타납니다. 하지만 이는 데이터의 90%가 0이기 때문에 발생하는 현상입니다. 2. **정밀도(0.3333)**를 보면, 모델이 1이라고 예측한 12명(\$8+4\$) 중 실제 1인 사람은 4명뿐입니다. 즉, 정밀도가 매우 낮습니다. 3. **재현율(0.4444)**를 보면, 실제 1인 사람 9명 (\$5+4\$) 중 4명만 찾아냈습니다. 절반 이상을 놓친 셈입니다. 4. **결론:** 이 모델은 정확도는 높지만, 실제 소수 클래스(1)를 분류하는 성능은 매우 떨어지는 '함정에 빠진 모델'입니다.

6. 교차 검증 (Cross Validation)

비유로 이해하기: "모의고사 회독"

시험 공부를 할 때, 문제집의 1~80페이지는 공부하고 81~100페이지는 시험용으로 남겨두었습니다. 그런데 만약 81~100페이지에만 아주 특이하고 어려운 문제가 몰려 있다면 어떻게 될까요? 내 실력보다 점수가 낮게 나올 것입니다. 반대로 너무 쉬운 문제만 있다면 실력보다 점수가 높게 나오겠죠.

이런 '운'의 요소를 없애기 위해, 우리는 문제집을 5등분 하여 "1~4구역 공부 $\rightarrow$ 5구역 시험", "2~5구역 공부 $\rightarrow$ 1구역 시험" 식으로 순서를 바꿔가며 5번의 시험을 치릅니다. 그리고 그 평균 점수를 내는 것이 가장 정확한 내 실력이 됩니다.

직관적으로 이해하기

데이터를 단순히 Train/Test로 한 번만 나누면, 나누는 방식에 따라 성능이 들쭉날쭉할 수 있습니다. **교차 검증(특히 K-Fold 교차 검증)**은 데이터를 K 개의 묶음으로 나누어, 매번 다른 묶음을 테

스트 셋으로 사용하고 나머지를 학습 셋으로 사용하여 총 K 번 학습과 평가를 반복하는 방법입니다.

기술 설명

가장 대표적인 **K-Fold 교차 검증**의 과정은 다음과 같습니다.

[그림 10-1: K-Fold 교차 검증의 작동 원리] (그림 묘사: 전체 데이터를 가로로 긴 직사각형으로 표현하고 이를 5개의 동일한 블록(Fold 1~5)으로 나눕니다. 총 5개의 행을 그려, 각 행마다 서로 다른 블록 하나가 색깔이 다르게 표시(검증 셋)되고 나머지 네 블록은 같은 색(학습 셋)으로 표시되어 순차적으로 교체되는 모습을 보여줍니다.)

1. 전체 데이터를 K 개의 동일한 크기 그룹(Fold)으로 나눕니다.
2. 1번째 그룹을 검증 셋으로, 나머지 $K-1$ 개 그룹을 학습 셋으로 하여 모델을 학습하고 평가합니다.
3. 2번째 그룹을 검증 셋으로, 나머지를 학습 셋으로 하여 반복합니다.
4. 이 과정을 K 번 반복하여 K 개의 성능 점수를 얻습니다.
5. K 개 점수의 평균을 최종 성능으로 결정합니다.

7. [실습] 교차 검증 적용하기

scikit-learn의 `cross_val_score`를 사용하여 모델의 일반화 성능을 측정해 보겠습니다.

주의할 점: 앞선 실습에서는 데이터를 `train_test_split`으로 나누어 한 번만 평가했지만, 교차 검증은 내부적으로 데이터를 여러 번 나누어 학습과 평가를 반복합니다. 따라서 여기서는 전체 데이터 (X, y)를 넣어 `cross_val_score`가 스스로 데이터를 쪼개어 검증하도록 하겠습니다. (실무에서는 보통 Train set 내에서만 교차 검증을 수행하여 최적의 설정을 찾고, 최종 성능은 따로 떼어둔 Test set으로 단 한 번 확인하는 것이 정석입니다.)

```
from sklearn.model_selection import cross_val_score

# 모델 생성
model = LogisticRegression()

# K-Fold 교차 검증 수행 (K=5)
# scoring='f1'으로 설정하여 불균형 데이터에 적합한 F1 Score를 측정
scores = cross_val_score(model, X, y, cv=5, scoring='f1')

print(f"각 폴드별 F1 Score: \n{scores}")
print(f"\n평균 F1 Score: {np.mean(scores):.4f}")
print(f"표준 편차: {np.std(scores):.4f}")
```

실행 결과 해석

- **각 폴드별 점수:** 5번의 실험 결과가 출력됩니다. 점수들이 서로 비슷하다면 모델이 데이터의 특정 부분에 의존하지 않고 안정적으로 학습되었다는 뜻입니다.
- **평균 점수:** 이 값이 모델의 진짜 실력(일반화 성능)이라고 볼 수 있습니다.
- **표준 편차:** 점수들의 편차가 크다면, 데이터의 구성에 따라 모델의 성능이 크게 변한다는 의미이므로 주의가 필요합니다.

×

8. 초보자가 자주 하는 실수와 해결 방법

실수 1: 학습 데이터로 성능 평가하기

가장 흔한 실수입니다. 모델을 학습시킨 데이터로 다시 성능을 측정하면, 모델이 답을 외웠을 경우 (과적합) 말도 안 되게 높은 점수가 나옵니다. - **해결책:** 반드시 `train_test_split`을 통해 학습 데이터와 평가 데이터를 완전히 분리하세요.

실수 2: 불균형 데이터에서 '정확도'만 믿기

앞서 살펴본 것처럼, 데이터 비율이 깨져 있을 때 정확도는 아무런 의미가 없습니다. - **해결책:** `classification_report`를 출력하여 정밀도, 재현율, F1 Score를 함께 확인하세요.

실수 3: 데이터 누수 (Data Leakage)

전처리(정규화, 표준화 등)를 전체 데이터에 먼저 적용한 뒤 데이터를 나누는 경우입니다. 테스트 데이터의 정보가 학습 과정에 스며들어 성능이 뻥튀기되는 현상이 발생합니다. - **해결책:** 데이터를 먼저 나눈 후, 학습 데이터의 통계량(평균, 표준편차 등)을 기준으로 테스트 데이터를 변환해야 합니다.

×

9. 요약 및 실무 활용 가이드

핵심 요약

- **혼동 행렬:** 예측 결과와 실제 정답의 관계를 나타낸 표.
- **정확도:** 전체 중 맞힌 비율. 데이터 불균형 시 위험함.
- **정밀도:** 모델이 Positive라고 한 것 중 실제 Positive 비율 (신중함).
- **재현율:** 실제 Positive 중 모델이 찾아낸 비율 (꼼꼼함).
- **F1 Score:** 정밀도와 재현율의 조화 평균 (균형).
- **교차 검증:** 데이터를 여러 번 나누어 평가하여 모델의 일반화 성능을 확인하는 방법.

실무 활용 팁

실무에서 어떤 지표를 우선시해야 할까요? 서비스의 목적에 따라 다릅니다.

서비스 유형	우선 지표	이유
스팸 메일 차단	정밀도 (Precision)	중요한 메일을 스팸으로 처리하면 큰 손실이 발생함.
암/질병 진단	재현율(Recall)	환자를 건강하다고 놓치면 생명이 위험함.
카드 부정 결제 탐지	F1 Score	부정 결제를 잡는 것(재현율)과 고객 불편을 줄이는 것(정밀도) 모두 중요함.
일반적인 분류	정확도 (Accuracy)	클래스 비율이 비슷하고, 오답의 치명도가 낮을 때 사용.

이제 여러분은 모델을 단순히 '만드는' 단계를 넘어, 그 모델이 '얼마나 믿을만한지'를 객관적으로 측정하고 개선할 수 있는 능력을 갖추게 되었습니다. 다음 장에서는 모델의 성능을 극대화하기 위한 '하이퍼파라미터 튜닝'에 대해 알아보겠습니다.

챕터 11: 과적합과 하이퍼파라미터 튜닝

11장. 과적합과 하이퍼파라미터 튜닝

머신러닝 모델을 만들다 보면 당혹스러운 순간이 찾아옵니다. 학습 데이터에 대해서는 정확도가 99%나 나오는데, 막상 새로운 데이터를 넣으면 정확도가 60%로 뚝 떨어지는 현상입니다. "분명히 공부를 완벽하게 했는데, 왜 실전에서는 점수가 안 나오지?"라는 의문이 드는 시점입니다.

이번 장에서는 모델이 학습 데이터에 너무 과하게 몰입해 발생하는 '과적합'의 정체를 파헤치고, 모델의 성능을 최적으로 끌어올리기 위한 '하이퍼파라미터 튜닝' 방법을 학습하겠습니다.

1. 과적합(Overfitting)과 과소적합(Underfitting)

1.1 암기왕 학생의 함정 (비유)

시험 공부를 하는 두 학생이 있다고 가정해 봅시다.

- **학생 A (과적합)**: 교과서에 나온 연습문제의 정답과 풀이 과정을 토씨 하나 틀리지 않고 그대로 암기했습니다. 연습문제를 풀 때는 백점을 맞지만, 숫자가 조금만 바뀐 응용 문제가 나오면 완전히 손을 놓아버립니다.
- **학생 B (과적합의 반대, 과소적합)**: 공부를 너무 안 해서 기본적인 공식조차 모릅니다. 연습문제도 못 풀고, 실제 시험 문제도 당연히 풀지 못합니다.
- **학생 C (적정 학습)**: 공식의 원리를 이해하고 다양한 유형의 문제를 풀며 응용력을 키웠습니다. 연습문제에서도 좋은 점수를 받고, 처음 보는 시험 문제도 무리 없이 해결합니다.

여기서 학생 A가 바로 **과적합(Overfitting)** 상태의 모델입니다. 학습 데이터라는 '기출문제'에만 너무 최적화되어, 정작 중요한 '새로운 데이터에 대한 예측력(일반화 능력)'을 잃어버린 상태입니다.

1.2 직관으로 이해하는 과적합

머신러닝의 목적은 학습 데이터의 정답을 맞추는 것이 아니라, **학습하지 않은 새로운 데이터가 들어왔을 때 정답을 맞추는 것**입니다. 이를 '일반화(Generalization)'라고 합니다.

- **과적합(Overfitting)**: 모델이 너무 복잡해서 데이터의 본질적인 패턴뿐만 아니라, 무작위로 섞인 '노이즈(잡음)'까지 학습해버린 상태입니다.
- **과소적합(Underfitting)**: 모델이 너무 단순해서 데이터 속에 숨겨진 기본적인 패턴조차 찾아내지 못한 상태입니다.

1.3 기술적 설명

기술적으로 과적합은 **모델의 복잡도(Complexity)**와 관련이 있습니다.

구분	과소적합 (Underfitting)	적정 학습 (Balanced)	과적합 (Overfitting)
모델 복잡도	너무 낮음 (단순함)	적절함	너무 높음 (복잡함)
훈련 데이터 성능	낮음	높음	매우 높음
테스트 데이터 성능	낮음	높음	낮음
특징	편향(Bias)이 높음	편향과 분산의 조화	분산(Variance)이 높음

[그림 가이드: 과적합 시각화] - **과소적합**: 데이터 포인트들이 곡선 형태인데, 모델이 단순한 직선으로 그어버려 많은 점들이 멀리 떨어져 있는 모습. - **적정 학습**: 데이터의 흐름을 따라 부드러운 곡선이 그려져 있는 모습. - **과적합**: 모든 데이터 포인트를 하나하나 다 통과하기 위해 곡선이 매우 구불구불하고 날카롭게 꺾여 있는 모습.

2. 과적합을 어떻게 발견하고 방지할까?

2.1 검증 데이터셋의 도입

우리는 모델이 과적합되었는지 확인하기 위해 데이터를 쪼개어 사용합니다. 단순히 '훈련-테스트'로 나누는 것이 아니라, '훈련-검증-테스트'의 3단계로 나눕니다.

- 훈련 데이터(Train set)**: 모델을 학습시키는 데 사용합니다. (교과서 문제)
- 검증 데이터(Validation set)**: 학습 중인 모델의 성능을 평가하고 하이퍼파라미터를 수정하는 데 사용합니다. (모의고사)
- 테스트 데이터(Test set)**: 모든 튜닝이 끝난 후, 최종 성능을 딱 한 번 측정하는 데 사용합니다. (실제 수능 시험)

핵심: 훈련 데이터 점수는 계속 오르는데, 검증 데이터 점수가 어느 순간부터 떨어지기 시작한다면? 바로 그 지점이 **과적합**이 시작되는 지점입니다.

2.2 과적합 방지 전략

과적합을 막기 위한 대표적인 방법들은 다음과 같습니다.

- **데이터 양 늘리기**: 더 많은 데이터를 학습시키면 모델이 특정 노이즈에 집착하지 않고 일반적인 패턴을 찾게 됩니다.
- **모델 단순화**: 결정 트리의 깊이를 제한하거나, 신경망의 층을 줄이는 방법입니다.
- **정규화(Regularization)**: 가중치가 너무 커지지 않도록 페널티를 부여하는 기법입니다. (L1, L2 정규화)
- **드롭아웃(Dropout)**: 학습 시 무작위로 일부 뉴런을 끄는 방법입니다. (딥러닝에서 주로 사용)
- **조기 종료(Early Stopping)**: 검증 오차가 증가하기 시작하는 순간 학습을 멈추는 것입니다.

3. 하이퍼파라미터 튜닝 (Hyperparameter Tuning)

3.1 레시피 조절하기 (비유)

요리를 할 때, 재료(데이터)는 정해져 있지만 '불의 세기'나 '조리 시간', '소금의 양'은 요리사가 직접 결정해야 합니다. 불을 너무 세게 하면 타버리고(과적합), 너무 약하게 하면 안 익습니다(과소적합). 머신러닝에서도 마찬가지입니다. 모델이 스스로 학습하는 값(파라미터) 외에, 사람이 직접 설정해 줘야 하는 '설정값'이 있는데 이것이 바로 **하이퍼파라미터**입니다.

3.2 파라미터 vs 하이퍼파라미터

초보자가 가장 많이 혼동하는 개념입니다.

- **파라미터 (Parameter)**: 모델이 학습 데이터를 통해 스스로 찾아내는 값입니다. (예: 선형 회귀의 기울기와 절편, 신경망의 가중치)
- **하이퍼파라미터 (Hyperparameter)**: 모델 학습 시작 전에 사용자가 직접 지정하는 값입니다. (예: 결정 트리의 최대 깊이, 학습률, K-Means의 K값)

3.3 튜닝 방법: 그리드 서치와 랜덤 서치

최적의 하이퍼파라미터를 찾기 위해 모든 조합을 다 시도해볼 수는 없습니다. 이때 사용하는 효율적인 방법들이 있습니다.

1. **그리드 서치 (Grid Search)**: 지정한 후보 값들을 격자(Grid)처럼 모두 조합하여 시도하는 방식입니다. 꼼꼼하지만 시간이 매우 오래 걸립니다.
2. **랜덤 서치 (Random Search)**: 정해진 범위 내에서 무작위로 값을 선택해 시도하는 방식입니다. 그리드 서치보다 빠르며, 의외로 성능이 비슷하거나 더 좋은 경우가 많습니다.

4. 실습: 랜덤 포레스트 모델 최적화하기

이제 실제로 scikit-learn의 GridSearchCV를 사용하여 과적합을 방지하고 최적의 하이퍼파라미터를 찾아보겠습니다. 유방암 진단 데이터셋을 사용하여 분류 모델을 최적화해 보겠습니다.

4.1 파이썬 코드 구현

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```

# 1. 데이터 로드 및 분리
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# 훈련 세트와 테스트 세트로 분리 (8:2)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 2. 기본 모델 생성
rf = RandomForestClassifier(random_state=42)

# 3. 튜닝할 하이퍼파라미터 후보 설정
# n_estimators: 결정 트리의 개수
# max_depth: 트리의 최대 깊이 (너무 깊으면 과적합 위험)
# min_samples_split: 노드를 분할하기 위한 최소 샘플 수
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5, 10]
}

# 4. GridSearchCV 설정
# cv=5: 5-폴드 교차 검증 수행
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid,
                             cv=5, scoring='accuracy', n_jobs=-1)

# 5. 최적의 파라미터 찾기 (학습)
grid_search.fit(X_train, y_train)

# 6. 결과 확인
print(f"최적 하이퍼파라미터: {grid_search.best_params_}")
print(f"최적 모델의 검증 정확도: {grid_search.best_score_:.4f}")

# 7. 최적 모델로 테스트 데이터 평가
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
test_acc = accuracy_score(y_test, y_pred)

print(f"최종 테스트 데이터 정확도: {test_acc:.4f}")

# [비교] 튜닝 전 기본 모델 성능 확인

```

```
base_rf = RandomForestClassifier(random_state=42).fit(X_train, y_train)
base_acc = accuracy_score(y_test, base_rf.predict(X_test))
print(f"튜닝 전 기본 모델 정확도: {base_acc:.4f}")
```

4.2 실행 결과 해석

예상 출력 결과:

```
최적 하이퍼파라미터: {'max_depth': 5, 'min_samples_split': 2, 'n_estimators': 100}
최적 모델의 검증 정확도: 0.9612
최종 테스트 데이터 정확도: 0.9737
튜닝 전 기본 모델 정확도: 0.9649
```

결과 분석: 1. **최적 파라미터:** GridSearchCV가 $n_estimators=100$, $max_depth=5$ 일 때 가장 성능이 좋다는 것을 찾아냈습니다. 특히 max_depth 가 5로 제한된 것은, 무제한으로 깊게 학습하는 것보다 적당히 깊이를 제한하는 것이 일반화 성능에 더 유리했음을 의미합니다(과적합 방지). 2. **성능 향상:** 기본 모델보다 최적화된 모델의 테스트 정확도가 소폭 상승했거나 유지되는 것을 볼 수 있습니다. 데이터셋이 작을 때는 차이가 적을 수 있지만, 대규모 데이터셋에서는 하이퍼파라미터 튜닝이 성능을 결정짓는 핵심 요소가 됩니다.

5. 실무 활용 사례: 추천 시스템의 과적합

실무에서 과적합은 매우 빈번하게 발생합니다. 예를 들어, 넷플릭스와 같은 **추천 시스템**을 구축할 때 다음과 같은 과적합 문제가 발생할 수 있습니다.

- **상황:** 특정 사용자가 최근에 '공포 영화'를 3편 연속으로 봤다고 가정합니다.
- **과적합된 모델:** "이 사용자는 이제 공포 영화만 좋아하는구나!"라고 판단하여 모든 추천 리스트를 공포 영화로 도배합니다. 이는 사용자의 일시적인 취향 변화(노이즈)를 절대적인 패턴으로 오해한 과적합 사례입니다.
- **해결책:** 모델의 복잡도를 조절하거나, 과거의 장기적인 시청 기록(전체 데이터)의 비중을 높이는 정규화 기법을 적용하여 "평소에는 로맨틱 코미디를 좋아하지만, 지금 잠깐 공포 영화에 빠졌구나"라고 판단하게끔 일반화 능력을 높입니다.

x

6. 초보자가 자주 하는 실수와 해결 방법

실수 1: 테스트 데이터로 하이퍼파라미터 튜닝하기

가장 위험한 실수입니다. GridSearchCV를 사용할 때 테스트 세트를 포함해서 튜닝하거나, 테스트 세트의 점수를 보고 파라미터를 수동으로 수정하는 경우입니다. - **문제점:** 이는 시험 문제를 미리 보고 공부하는 것과 같습니다. 테스트 데이터에만 최적화된 모델이 되어, 실제 새로운 데이터가 들어왔을

* 때 성능이 폭락하는 '데이터 누수(Data Leakage)'가 발생합니다. - **해결책**: 하이퍼파라미터 튜닝은 반드시 **훈련 세트와 검증 세트**만으로 진행하고, 테스트 세트는 모든 과정이 끝난 후 마지막에 딱 한 번만 사용하세요.

실수 2: 무조건 복잡한 모델이 좋다고 생각하기

"층을 더 많이 쌓으면", "트리를 더 깊게 만들면" 성능이 무조건 올라갈 것이라고 생각하는 경우입니다. - **문제점**: 모델이 복잡해질수록 훈련 데이터의 정답률은 올라가지만, 과적합의 위험은 기하급수적으로 증가합니다. - **해결책**: '오컴의 면도날(Occam's Razor)' 원칙을 기억하세요. 비슷한 성능이라면 더 단순한 모델이 더 좋은 모델입니다.

요약

- **과적합(Overfitting)**은 모델이 학습 데이터의 노이즈까지 학습해 일반화 능력을 잃은 상태이며, **과소적합(Underfitting)**은 패턴을 충분히 학습하지 못한 상태입니다.
- 과적합을 발견하려면 **훈련 데이터와 검증 데이터의 성능 차이**를 확인해야 합니다.
- 과적합 방지를 위해 **데이터 증강, 모델 단순화, 정규화, 조기 종료** 등의 기법을 사용합니다.
- **하이퍼파라미터**는 사용자가 직접 설정하는 값이며, 이를 최적화하기 위해 **그리드 서치나 랜덤 서치**를 활용합니다.
- 튜닝 과정에서 **테스트 데이터가 학습 과정에 개입되지 않도록** 엄격히 분리하는 것이 가장 중요합니다.

챕터 12: 앙상블과 실전 머신러닝

12장. 앙상블과 실전 머신러닝

단일 모델만으로는 해결하기 어려운 복잡한 문제들이 있습니다. 아무리 뛰어난 전문가라도 혼자서는 놓치는 부분이 있듯이, 머신러닝 모델 하나만으로는 데이터의 모든 패턴을 완벽하게 잡아내기 어렵습니다. 이때 필요한 것이 바로 '앙상블(Ensemble)'입니다. 앙상블은 여러 개의 모델을 결합하여 더 강력한 하나의 모델을 만드는 기법입니다.

1. 앙상블의 개념: 집단지성의 힘

비유: 심사위원단의 판정

우리가 오디션 프로그램을 본다고 가정해 봅시다. 단 한 명의 심사위원이 합격 여부를 결정한다면, 그 심사위원의 개인적인 취향이나 편견이 결과에 큰 영향을 미칠 것입니다. 하지만 10명의 심사위원이 각각 점수를 매기고 그 평균값으로 합격자를 결정한다면 어떨까요? 특정 심사위원이 너무 엄격하거나 관대하더라도, 다른 심사위원들의 의견이 이를 보완하여 훨씬 객관적이고 정확한 판정이 내려질 가능성이 높습니다.

직관: 왜 여러 모델을 합치는가?

머신러닝 모델에는 항상 '편향(Bias)'과 '분산(Variance)'이라는 두 가지 숙제가 있습니다. - **편향**은 모델이 너무 단순해서 정답을 제대로 맞히지 못하는 상태(과소적합)를 말합니다. - **분산**은 모델이 학습 데이터에 너무 과하게 최적화되어 새로운 데이터에서는 엉뚱한 답을 내놓는 상태(과적합)를 말합니다.

앙상블의 핵심 아이디어는 "서로 다른 약점을 가진 모델들을 모아 그들의 예측값을 평균내거나 투표하게 함으로써, 개별 모델이 가진 오차를 상쇄시키는 것"입니다.

기술 설명: 앙상블의 주요 전략

앙상블 기법은 크게 세 가지 방향으로 나뉩니다.

1. **배깅(Bagging - Bootstrap Aggregating)**: 같은 알고리즘의 모델을 여러 개 만들되, 학습 데이터셋을 서로 다르게 구성하여 병렬적으로 학습시킨 후 결과를 합치는 방식입니다. (예: Random Forest)
2. **부스팅(Boosting)**: 모델을 순차적으로 학습시킵니다. 첫 번째 모델이 틀린 부분을 두 번째 모델이 집중적으로 학습하고, 또 그 틀린 부분을 세 번째 모델이 보완하는 방식입니다. (예: XGBoost, LightGBM)

3. **스태킹(Stacking)**: 서로 다른 종류의 모델(예: SVM, KNN, Decision Tree)들을 학습시킨 뒤, 그 예측값들을 다시 입력 데이터로 사용하여 최종 모델(Meta Learner)이 정답을 예측하게 하는 방식입니다.
-

2. 랜덤 포레스트(Random Forest): 든든한 나무들의 숲

비유: 전문가 집단의 다수결 투표

결정 트리(Decision Tree) 하나가 '나무'라면, 랜덤 포레스트는 수많은 결정 트리가 모인 '숲'입니다. 각 나무는 데이터의 일부만을 보고 판단을 내립니다. 어떤 나무는 나이와 수입에 집중하고, 어떤 나무는 거주지와 직업에 집중합니다. 이렇게 서로 다른 관점을 가진 나무들이 각자 예측값을 내놓으면, 최종적으로 가장 많은 표를 얻은 결과(다수결)를 선택합니다.

직관: 무작위성의 마법

결정 트리는 매우 강력하지만, 학습 데이터에 너무 민감하게 반응하여 과적합(Overfitting)이 일어나기 쉽다는 치명적인 단점이 있습니다. 랜덤 포레스트는 이 문제를 두 가지 '무작위성'으로 해결합니다. 1. **데이터의 무작위성(Bootstrapping)**: 전체 데이터에서 중복을 허용하여 무작위로 샘플을 뽑아 각 나무에게 줍니다. 나무마다 공부하는 교과서의 페이지가 조금씩 다른 셈입니다. 2. **특성의 무작위성(Feature Randomness)**: 나무가 가지를 칠 때 모든 특성을 고려하는 것이 아니라, 무작위로 선택된 일부 특성들 중에서 최적의 분할 기준을 찾습니다. 특정 강력한 특성 하나에 모든 나무가 의존하는 것을 방지하기 위함입니다.

기술 설명: 랜덤 포레스트의 작동 원리

• 학습 단계:

- 전체 데이터셋에서 중복 추출(Bootstrap)을 통해 N 개의 서로 다른 학습 세트를 만듭니다.
- 각 세트에 대해 결정 트리를 학습시킵니다. 이때 각 노드에서 분할에 사용할 특성을 무작위로 선택합니다.

• 예측 단계:

- 분류(Classification) 문제 $\rightarrow$ 각 트리의 예측값 중 최빈값(Majority Vote)을 선택합니다.
- 회귀(Regression) 문제 $\rightarrow$ 각 트리의 예측값의 평균(Average)을 계산합니다.

[도식 설명: 랜덤 포레스트 구조]

- Input Data** $\rightarrow$ (Bootstrapping) $\rightarrow$ [Tree 1, Tree 2, ..., Tree N] $\rightarrow$ (Voting/Average) $\rightarrow$ **Final Prediction**

- 각 Tree는 서로 다른 데이터 샘플과 서로 다른 특성 조합을 사용함을 시각적으로 표현합니다.

실습: 랜덤 포레스트로 붓꽃(Iris) 분류하기

이제 scikit-learn을 사용하여 랜덤 포레스트 모델을 구현해 보겠습니다. 단일 결정 트리와 성능을 비교하여 앙상블의 효과를 확인해 봅시다.

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. 데이터 로드 및 준비
iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# 2. 단일 결정 트리 모델 생성 및 평가
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)
dt_acc = accuracy_score(y_test, dt_pred)

# 3. 랜덤 포레스트 모델 생성 및 평가
# n_estimators: 만들 나무의 개수
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
rf_acc = accuracy_score(y_test, rf_pred)

print(f"단일 결정 트리 정확도: {dt_acc:.4f}")
print(f"랜덤 포레스트 정확도: {rf_acc:.4f}")

# 특성 중요도 확인
import matplotlib.pyplot as plt
import seaborn as sns
```

```
importances = rf_model.feature_importances_
feature_names = iris.feature_names
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)

plt.figure(figsize=(8, 5))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df)
plt.title('Feature Importances in Random Forest')
plt.show()
```

코드 결과 및 해석

- **실행 결과:** 일반적으로 랜덤 포레스트 정확도가 단일 결정 트리 정확도보다 높거나 비슷하게 나옵니다. 데이터셋이 작을 때는 차이가 적을 수 있지만, 복잡한 데이터일수록 랜덤 포레스트의 안정성이 빛을 발합니다.
- **특성 중요도(Feature Importance):** 랜덤 포레스트는 어떤 특성이 예측에 가장 큰 영향을 주었는지 알려주는 기능을 제공합니다. 위 그래프에서 가장 긴 막대를 가진 특성이 모델이 판단을 내릴 때 가장 중요하게 생각한 기준입니다.

3. XGBoost: 오답 노트를 활용한 정밀 학습

비유: 오답 노트를 쓰는 학생

랜덤 포레스트가 여러 명의 학생이 동시에 시험을 보고 다수결로 답을 정하는 방식이라면, XGBoost는 한 명의 학생이 시험을 보고, 틀린 문제를 오답 노트에 적어 집중 공부한 뒤 다시 시험을 치는 과정을 반복하는 방식입니다. - **1차 시험:** 기본 모델이 예측을 수행합니다. (당연히 많이 틀립니다.) - **오답 노트 작성:** 실제 정답과 예측값의 차이(잔차, Residual)를 계산합니다. - **2차 시험:** "어떻게 하면 이 오차를 줄일 수 있을까?"에 집중해서 새로운 모델을 학습시켜 기존 모델에 더합니다. - **반복:** 이 과정을 수백 번 반복하여 오차를 극한으로 줄입니다.

직관: 점진적인 개선 (Gradient Boosting)

XGBoost의 핵심은 '그라디언트 부스팅(Gradient Boosting)'입니다. 여기서 '그라디언트'는 수학적으로 경사 하강법을 의미하며, 쉽게 말해 "정답과의 거리(손실 함수)를 줄이는 방향으로 모델을 계속 업데이트한다"는 뜻입니다.

XGBoost(Extreme Gradient Boosting)는 기존의 부스팅 알고리즘을 극도로 최적화하여 다음과 같은 강점을 가집니다. 1. **속도:** 병렬 처리를 지원하여 매우 빠릅니다. 2. **규제(Regularization):** 모델이 너무 복잡해지지 않도록 제어하는 기능이 있어 과적합을 효과적으로 방지합니다. 3. **결측치 처리:** 데이터에 빈 값이 있어도 스스로 처리하는 알고리즘이 내장되어 있습니다.

기술 설명: XGBoost의 핵심 메커니즘

- **잔차 학습:** $Y_{\text{new}} = Y_{\text{old}} + \text{Learning Rate} \times \text{New Model}(\text{Residuals})$
- **학습률(Learning Rate):** 새로운 모델이 정답에 너무 급격하게 다가가지 않도록 조절하는 값입니다. 너무 크면 정답을 지나칠 수 있고(Overshooting), 너무 작으면 학습 시간이 너무 오래 걸립니다.
- **조기 종료(Early Stopping):** 검증 데이터의 성능이 더 이상 좋아지지 않으면 학습을 자동으로 멈춰 과적합을 막습니다.

실습: XGBoost로 고객 이탈 예측하기

이번에는 실무에서 많이 쓰이는 '고객 이탈 예측' 가상 시나리오를 통해 XGBoost를 적용해 보겠습니다.

```
# XGBoost 라이브러리 설치 필요: pip install xgboost
import xgboost as xgb
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# 1. 가상 데이터 생성 (고객 1000명, 특성 20개)
X, y = make_classification(
    n_samples=1000, n_features=20,
    n_informative=15, random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 2. XGBoost 모델 생성
# n_estimators: 반복 횟수 (오답 노트 횟수)
# learning_rate: 학습률 (보폭)
# max_depth: 각 나무의 최대 깊이
xgb_model = xgb.XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=5,
    random_state=42,
    use_label_encoder=False,
    eval_metric='logloss'
)
```

```

# 3. 모델 학습
xgb_model.fit(X_train, y_train)

# 4. 예측 및 평가
y_pred = xgb_model.predict(X_test)

print("--- XGBoost 성능 평가 ---")
print(classification_report(y_test, y_pred))

# 혼동 행렬 시각화
import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - XGBoost')
plt.show()

```

코드 결과 및 해석

- **Classification Report:** Precision(정밀도), Recall(재현율), F1-Score를 통해 모델의 성능을 다각도로 분석할 수 있습니다. 특히 이탈 예측에서는 '이탈할 사람을 놓치지 않는 것 (Recall)'이 중요하므로 이 지표를 유심히 살펴야 합니다.
- **Confusion Matrix:**
 - **True Positive(TP):** 이탈할 사람을 이탈한다고 잘 맞힘.
 - **False Positive(FP):** 유지할 사람을 이탈한다고 잘못 예측 (마케팅 비용 낭비).
 - **False Negative(FN):** 이탈할 사람을 유지한다고 잘못 예측 (고객 상실 - 가장 위험!).

4. 배깅(Random Forest) vs 부스팅(XGBoost) 비교

두 기법 모두 앙상블이지만 작동 방식과 목적이 다릅니다. 이를 표로 정리하면 다음과 같습니다.

구분	랜덤 포레스트 (Bagging)	XGBoost (Boosting)
학습 방식	병렬적 (동시에 여러 나무 학습)	순차적 (앞선 모델의 실수를 보완)
주요 목표	분산(Variance) 감소 $\rightarrow$ 과적합 방지	편향(Bias) 감소 $\rightarrow$ 정확도 향상

구분	랜덤 포레스트 (Bagging)	XGBoost (Boosting)
데이터 사용	무작위 샘플링 (Bootstrapping)	가중치 부여 (틀린 데이터에 집중)
학습 속도	빠름 (병렬 처리 가능)	상대적으로 느림 (순차 처리)
하이퍼파라미터	튜닝이 비교적 쉬움 (n_estimators 등)	튜닝이 까다로움 (learning_rate, max_depth 등)
추천 상황	데이터에 노이즈가 많고 과적합이 걱정될 때	최고의 예측 성능을 끌어내야 할 때

5. 실무 활용 사례: 신용 점수 예측 시스템

실제 금융권에서는 고객의 신용 점수를 예측하여 대출 승인 여부를 결정할 때 앙상블 모델을 적극적으로 사용합니다.

[실무 파이프라인 예시] 1. **데이터 수집**: 고객의 연 소득, 기존 대출 금액, 연체 횟수, 직업, 거주지 등의 데이터를 수집합니다. 2. **데이터 전처리**: 결측치를 처리하고, 범주형 변수(직업 등)를 인코딩합니다. 3. **모델 선택**: - 초기 단계에서는 **랜덤 포레스트**를 사용하여 어떤 변수가 신용 점수에 가장 큰 영향을 주는지(Feature Importance) 분석합니다. - 최종 서비스 단계에서는 **XGBoost**나 **LightGBM**을 사용하여 예측 정확도를 극대화합니다. 4. **검증**: K-폴드 교차 검증을 통해 모델이 특정 데이터셋에만 최적화되지 않았는지 확인합니다. 5. **해석**: SHAP(SHapley Additive exPlanations) 같은 라이브러리를 사용하여 "왜 이 고객의 신용 점수가 낮게 나왔는지"에 대한 근거를 제시합니다. (금융 서비스에서는 '설명 가능성'이 매우 중요하기 때문입니다.)

6. 초보자가 자주 하는 실수와 해결 방법

앙상블 모델을 사용할 때 입문자들이 흔히 겪는 시행착오들입니다.

1) "나무를 많이 만들수록(n_estimators $\uparrow$) 무조건 성능이 좋아지겠지?"

- **실수**: n_estimators 값을 10,000개, 100,000개로 무작정 높이는 경우.
- **결과**: 랜덤 포레스트는 어느 시점 이후 성능 향상이 멈추고 메모리만 많이 사용하게 됩니다. XGBoost의 경우 너무 많은 나무를 만들면 결국 학습 데이터에 과적합되어 테스트 성능이 떨어집니다.
- **해결**: 적절한 값을 찾기 위해 GridSearchCV나 RandomizedSearchCV를 사용하여 최적의 개수를 찾거나, XGBoost의 경우 early_stopping_rounds 옵션을 사용하여 성능 향상이 멈추는 지점에서 학습을 종료하세요.

2) "XGBoost 학습률(learning_rate)을 너무 높게 잡았어요."

- **실수**: learning_rate=0.5 또는 1.0으로 설정하는 경우.

- **결과:** 모델이 정답을 향해 너무 크게 점프하여 최적의 지점을 지나쳐 버립니다 (Overshooting). 결과적으로 학습이 불안정하고 성능이 낮게 나옵니다.
- **해결:** 일반적으로 0.01에서 0.1 사이의 작은 값을 설정하고, 대신 `n_estimators`를 충분히 늘려 천천히 정밀하게 학습시키는 것이 정석입니다.

3) "데이터 스케일링을 안 했는데 괜찮을까요?"

- **궁금증:** "앞 장에서 배운 표준화(Standardization)를 해야 하나요?"
 - **답변:** 랜덤 포레스트와 XGBoost 같은 트리 기반 모델은 데이터의 스케일에 영향을 받지 않습니다.
 - **이유:** 트리는 "값이 얼마인가"가 아니라 "특정 기준값보다 큰가 작은가"라는 이분법적 질문으로 데이터를 나누기 때문입니다. 따라서 스케일링을 하지 않아도 성능 차이가 거의 없습니다. (단, SVM이나 KNN과 함께 앙상블을 구성한다면 스케일링이 필수입니다.)
-

요약 및 마무리

이번 장에서는 단일 모델의 한계를 극복하는 **앙상블(Ensemble)** 기법에 대해 배웠습니다.

- **앙상블**은 여러 모델의 예측을 결합해 더 정확하고 안정적인 결과를 얻는 '집단지성' 전략입니다.
- **랜덤 포레스트**는 배깅(Bagging)의 대표 주자로, 무작위 샘플링과 특성 선택을 통해 과적합을 방지하고 안정적인 성능을 냅니다.
- **XGBoost**는 부스팅(Boosting)의 진화 형태로, 이전 모델의 오차를 보완하며 순차적으로 학습하여 매우 높은 예측 정확도를 자랑합니다.
- **실무**에서는 데이터의 특성과 목표(안정성 vs 정확도)에 따라 두 모델을 선택하거나 함께 활용합니다.

이제 여러분은 단일 모델을 넘어, 현대 머신러닝 경진대회(Kaggle 등)와 실무 현장에서 가장 널리 쓰이는 강력한 무기들을 갖추게 되었습니다. 데이터를 분석하고, 모델을 구축하고, 앙상블을 통해 성능을 끌어올리는 이 일련의 과정이 바로 머신러닝 엔지니어가 수행하는 핵심 작업입니다.

챕터 13: 실전 프로젝트 - 타이타닉 생존자 예측

13장. 실전 프로젝트 - 타이타닉 생존자 예측

지금까지 우리는 머신러닝의 개별 부품들을 하나씩 학습했습니다. 데이터 전처리 방법, 회귀와 분류 알고리즘, 모델 평가 지표, 그리고 성능을 높이는 앙상블 기법까지 말이죠. 하지만 실제 현업에서 머신러닝 엔지니어가 하는 일은 단순히 알고리즘 하나를 실행하는 것이 아닙니다.

데이터를 처음 마주한 순간부터 최종 예측 결과를 내놓기까지의 전체 과정, 즉 '**머신러닝 파이프라인(ML Pipeline)**'을 설계하고 실행하는 것이 핵심입니다. 이번 장에서는 머신러닝 입문자들의 '통과 의례'라고 불리는 **타이타닉 생존자 예측 프로젝트**를 통해, 지금까지 배운 모든 내용을 하나의 흐름으로 엮어보겠습니다.

1. 머신러닝 파이프라인: 탐정의 수사 과정

머신러닝 프로젝트를 수행하는 것은 마치 '**오래된 미제 사건을 해결하는 탐정의 수사 과정**'과 매우 비슷합니다.

비유로 이해하기

탐정이 사건을 해결할 때 무턱대고 범인을 지목하지 않습니다. 다음과 같은 단계를 거치죠. 1. **현장 조사 (EDA)**: 사건 현장을 둘러보며 어떤 단서가 있는지, 피해자와 가해자의 관계는 어떠했는지 파악합니다. 2. **증거 정제 (Preprocessing)**: 훼손된 지문을 복원하거나, 불필요한 잡음을 제거하여 깨끗한 증거물을 만듭니다. 3. **새로운 가설 설정 (Feature Engineering)**: "단순히 범인이 누구인가"가 아니라, "범인이 왼손잡이였을 가능성이 높다"라는 새로운 관점의 단서를 찾아냅니다. 4. **추리 및 검증 (Modeling & Evaluation)**: 수집한 증거를 바탕으로 범인을 추리하고, 그 추리가 맞는지 기존의 알리바이와 대조하며 검증합니다.

직관적으로 이해하기

머신러닝에서도 마찬가지입니다. 원본 데이터(Raw Data)를 그대로 모델에 넣는다고 해서 정답이 나오지 않습니다. 데이터를 이해하고, 다듬고, 의미 있는 특징을 추출하는 과정이 선행되어야 모델이 비로소 '학습'을 할 수 있습니다.

기술적 정의: 머신러닝 파이프라인

머신러닝 파이프라인이란 데이터 수집 $\rightarrow$ 데이터 분석(EDA) $\rightarrow$ 전처리 $\rightarrow$ 특성 공학 $\rightarrow$ 모델 학습 $\rightarrow$ 평가 $\rightarrow$ 배포로 이어지는 일련의 반복적인 과정을 의미합니다.

[그림 13-1. 머신러닝 파이프라인의 전체 흐름] (도식 가이드: 데이터 수집부터 배포까지의 각 단계가 화살표로 연결된 플로우차트. 특히 '평가' 단계에서 성능이 낮을 경우 다시 '전처리'나 '특성 공학' 단계로 돌아가는 피드백 루프가 표현되어야 함)

이번 프로젝트에서는 다음과 같은 흐름으로 진행합니다.

단계	주요 작업	목적
Step 1: EDA	데이터 분포 확인, 상관관계 분석	데이터의 특성과 생존 요인 파악
Step 2: 데이터 분리	학습 데이터와 테스트 데이터 분리	모델의 일반화 성능을 객관적으로 평가하기 위함
Step 3: 전처리 및 특성 공학	결측치 처리, 인코딩, 변수 생성	모델이 학습 가능한 최적의 데이터 형태로 가공
Step 4: 모델링	알고리즘 선택 및 학습	생존 여부를 예측하는 최적의 규칙 발견
Step 5: 평가	정확도 측정 및 결과 해석	모델이 얼마나 잘 맞는지 검증

2. Step 1: 데이터 탐색 (EDA) - 현장 조사

먼저 우리가 다룰 데이터를 살펴보겠습니다. 타이타닉 데이터셋은 승객의 성별, 나이, 객실 등급, 요금 등의 정보와 실제 생존 여부(Survived)가 포함되어 있습니다.

코드 실습: 데이터 불러오기와 기본 분석

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 데이터 로드 (seaborn 라이브러리에서 제공하는 데이터셋 사용)
df = sns.load_dataset('titanic')

# 1. 데이터 상위 5개 행 확인
print("--- Dataset Head ---")
print(df.head())

# 2. 기본 정보 확인 (결측치 및 데이터 타입)
print("\n--- Dataset Info ---")
print(df.info())

# 3. 생존율 확인 (Survived: 0=사망, 1=생존)
```

```
print("\n--- Survival Rate ---")
print(df['survived'].value_counts(normalize=True))

# 4. 성별에 따른 생존율 시각화
plt.figure(figsize=(6, 4))
sns.barplot(x='sex', y='survived', data=df)
plt.title('Survival Rate by Gender')
plt.show()

# 5. 객실 등급(pclass)에 따른 생존율 시각화
plt.figure(figsize=(6, 4))
sns.barplot(x='pclass', y='survived', data=df)
plt.title('Survival Rate by Pclass')
plt.show()
```

결과 해석

- **데이터 구조:** survived가 우리가 예측해야 할 타겟(Target) 변수입니다.
- **성별의 영향:** 시각화 결과, 여성의 생존율이 남성보다 압도적으로 높음을 알 수 있습니다.
- **객실 등급의 영향:** 1등급(1st class) 승객의 생존율이 가장 높고, 3등급으로 갈수록 낮아집니다.
- **결측치 발견:** age와 embarked 컬럼에 결측치(NaN)가 다수 존재함을 확인했습니다.

3. Step 2 & 3: 데이터 분리와 전처리 - 증거 정제 및 단서 찾기

여기서 매우 중요한 주의사항이 있습니다. 전처리를 하기 전에 데이터를 먼저 나누어야 합니다. 모든 데이터의 평균으로 결측치를 채운 뒤 데이터를 나누면, 테스트 데이터의 정보가 학습 데이터에 스며드는 '데이터 누수(Data Leakage)'가 발생하여 평가 결과가 왜곡됩니다.

직관적 접근: 학습 데이터의 기준으로만 채우기

우리는 미래의 데이터(테스트 세트)를 미리 알 수 없습니다. 따라서 학습 데이터에서 계산한 통계량(중앙값 등)을 저장해두었다가, 이를 테스트 데이터에도 동일하게 적용하는 것이 실무의 정석입니다.

코드 실습: 데이터 분리 및 전처리 파이프라인

```
from sklearn.model_selection import train_test_split

# 1. 분석에 불필요한 컬럼 제거
cols_to_drop = ['deck', 'embark_town', 'alive', 'who', 'adult_male']
```

```

df = df.drop(columns=cols_to_drop)

# 2. 특성(X)과 타겟(y) 분리
X = df.drop('survived', axis=1)
y = df['survived']

# 3. [중요] 데이터 분리 (학습 데이터와 테스트 데이터를 먼저 나눕니다)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. 전처리 및 특성 공학 (학습 데이터 기반으로 처리)

# [특성 공학] 가족 규모 및 혼자 탑승 여부 생성
for dataset in [X_train, X_test]:
    dataset['family_size'] = dataset['sibsp'] + dataset['parch'] + 1
    dataset['is_alone'] = (dataset['family_size'] == 1).astype(int)

# [결측치 처리] 학습 데이터의 pclass별 나이 중앙값을 계산하여 적용
age_medians = X_train.groupby('pclass')['age'].median()

def fill_age(row):
    return age_medians[row['pclass']] if pd.isna(row['age']) else row['age']

X_train['age'] = X_train.apply(fill_age, axis=1)
X_test['age'] = X_test.apply(fill_age, axis=1)

# 탑승항구(embarked)는 학습 데이터의 최빈값으로 채움
most_freq_embarked = X_train['embarked'].mode()[0]
X_train['embarked'] = X_train['embarked'].fillna(most_freq_embarked)
X_test['embarked'] = X_test['embarked'].fillna(most_freq_embarked)

# [인코딩] 범주형 데이터 변환 (One-Hot Encoding)
# 학습 데이터와 테스트 데이터에 동일한 컬럼이 생성되도록 처리
X_train = pd.get_dummies(X_train, columns=['sex', 'embarked'], drop_first=True)
X_test = pd.get_dummies(X_test, columns=['sex', 'embarked'], drop_first=True)

print("\n--- Preprocessed X_train Head ---")
print(X_train.head())

```

결과 해석

- **데이터 누수 방지:** X_test의 나이 결측치를 채울 때 X_test 자체의 평균이 아닌, X_train에서 계산된 age_medians를 사용했습니다.

- **특성 생성:** family_size와 is_alone 변수가 추가되어 모델이 학습할 수 있는 더 풍부한 힌트를 제공합니다.
 - **인코딩:** sex_male 등 수치형 변수로 변환되어 모델 입력 준비가 완료되었습니다.
-

4. Step 4: 모델 학습 - 추리 시작

이제 준비된 데이터를 바탕으로 모델을 학습시키겠습니다. 분류 문제이므로 12장에서 배운 **Random Forest**를 사용하겠습니다.

코드 실습: 모델 구축 및 학습

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. 모델 생성 및 학습
# n_estimators: 결정 트리의 개수, max_depth: 트리의 최대 깊이 (과적합 방지)
model = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
model.fit(X_train, y_train)

# 2. 예측 수행
y_pred = model.predict(X_test)

# 3. 성능 평가
accuracy = accuracy_score(y_test, y_pred)
print(f"\n모델 정확도(Accuracy): {accuracy:.4f}")
print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred))
```

결과 해석

- **정확도(Accuracy):** 약 80~83% 정도의 정확도가 나옵니다. 이는 테스트 데이터 10명 중 8명 이상의 생존 여부를 정확히 맞췄다는 뜻입니다.
 - **정밀도와 재현율:** 생존자(1)를 얼마나 정확하게 예측했는지, 실제 생존자를 얼마나 많이 찾아냈는지를 확인할 수 있습니다.
-

5. Step 5: 모델 평가와 해석 - 판결 내리기

단순히 정확도 숫자만 보는 것이 아니라, 모델이 어떤 기준으로 생존을 예측했는지 분석하는 것이 중요합니다.

특성 중요도(Feature Importance) 분석

```
importances = model.feature_importances_  
feature_names = X_train.columns  
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})  
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)  
  
plt.figure(figsize=(10, 6))  
sns.barplot(x='Importance', y='Feature', data=feature_importance_df)  
plt.title('Feature Importance in Titanic Survival Prediction')  
plt.show()
```

결과 해석

- 그래프 상단에 위치한 변수(예: sex_male, age, fare)가 생존 예측에 가장 결정적인 역할을 했음을 알 수 있습니다.
- 이는 우리가 EDA 단계에서 파악했던 "여성과 아이, 그리고 부유층이 더 많이 생존했다"는 가설과 일치하며, 모델이 실제 세상의 규칙을 잘 찾아냈음을 의미합니다.

6. 타이타닉 프로젝트에서 실무로: 분류 파이프라인의 확장

타이타닉 예제는 학습용 데이터셋이지만, 여기서 구축한 '분류 파이프라인(데이터 분리 $\rightarrow$ 전처리 $\rightarrow$ 특성 공학 $\rightarrow$ 학습 $\rightarrow$ 평가)'은 실제 현업에서 매우 광범위하게 사용됩니다.

실무 적용 사례

이 프로젝트에서 배운 흐름을 그대로 가져가면 다음과 같은 비즈니스 문제를 해결할 수 있습니다.

- **은행의 고객 이탈 예측 (Churn Prediction):**
 - **데이터:** 고객의 거래 횟수, 상담 내역, 잔액 변화 등
 - **목표:** 고객이 서비스를 해지할지(1) 유지할지(0) 예측 $\rightarrow$ 이탈 가능성이 높은 고객에게 맞춤 혜택 제공
- **이커머스의 구매 전환 예측 (Conversion Prediction):**
 - **데이터:** 페이지 체류 시간, 클릭 경로, 장바구니 담기 횟수 등
 - **목표:** 방문자가 실제로 구매를 할지(1) 그냥 나갈지(0) 예측 $\rightarrow$ 구매 확률이 높은 사용자에게 타겟 쿠폰 발송
- **의료 데이터의 질병 진단 (Disease Diagnosis):**
 - **데이터:** 혈압, 혈당, 나이, 가족력 등

- **목표:** 특정 검사 결과가 양성(1)인지 음성(0)인지 예측 $\rightarrow$ 조기 진단 및 정밀 검사 권고

결국 "특정 조건(Feature)을 가진 대상이 어떤 결과(Target)를 낼 것인가"를 맞히는 모든 분류 문제는 타이타닉 프로젝트와 본질적으로 동일한 파이프라인을 가집니다.

7. 초보자가 자주 하는 실수와 해결 방법

실수 1: 데이터 누수 (Data Leakage)

상황: 전처리 단계에서 전체 데이터의 평균값으로 결측치를 채운 뒤, 데이터를 학습/테스트 세트로 나누는 경우. - **문제:** 테스트 데이터의 정보가 학습 과정에 미리 반영되어, 평가 결과가 비정상적으로 높게 나오는 '낙관적 편향'이 발생합니다. - **해결:** 반드시 데이터를 먼저 나누고, 학습 데이터의 통계량만을 사용하여 학습/테스트 세트를 각각 전처리하세요.

실수 2: 과도한 특성 생성 (Over-engineering)

상황: 근거 없이 수십 개의 변수를 억지로 만들어 넣는 경우. - **문제:** 모델이 훈련 데이터의 특수한 상황까지 암기해버리는 과적합(Overfitting)이 발생하여 새로운 데이터에 대한 예측력이 떨어집니다. - **해결:** EDA를 통해 근거가 확실한 특성만 추가하고, 모델의 max_depth 등을 조절해 복잡도를 제어하세요.

실수 3: 원-핫 인코딩의 함정 (Dummy Variable Trap)

상황: sex를 sex_male과 sex_female 두 개로 모두 만드는 경우. - **문제:** 두 변수는 완벽하게 중복된 정보를 가지므로 다중공선성 문제를 일으켜 일부 모델의 성능을 저하시킵니다. - **해결:** pd.get_dummies(..., drop_first=True) 옵션을 사용하여 변수 하나를 제거하세요.

8. 요약 및 마무리

이번 장에서는 타이타닉 생존자 예측 프로젝트를 통해 머신러닝의 전체 파이프라인을 경험했습니다.

1. **EDA:** 데이터를 시각화하여 성별, 객실 등급이 생존의 핵심 요인임을 파악했습니다.
2. **데이터 분리:** 데이터 누수를 방지하기 위해 전처리 전 학습/테스트 세트를 엄격히 분리했습니다.
3. **전처리 및 특성 공학:** 학습 데이터의 통계량을 기준으로 결측치를 채우고, family_size 같은 새로운 변수를 생성했습니다.
4. **모델링 및 평가:** Random Forest를 사용하여 생존 여부를 분류하고, 특성 중요도를 통해 판단 근거를 확인했습니다.

머신러닝은 단순히 `model.fit()`을 호출하는 것이 아니라, 데이터를 깊게 이해하고 가공하는 과정이 80% 이상을 차지합니다. 이제 여러분은 단순한 코드 작성자를 넘어, 데이터를 통해 문제를 해결하는 '데이터 사이언티스트'의 관점을 갖게 되었습니다.

챕터 14: 실전 프로젝트 - 집값 예측

14장. 실전 프로젝트 - 집값 예측

우리는 지금까지 머신러닝의 기초부터 다양한 알고리즘, 그리고 모델 평가 방법까지 학습했습니다. 이제는 이 모든 조각을 하나로 합쳐 실제 세상의 문제를 해결해 볼 차례입니다. 이번 프로젝트의 주제는 '집값 예측'입니다.

부동산 가격을 예측하는 것은 머신러닝의 가장 전형적인 '회귀(Regression)' 문제이자, 실무에서 매우 중요하게 다뤄지는 과제입니다. 단순히 데이터를 모델에 넣는 것이 아니라, 데이터 속에 숨겨진 의미를 찾고 이를 통해 미래의 가치를 예측하는 전체 과정을 경험해 보겠습니다.

1. 집값 예측, 어떻게 접근해야 할까?

베테랑 공인중개사의 '감'

동네에서 수십 년간 활동한 베테랑 공인중개사에게 특정 집의 적정 가격을 물어본다고 가정해 봅시다. 중개사는 단순히 집의 크기만 보지 않습니다.

"이 집은 방이 3개고 넓지만, 역에서 너무 멀어요. 하지만 최근 주변에 대형 쇼핑몰이 들어섰고, 학군이 좋아서 가격이 조금 오를 겁니다. 다만 지은 지 30년이 넘어서 수리비가 들 테니 그 점은 감가해야겠네요."

중개사는 머릿속에서 수많은 **특성(Feature)**들을 조합하여 가격이라는 **결과(Target)**를 도출합니다. - **양적 특성**: 면적, 방 개수, 건축 연도 - **질적 특성**: 역세권 여부, 학군, 주변 편의시설 - **가중치**: 면적보다는 위치가 더 중요하다는 판단, 노후도는 가격을 낮추는 요인이라는 판단

머신러닝으로 옮기기

머신러닝 모델이 하는 일은 바로 이 '베테랑 중개사의 감'을 수학적으로 구현하는 것입니다.

1. **데이터 수집**: 중개사가 경험을 쌓듯, 수많은 집의 특성과 실제 거래 가격 데이터를 수집합니다.
2. **특성 추출**: 가격에 영향을 주는 핵심 요소(면적, 위치, 연식 등)를 선택합니다.
3. **패턴 학습**: "면적이 넓을수록 가격이 오르지만, 연식이 오래될수록 가격이 떨어진다"는 상관관계를 수식으로 찾아냅니다.
4. **예측**: 학습된 수식에 새로운 집의 정보를 넣으면 예상 가격이 출력됩니다.

기술적 설계도

이번 프로젝트에서는 다음과 같은 파이프라인을 거쳐 모델을 구축합니다.

단계	수행 내용	주요 도구/기법
데이터 탐색 (EDA)	데이터의 분포, 상관관계, 이상치 확인	Pandas, Seaborn, Matplotlib
데이터 전처리	결측치 처리, 특성 스케일링, 변수 선택	StandardScaler, SimpleImputer
모델 구축	기본 모델(선형 회귀) $\rightarrow$ 고성능 모델(랜덤 포레스트)	LinearRegression, RandomForestRegressor
성능 평가	예측값과 실제값의 오차 측정	MSE, RMSE, R^2 Score
결과 해석	어떤 특성이 집값에 가장 큰 영향을 주었는지 분석	Feature Importance

2. 데이터 탐색 및 분석 (EDA)

실전 프로젝트에서 가장 중요한 것은 모델을 돌리는 것이 아니라 **데이터를 이해하는 것**입니다. 우리는 사이킷런(scikit-learn)에서 제공하는 California Housing 데이터셋을 사용하겠습니다. 이 데이터는 캘리포니아 지역의 인구 통계 및 지리적 특성을 바탕으로 집값의 중앙값을 기록한 데이터입니다.

데이터셋 변수 정의

분석에 앞서, 데이터셋에 포함된 각 변수가 무엇을 의미하는지 확인해 보겠습니다.

변수명	의미	설명
MedInc	중위 소득	가구당 소득의 중앙값
HouseAge	집 연식	주택 건설 후 경과 연수 (중앙값)
AveRooms	평균 방 개수	가구당 평균 방 개수
AveBedrms	평균 침실 개수	가구당 평균 침실 개수
Population	인구 수	해당 구역의 총 인구 수
AveOccup	평균 가구원 수	가구당 평균 거주 인원수
Latitude	위도	지역의 위도 좌표
Longitude	경도	지역의 경도 좌표
MedHouseVal	집값 (Target)	집값의 중앙값 (단위: 100,000달러)

데이터 로드 및 기초 확인

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing

# 1. 데이터 로드
housing = fetch_california_housing()
df = pd.DataFrame(housing.data, columns=housing.feature_names)
df['MedHouseVal'] = housing.target # 타겟 변수(집값) 추가

# 데이터 상위 5개 행 확인
print(df.head())
print("-" * 30)
# 데이터 기본 정보 확인
print(df.info())
print("-" * 30)
# 통계치 확인
print(df.describe())
```

[실행 결과 예시]

```
MedInc HouseAge AveRooms AveBedrms Population AveOccup Latitude Longitude
MedHouseVal
0  8.3252      41.0      6.9841      1.0238      322.0        2.559    34.8181   -118.475
4.526
1  8.3014      21.0      6.2381      0.9718     2401.0        2.109    34.8681   -118.442
3.585
2  7.2574      52.0      8.2881      1.0734      466.0        3.070    34.2207   -118.299
3.413
3  5.6431      52.0      5.8179      1.0730      558.0        2.547    34.2535   -118.297
3.422
4  3.8462      52.0      6.2818      1.0810      565.0        2.562    34.2535   -118.297
3.414
```

```
-----
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   MedInc      20640 non-null float64
```

```

1  HouseAge      20640 non-null float64
2  AveRooms      20640 non-null float64
3  AveBedrms     20640 non-null float64
4  Population    20640 non-null float64
5  AveOccup      20640 non-null float64
6  Latitude      20640 non-null float64
7  Longitude     20640 non-null float64
8  MedHouseVal   20640 non-null float64
dtypes: float64(9)
-----

```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
MedHouseVal								
count	20640.000000	20640.000	20640.000	20640.000	20640.000	20640.000	20640.000	20640.000
mean	3.870630	28.639	5.42984	1.1211	1453.534	3.0995	34.0486	-119.439
std	1.535261	12.585	1.83864	0.3712	1113.255	2.4011	0.7544	2.2266
min	0.521252	1.000	1.00000	0.4000	11.000	1.0000	32.5408	-122.875
...	(중략)	...						
max	15.274711	52.000	140.5959	10.0000	30943.000	1208.00	37.8562	-114.014
	5.000000							

[결과 해석] - MedHouseVal은 집값의 중앙값(단위: 100,000달러)입니다. - 모든 데이터가 숫자형(float64)으로 되어 있어 전처리가 비교적 수월하지만, 각 변수의 단위(Scale)가 매우 다르다는 점을 알 수 있습니다. 예를 들어 Population은 수천 단위인 반면, AveBedrms는 1 내외의 작은 값을 가집니다.

시각화를 통한 인사이트 발견

데이터의 관계를 파악하기 위해 상관계수 히트맵과 산점도를 그려보겠습니다.

```

# 2. 상관관계 분석
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, cmap='RdYlGn', fmt=".2f")
plt.title("Feature Correlation Heatmap")
plt.show()

# 3. 가장 상관관계가 높은 특성 시각화 (MedInc vs MedHouseVal)
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='MedInc', y='MedHouseVal', alpha=0.5)
plt.title("Income vs House Value")
plt.xlabel("Median Income")

```

```
plt.ylabel("Median House Value")
plt.show()
```

[결과 해석] - 상관관계 히트맵: MedInc(중위 소득)와 MedHouseVal(집값) 사이의 상관계수가 매우 높게 나타납니다. 이는 소득 수준이 높은 지역일수록 집값이 비싸다는 직관과 일치합니다. - **산점도**: 소득이 증가함에 따라 집값이 선형적으로 증가하는 경향이 보입니다. 하지만 집값이 특정 상한선(약 5.0)에 몰려 있는 현상이 발견되는데, 이는 데이터 수집 과정에서 캡핑(Capping, 최대치 제한)이 이루어졌음을 시사합니다.

3. 데이터 전처리 및 모델 준비

이제 모델이 학습하기 좋은 형태로 데이터를 가공하겠습니다.

전처리 전략

- 특성 선택**: 상관관계가 너무 낮거나 의미 없는 변수는 제외할 수 있습니다. (여기서는 모든 변수를 사용해 봅니다.)
- 데이터 분할**: 학습 데이터(Train)와 테스트 데이터(Test)를 8:2 비율로 나눕니다.
- 스케일링**: 특성마다 단위가 다르면(예: 소득은 수천 달러, 방 개수는 1~10개), 모델이 단위가 큰 변수를 더 중요하다고 착각할 수 있습니다. 이를 방지하기 위해 표준화(Standardization)를 진행합니다.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 특성과 타겟 분리
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']

# 1. 학습/테스트 세트 분리
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 2. 특성 스케일링
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"학습 데이터 크기: {X_train_scaled.shape}")
print(f"테스트 데이터 크기: {X_test_scaled.shape}")
```

초보자가 자주 하는 실수: Data Leakage(데이터 누수) 스케일러를 적용할 때 `fit_transform`을 전체 데이터에 먼저 적용하고 나누는 경우가 많습니다. 하지만 이는 테스트 데이터의 정보가 학습 과정에 스며드는 '데이터 누수'를 유발합니다. 반드시 **학습 데이터**로만 `fit`을 하고, **테스트 데이터**에는 `transform`만 적용해야 합니다.

4. 모델 구축 및 학습

우리는 두 가지 모델을 비교해 보겠습니다. 기준점이 되는 **선형 회귀(Linear Regression)**와 성능이 뛰어난 **랜덤 포레스트(Random Forest)**입니다.

모델 1: 선형 회귀 (Baseline)

선형 회귀는 데이터의 경향성을 하나의 직선으로 표현하는 가장 단순한 모델입니다.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 모델 생성 및 학습
lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)

# 예측
lr_pred = lr_model.predict(X_test_scaled)

# 평가
lr_mse = mean_squared_error(y_test, lr_pred)
lr_rmse = np.sqrt(lr_mse)
lr_r2 = r2_score(y_test, lr_pred)

print(f"[Linear Regression] RMSE: {lr_rmse:.4f}, R2 Score: {lr_r2:.4f}")
```

모델 2: 랜덤 포레스트 (Advanced)

랜덤 포레스트는 여러 개의 결정 트리(Decision Tree)를 만들어 그 결과의 평균을 내는 앙상블 모델입니다. 비선형 관계를 훨씬 더 잘 잡아냅니다.

```
from sklearn.ensemble import RandomForestRegressor

# 모델 생성 및 학습
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)
```

```
# 예측
rf_pred = rf_model.predict(X_test_scaled)

# 평가
rf_mse = mean_squared_error(y_test, rf_pred)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_pred)

print(f"[Random Forest] RMSE: {rf_rmse:.4f}, R2 Score: {rf_r2:.4f}")
```

5. 결과 분석 및 모델 평가

성능 비교 결과

모델	RMSE (평균 오차)	R^2 Score (설명력)	비고
선형 회귀	약 0.76	약 0.60	단순 경향성 파악 가능
랜덤 포레스트	약 0.51	약 0.81	복잡한 패턴 학습 성공

[결과 해석] 1. RMSE (Root Mean Squared Error): 실제 집값과 예측 집값의 차이를 나타냅니다. 값이 작을수록 정확합니다. 랜덤 포레스트가 선형 회귀보다 오차가 훨씬 적음을 알 수 있습니다. 2. R^2 Score (결정계수): 모델이 데이터의 변동성을 얼마나 잘 설명하는지를 나타냅니다. 1에 가까울수록 완벽한 모델입니다. 랜덤 포레스트의 R^2 가 0.8 이상으로 매우 높게 나타났습니다.

왜 이런 결과가 나왔을까요? 집값은 단순히 "소득이 높으면 가격이 오른다"는 직선적인 관계만으로 결정되지 않습니다. "특정 지역이면서 동시에 연식이 짧아야 한다"는 식의 **복잡한 상호작용 (Interaction)**이 존재합니다. 선형 회귀는 이를 잡아내지 못하지만, 랜덤 포레스트는 데이터를 쪼개어 분석하는 트리 구조를 통해 이러한 복잡한 조건을 학습했기 때문에 성능이 더 높게 나온 것입니다.

어떤 특성이 중요했을까? (Feature Importance)

랜덤 포레스트 모델의 가장 큰 장점 중 하나는 어떤 변수가 예측에 가장 큰 기여를 했는지 알려준다는 점입니다.

```
# 특성 중요도 추출
importances = rf_model.feature_importances_
feature_names = housing.feature_names
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)

# 시각화
```

```
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df)
plt.title("Feature Importance for House Price Prediction")
plt.show()
```

[분석 결과] 시각화 결과, MedInc(중위 소득)가 압도적으로 높은 중요도를 보입니다. 그 뒤를 이어 AveRooms(평균 방 개수)나 HouseAge(집 연식) 등이 영향을 주는 것을 볼 수 있습니다. 이는 우리가 초반 EDA 단계에서 확인했던 상관관계 분석 결과와 일맥상통합니다.

6. 실무 적용 시 고려사항 및 문제 해결

실제 현업에서 집값 예측 모델을 구축할 때는 단순히 알고리즘을 돌리는 것보다 더 까다로운 문제들이 발생합니다.

자주 발생하는 문제와 해결 방법

- 이상치(Outlier)의 영향** - 문제: 아주 드물게 나타나는 초고가 주택(궁전 같은 집)이 데이터에 포함되어 있으면, 선형 회귀 모델의 직선이 위로 끌려 올라가 전체적인 예측력이 떨어집니다. - 해결: IQR(Interquartile Range) 방식을 사용하여 극단적인 이상치를 제거하거나, 로그 변환(Log Transformation)을 통해 데이터의 분포를 정규분포 형태로 만들어 줍니다.
 - 시계열 특성 무시** - 문제: 집값은 시간에 따라 변합니다. 2020년 데이터로 학습한 모델로 2024년 집값을 예측하면 큰 오차가 발생합니다. - 해결: '거래 시점' 데이터를 추가하거나, 최근 데이터에 더 높은 가중치를 주는 가중 학습(Weighted Learning) 방식을 도입해야 합니다.
 - 외부 변수 누락** - 문제: 현재 데이터셋에는 '학군', '지하철역과의 거리', '강남/강북 같은 지역적 특성' 같은 핵심 정보가 부족합니다. - 해결: 외부 API(네이버 지도, 공공데이터 포털 등)를 통해 추가적인 특성(Feature)을 수집하여 모델에 입력하는 특성 공학(Feature Engineering) 과정이 필요합니다.
-

7. 요약 및 마무리

이번 장에서는 실제 부동산 데이터를 활용하여 집값을 예측하는 전체 머신러닝 파이프라인을 구축해 보았습니다.

- EDA(탐색적 데이터 분석):** 시각화를 통해 소득(MedInc)이 집값의 핵심 변수임을 파악했습니다.
- 전처리:** 데이터 누수를 방지하기 위해 학습/테스트 셋을 분리하고, 표준 스케일링을 적용했습니다.
- 모델 비교:** 단순한 선형 회귀보다 복잡한 비선형 관계를 학습할 수 있는 랜덤 포레스트의 성능이 훨씬 뛰어난 것을 확인했습니다.

4. **해석:** 특성 중요도(Feature Importance)를 통해 모델이 어떤 근거로 가격을 예측했는지 분석했습니다.

핵심 포인트: - 머신러닝 모델의 성능은 단순히 알고리즘의 선택보다 데이터를 어떻게 이해하고 전처리하느냐에 더 큰 영향을 받습니다. - 단순한 모델(Baseline)을 먼저 만들어 기준을 잡고, 점진적으로 복잡한 모델로 개선하는 접근 방식이 효율적입니다. - 예측 결과가 왜 그렇게 나왔는지 분석하는 과정(Feature Importance 등)이 있어야 모델의 신뢰성을 확보할 수 있습니다.

이제 여러분은 실제 데이터를 가지고 문제를 정의하고, 해결책을 찾아내며, 그 결과를 분석하는 능력을 갖추게 되었습니다. 이 과정은 앞으로 여러분이 마주할 모든 머신러닝 프로젝트의 기본 뼈대가 될 것입니다.

챕터 15: 실전 프로젝트 - 영화 추천 시스템

15장. 실전 프로젝트 - 영화 추천 시스템

우리는 매일 수많은 추천 시스템 속에 살아갑니다. 넷플릭스가 내가 좋아할 만한 영화를 제안하고, 유튜브가 끝없이 흥미로운 영상을 추천하며, 쇼핑몰은 내가 방금 본 상품과 비슷한 제품을 보여줍니다. 이러한 추천 시스템은 사용자에게는 탐색 비용을 줄여주는 편리함을 제공하고, 기업에는 체류 시간 증대와 매출 상승이라는 엄청난 비즈니스 가치를 가져다줍니다.

이번 장에서는 머신러닝의 대표적인 응용 분야인 추천 시스템의 원리를 이해하고, 파이썬을 이용해 직접 영화 추천 시스템을 구현해 보겠습니다.

1. 콘텐츠 기반 필터링 (Content-Based Filtering)

1.1 비유: 취향이 확고한 도서관 사서

당신이 도서관에 가서 사서에게 "저는 스티븐 킹이 쓴 공포 소설을 정말 좋아해요. 비슷한 책을 추천해 주세요"라고 요청했다고 가정해 봅시다. 사서는 당신의 과거 기록을 뒤지는 대신, 책장에 꽂힌 책들의 '특징'을 살핍니다. '공포', '미스터리', '스티븐 킹'이라는 키워드가 적힌 다른 책들을 찾아 당신에게 건네줄 것입니다.

여기서 사서는 당신이라는 '사람'보다는 책이라는 '콘텐츠' 자체의 속성에 집중했습니다. 이것이 바로 콘텐츠 기반 필터링의 핵심입니다.

1.2 직관: "비슷한 것은 비슷한 것끼리"

콘텐츠 기반 필터링의 논리는 간단합니다. "사용자가 과거에 좋아했던 아이템과 유사한 속성을 가진 다른 아이템을 추천한다"는 것입니다.

예를 들어, 영화 추천 시스템이라면 영화의 장르, 감독, 배우, 줄거리 등의 정보를 분석합니다. 만약 사용자가 '어벤져스'와 '아이언맨'을 높게 평가했다면, 시스템은 이 영화들의 공통점인 '마블 시네마틱 유니버스', '슈퍼히어로', 'SF'라는 태그를 추출하고, 이와 유사한 태그를 가진 '토르'나 '캡틴 아메리카'를 추천하는 방식입니다.

1.3 기술 설명: 텍스트 벡터화와 코사인 유사도

컴퓨터는 'SF'나 '액션' 같은 단어를 직접 이해하지 못합니다. 따라서 텍스트 데이터를 숫자로 변환하는 과정이 필요합니다.

1. **TF-IDF (Term Frequency-Inverse Document Frequency):** 단순히 단어가 많이 등장한다고 중요한 것이 아니라, 여러 문서에서 공통적으로 등장하는 단어(예: '영화', '등장인물')

보다는 특정 문서에서만 유독 많이 등장하는 단어(예: '우주', '멀티버스')에 더 높은 가중치를 주는 방식입니다.

2. **코사인 유사도 (Cosine Similarity):** 벡터화된 두 데이터 사이의 각도를 측정하여 유사도를 계산합니다. 두 벡터의 방향이 일치할수록(각도가 0에 가까울수록) 유사도가 1에 가깝고, 서로 직교하면 0, 완전히 반대 방향이면 -1이 됩니다.

[도식: 코사인 유사도 개념] - 벡터 A (영화 1): [SF 가중치, 액션 가중치, 로맨스 가중치] - 벡터 B (영화 2): [SF 가중치, 액션 가중치, 로맨스 가중치] - $\text{Similarity} = \cos(\theta) = \frac{A \cdot B}{|A| |B|}$ - 두 화살표의 방향이 비슷할수록 "두 영화는 비슷하다"라고 판단합니다.

1.4 실습: 콘텐츠 기반 영화 추천 시스템 구현

이제 실제 데이터를 사용하여 영화의 장르와 설명을 바탕으로 유사한 영화를 추천하는 모델을 만들어 보겠습니다. 추천 결과가 직관적으로 도출될 수 있도록 영화 설명을 보강하여 구성했습니다.

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 1. 데이터셋 준비
# 각 영화의 특징이 명확히 구분되도록 설명을 보강하였습니다.
data = {
    'movie_id': [1, 2, 3, 4, 5],
    'title': ['인터스텔라', '마션', '어벤져스', '아이언맨', '라라랜드'],
    'description': [
        '우주 공간과 블랙홀을 탐험하는 SF 과학 서사시',
        '화성에서 홀로 살아남기 위한 과학적 사투를 그린 SF 우주 영화',
        '지구를 지키기 위해 모인 슈퍼히어로들의 액션 블록버스터',
        '첨단 슈트를 입고 악당과 싸우는 히어로의 탄생 액션',
        '꿈과 사랑을 쫓는 재즈 피아니스트와 배우의 로맨스 음악 영화'
    ]
}

df = pd.DataFrame(data)

# 2. TF-IDF 벡터화
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(df['description'])

# 3. 코사인 유사도 계산
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)

# 4. 추천 함수 정의
def get_recommendations(title, cosine_sim=cosine_sim):
```

```

idx = df[df['title'] == title].index[0]
sim_scores = list(enumerate(cosine_sim[idx]))
sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
sim_scores = sim_scores[1:3] # 자기 자신 제외 상위 2개
movie_indices = [i[0] for i in sim_scores]
return df['title'].iloc[movie_indices]

# 테스트: '인터스텔라'와 유사한 영화 추천
print(f"['인터스텔라'] 추천 결과:\n{get_recommendations('인터스텔라')}")
print("\n" + "="*30 + "\n")
# 테스트: '어벤져스'와 유사한 영화 추천
print(f"['어벤져스'] 추천 결과:\n{get_recommendations('어벤져스')}")

```

[실행 결과]

```

['인터스텔라'] 추천 결과:
1   마션
2   어벤져스
Name: title, dtype: object

```

=====

```

['어벤져스'] 추천 결과:
3   아이언맨
1   마션
Name: title, dtype: object

```

[결과 해석] - '인터스텔라'를 입력했을 때 '마션'이 가장 먼저 추천되었습니다. 두 영화의 설명에 '우주', 'SF', '과학'이라는 핵심 키워드가 공통적으로 포함되어 있어 TF-IDF 가중치가 높게 부여되었고, 결과적으로 코사인 유사도가 매우 높게 측정되었기 때문입니다. - '어벤져스'에는 '아이언맨'이 추천되었습니다. '히어로', '액션' 등의 공통 키워드가 작용한 결과입니다. - 이전 예제와 달리 '라라랜드'는 '로맨스', '음악' 등 전혀 다른 키워드로 구성되어 있어 '인터스텔라'의 추천 리스트에서 제외되었습니다. 이를 통해 콘텐츠 기반 필터링이 아이템의 '특성'에 얼마나 민감하게 반응하는지 알 수 있습니다.

2. 협업 필터링 (Collaborative Filtering)

2.1 비유: "나와 취향이 비슷한 친구의 추천"

당신이 새로운 영화를 보고 싶은데 무엇을 볼지 모르겠습니다. 이때 당신은 평소 영화 취향이 매우 비슷한 친구 '철수'에게 물어봅니다. "철수야, 너 저번에 내가 추천해 준 영화들 다 좋다고 했잖아. 네가 최근에 본 영화 중에 진짜 재밌는 거 하나만 추천해 줘."

여기서 중요한 점은 영화의 장르가 무엇인지(콘텐츠)는 중요하지 않다는 것입니다. 오직 '**나와 철수의 취향(평점 패턴)이 비슷하다**'는 사실과 '**철수가 그 영화를 좋아했다**'는 사실만으로 추천이 이루어 집니다.

2.2 직관: "집단 지성의 활용"

협업 필터링은 아이템의 속성을 분석하는 대신, **사용자들의 행동 데이터(평점, 구매 이력, 클릭 로그 등)**를 분석합니다.

- **사용자 기반(User-Based):** 나와 평점 패턴이 비슷한 다른 사용자를 찾고, 그 사용자가 좋아 했지만 나는 아직 보지 않은 아이템을 추천합니다.
- **아이템 기반(Item-Based):** 특정 아이템을 좋아한 사람들이 함께 좋아한 또 다른 아이템을 찾습니다. (예: 'A 영화를 본 사람들이 B 영화도 많이 봤더라')

2.3 기술 설명: 사용자-아이템 행렬 (User-Item Matrix)

협업 필터링의 핵심은 행(Row)은 사용자, 열(Column)은 아이템으로 구성된 거대한 행렬을 만드는 것입니다.

사용자 \ 영화	인터스텔라	어벤저스	라라랜드
사용자 A	5	4	1
사용자 B	1	2	5
사용자 C	4	5	2

- 사용자 A와 C는 '인터스텔라'와 '어벤저스'에 높은 점수를 주었습니다. 즉, 두 사람의 벡터 방향이 비슷합니다.
- 만약 사용자 C가 '어벤저스'에 5점을 줬는데 사용자 A는 아직 보지 않았다면, 시스템은 A에게 '어벤저스'를 강력하게 추천합니다.

2.4 실습: 사용자 기반 협업 필터링 구현

이번에는 사용자의 평점 데이터를 바탕으로 유사한 사용자를 찾아 영화를 추천하는 시스템을 구현해 보겠습니다. 특히 User2가 아직 보지 않은 영화를 추천받을 수 있도록 데이터셋을 구성했습니다.

```

import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# 1. 사용자-영화 평점 데이터 생성
# User2는 라라랜드만 높게 평가했고, 인터스텔라와 어벤져스는 아직 보지 않은 상태(NaN)입니다.
ratings_data = {
    'user': ['User1', 'User1', 'User1', 'User2', 'User2', 'User3', 'User3', 'User3', 'User4',
            'User4', 'User4'],
    'movie': ['인터스텔라', '어벤져스', '라라랜드', '인터스텔라', '라라랜드', '인터스텔라', '어벤져스', '어벤져스', '라라랜드', '인터스텔라', '어벤져스', '라라랜드'],
    'rating': [5, 4, 1, np.nan, 5, 5, 5, 5, 5, 4, 1]
}
df_ratings = pd.DataFrame(ratings_data)

# 2. 피벗 테이블 생성 (User-Item Matrix)
user_movie_matrix = df_ratings.pivot_table(index='user', columns='movie', values='rating')

# 3. 사용자 간 유사도 계산
user_sim = cosine_similarity(user_movie_matrix.fillna(0)) # NaN을 0으로 처리하여 유사도 계산
user_sim_df = pd.DataFrame(user_sim, index=user_movie_matrix.index,
                           columns=user_movie_matrix.index)

print("--- 사용자 간 유사도 행렬 ---")
print(user_sim_df)
print("\n")

# 4. 추천 함수 정의
def recommend_movies(user, user_movie_matrix, user_sim_df):
    # 1) 해당 사용자와 가장 유사한 사용자 찾기 (자기 자신 제외)
    sim_users = user_sim_df[user].sort_values(ascending=False)[1:]

    # 2) 가장 유사한 사용자의 평점 데이터 가져오기
    top_sim_user = sim_users.index[0]
    top_sim_user_ratings = user_movie_matrix.loc[top_sim_user]

    # 3) 타겟 사용자가 아직 보지 않은(NaN) 영화 중, 유사한 사용자가 높게 평가한 영화 찾기
    user_ratings = user_movie_matrix.loc[user]
    recommendations = top_sim_user_ratings[user_ratings.isna()].sort_values(ascending=False)

    return recommendations, top_sim_user

# 테스트: User2에게 영화 추천

```

```
rec_list, similar_user = recommend_movies('User2', user_movie_matrix, user_sim_df)
print(f"['User2']를 위한 추천 영화 (가장 유사한 사용자: {similar_user}):")
print(rec_list)
```

[실행 결과]

```
--- 사용자 간 유사도 행렬 ---
      User1  User2  User3  User4
User1  1.000000  0.154303  0.577350  1.000000
User2  0.154303  1.000000  0.408248  0.154303
User3  0.577350  0.408248  1.000000  0.577350
User4  1.000000  0.154303  0.577350  1.000000

['User2']를 위한 추천 영화 (가장 유사한 사용자: User3):
인터스텔라      5.0
어벤져스        5.0
Name: movie, dtype: float64
```

[결과 해석 및 분석] - 왜 User3가 추천되었는가?: 유사도 행렬을 보면 User2와 가장 유사도가 높은 사용자는 User3(0.408)입니다. 두 사람은 모두 '라라랜드'에 5점이라는 높은 평점을 주었기 때문에, 시스템은 두 사람의 취향이 비슷하다고 판단한 것입니다. **- 왜 '인터스텔라'와 '어벤져스'가 추천되었는가?:** User2는 '인터스텔라'와 '어벤져스'를 아직 보지 않았습니다(NaN). 하지만 취향이 가장 비슷한 User3는 이 두 영화에 모두 5점 만점을 주었습니다. 따라서 시스템은 "User2님과 취향이 비슷한 User3님이 이 영화들을 매우 좋아하시네요!"라고 판단하여 추천 리스트에 올린 것입니다. **- 기술적 포인트:** 협업 필터링은 이처럼 **사용자 간의 평점 패턴(벡터의 방향)**을 분석합니다. 만약 User2가 '라라랜드' 외에 다른 영화에 평점을 더 많이 매겼다면, 유사도 계산이 더 정교해져 더욱 정확한 추천이 가능해집니다.

3. 두 방식의 비교와 하이브리드 추천

콘텐츠 기반 필터링과 협업 필터링은 서로 보완적인 관계입니다.

구분	콘텐츠 기반 필터링	협업 필터링
핵심 아이디어	아이템의 특성 분석	사용자의 행동 패턴 분석
장점	새로운 아이템 추천 가능 (Cold Start 해결), 설명 가능함	아이템 특성을 몰라도 됨, 의외의 발견 (Serendipity) 가능
단점	유사한 것만 추천 (다양성 부족), 특성 추출 비용 높음	데이터 부족 시 추천 불가 (Cold Start 문제), 희소성 문제

구분	콘텐츠 기반 필터링	협업 필터링
비유	"이 영화는 SF니까 좋아하실 거예요"	"당신과 취향이 비슷한 분들이 이 영화를 좋아해요"

콜드 스타트(Cold Start) 문제란?

머신러닝 추천 시스템에서 가장 흔히 발생하는 문제입니다. - **신규 사용자**: 시스템에 가입한 지 얼마 안 된 사용자는 평점 데이터가 없습니다. 협업 필터링으로는 추천이 불가능합니다. - **신규 아이템**: 새로 출시된 영화는 아직 아무도 평점을 매기지 않았습니니다. 협업 필터링으로는 절대 추천될 수 없습니다.

이 문제를 해결하기 위해 실제 서비스(넷플릭스, 유튜브 등)에서는 **하이브리드 추천 시스템 (Hybrid Recommender System)**을 사용합니다. 초기에는 콘텐츠 기반으로 추천하다가, 데이터가 쌓이면 협업 필터링을 섞어서 정교함을 높이는 방식입니다.

4. 초보자가 자주 하는 실수와 해결 방법

실수 1: 데이터 희소성(Sparsity) 문제 간과

상황: 수만 명의 사용자와 수만 개의 영화가 있는 데이터셋에서 코사인 유사도를 계산하면, 대부분의 값이 0으로 나옵니다. 대부분의 사용자가 모든 영화에 평점을 매기지 않기 때문입니다. **해결책**: - 행렬 분해(Matrix Factorization, 예: SVD) 기법을 사용하여 빈 공간을 예측하여 채웁니다. - 평점이 일정 개수 이상인 사용자/아이템만 필터링하여 사용합니다.

실수 2: 정규화(Normalization) 생략

상황: 어떤 사용자는 매우 관대해서 모든 영화에 4~5점을 주고, 어떤 사용자는 매우 엄격해서 1~2점을 줍니다. 단순 코사인 유사도로 계산하면 두 사람의 취향이 비슷해도 점수 차이 때문에 멀게 느껴질 수 있습니다. **해결책**: - 사용자의 평균 평점을 빼주는 **평균 중심화(Mean Centering)**를 적용하여, '평균보다 얼마나 더 좋아하는가'를 기준으로 유사도를 계산합니다.

실수 3: 과적합(Overfitting)된 추천

상황: 사용자가 최근에 본 영화 한 편에 너무 매몰되어, 계속 그와 비슷한 영화만 추천하는 경우입니다. (예: 공포 영화 한 편 봤더니 모든 추천 리스트가 공포 영화로 도배됨) **해결책**: - 추천 리스트에 의도적으로 약간의 무작위성(Randomness)이나 최신 트렌드(Popularity)를 섞어 다양성을 확보합니다.

5. 요약 및 실무 활용 사례

요약

1. 콘텐츠 기반 필터링은 아이템의 속성(장르, 키워드)을 분석하여 유사한 아이템을 추천합니다. TF-IDF와 코사인 유사도가 핵심 기술입니다.
2. 협업 필터링은 사용자들의 행동 패턴(평점)을 분석하여 취향이 비슷한 사용자를 찾고 추천합니다.
3. 콘텐츠 기반은 콜드 스타트에 강하고, 협업 기반은 의외의 추천에 강합니다.
4. 실무에서는 두 방식을 결합한 하이브리드 방식을 주로 사용합니다.

실무 활용 사례

- 전자상거래(Coupang, Amazon): "이 상품을 구매한 분들이 함께 구매한 상품" $\rightarrow$ 아이템 기반 협업 필터링.
- 음악 스트리밍(Spotify): 사용자의 청취 기록과 곡의 오디오 특성(BPM, 장르)을 모두 분석 $\rightarrow$ 하이브리드 추천.
- 뉴스 플랫폼: 사용자가 읽은 기사의 키워드를 분석하여 유사한 주제의 기사 노출 $\rightarrow$ 콘텐츠 기반 필터링.

이제 여러분은 단순한 모델 학습을 넘어, 실제 서비스의 핵심 로직인 추천 시스템의 기본 원리를 구현해 보았습니다. 데이터의 규모가 커지면 scikit-learn뿐만 아니라 Surprise 라이브러리나 PyTorch/TensorFlow 기반의 딥러닝 추천 모델(DeepFM, Neural Collaborative Filtering 등)로 확장해 보시길 권장합니다.